How *Token Economics* cuts what coding agents cost

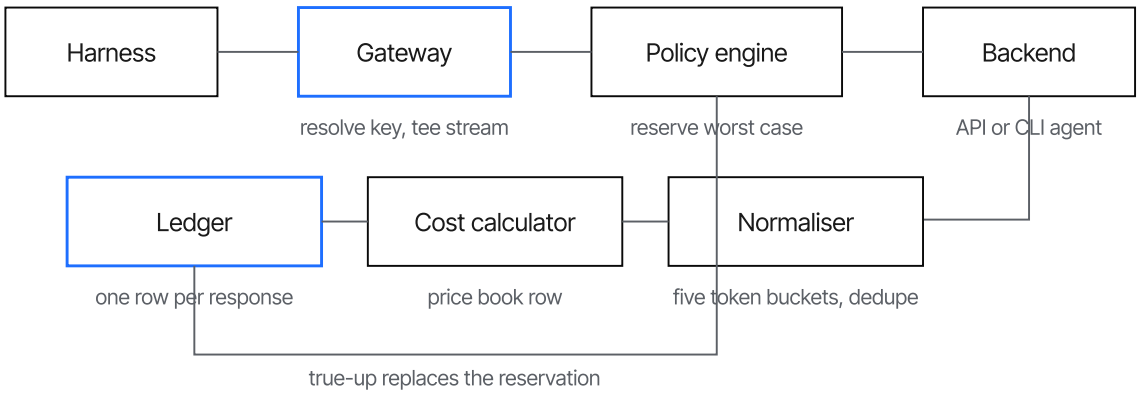
Token Economics is a control plane that sits between coding harnesses (Claude Code, Codex CLI, Aider, SDK scripts) and any number of model backends, API or CLI. It measures cost per successful task, enforces budgets outside the agent, and turns on each saving only after a paired test shows it works.

`Based on the plan in plan/. 423 sources, 8 stages, 79 components, 233 requirements.`

Build in this order. Measure for four weeks before you block anything. Enforce budgets next. Optimise only once a 30 day baseline exists. Harden for production last. Without a baseline you cannot tell whether a saving is real.

One request, end to end

Every harness reaches Token Economics through a gateway URL. The gateway is the source of truth for cost. Hooks, OpenTelemetry and local transcripts add attribution detail but never carry enforcement, because a developer can switch them off.



The ledger is the one place cost lives. Dashboards, the reconciler and the eval harness are queries over it.

The policy engine reserves the worst-case cost before dispatch and replaces it with the true cost once the response finishes. That makes a budget a hard cap at the gateway instead of a request the agent may ignore.

The eight stages

The full diagram is `token-economics-lld.html` with a PDF twin, `token-economics-lld.pdf`. It drills from this overview to a spec card for each of the 79 components.

Stage	Job	Contains
Ingress and adapters	Entry, per-harness control	Gateway edge, harness adapters, client signals
Backends	One interface over API and CLI backends	API providers, CLI backends, backend health
Accounting	The truth about cost	Price book, metering, reconciliation
Policy engine	Decisions before dispatch	Budgets, run control, routing, tier and cache
Context guard	Smaller prompts	Schema deferral, output caps, masking, clearing
Insight	People-facing views	Stores, attribution, dashboards, alerts
Eval and rollout	Proof and safety	Paired tests, shadow, canary, kill-switches
Control plane	Operators	Admin API, config, security and data

Make it pluggable with an adapter per harness

Each harness exposes different hooks, so Token Economics wraps each one in an adapter. An adapter has five functions.

- `route` points the harness at the gateway.
- `identify` returns tenant, team, session and prompt ids.
- `observe` returns one normalised usage event.
- `cap` turns a policy into the controls that harness supports.
- `replay` rebuilds usage from local transcripts.

Each adapter also declares eight capability flags. When a flag is false, the policy engine downgrades that control to observation and says so.

Flag	Meaning
`baseUrl`	The harness can be pointed at a gateway. This is the only flag that makes enforcement possible.
`oauthPassthrough`	Subscription tokens can be forwarded without storing them.
`hooksBlocking`	A hook can block a call. Developers can disable hooks, so this is weak.
`otel`	The harness emits OpenTelemetry metrics and events.
`budgetFlag`, `headless`	The CLI accepts a budget or turn flag and runs without a terminal.
`modelRewrite`	The gateway may change the model id in flight.
`transcriptPath`	Local transcripts exist for replay.

Never mark a harness with `baseUrl = false` as enforced. Pin the route in managed settings where the harness allows it, so a developer cannot bypass the gateway.

Count tokens in five buckets before pricing anything

Providers report usage in incompatible shapes. Token Economics converts each response into five mutually exclusive buckets: `fresh_input`, `cache_read`, `cache_write_5m`, `cache_write_1h` and `output`. Reasoning tokens sit inside `output` and are never added a second time.

Provider	What the input field means	Conversion
Anthropic	`input_tokens` excludes cache	Total input is fresh plus read plus creation.
OpenAI, xAI, OpenRouter	`prompt_tokens` includes cached tokens	Fresh is prompt minus cached. 17,903 prompt with 17,817 cached leaves 86 fresh.
Bedrock Converse	Cache fields reported beside `inputTokens`	Total is the sum of the three.

The ledger then enforces nine rules. The four that bite first:

1. One row per provider response id. Parallel tool calls that share a message id count once.
2. Subagent spend gets its own row, because the parent result excludes it.

3. Use exact model ids. Partial-name matching produced a silent wrong price in one tool.
4. Failed and retried attempts are spend. Failed runs were 17 percent of tasks and 32 percent of spend in one study.

Prices live in an append-only price book. Every ledger row stores a `price_row_id`, a new price never rewrites old rows, and a weekly canary compares computed cost with the invoice.

Control spend at the gateway

The policy engine decides before each dispatch. The controls, in the order you should build them:

Control	How it works	Phase
Hard budgets	Reserve worst case, true-up after, answer 402 or 429 when the budget is gone. Alerts at 75, 90 and 100 percent.	P1
Turn cap	Set at the 75th percentile of each model's own baseline, with a remaining-turns reminder.	P1
Loop guard	Detect the first repeated-failure signature, redirect once, then stop.	P1
Cache planner	Choose the cache lifetime, or time to live (TTL) from the observed reuse gap. Watch cache hit per caller.	P2
Router	Rules first, then a static router or cascade only if it beats the best single model after its own overhead and cache loss.	P2
Strong-model ceiling	Cap each tenant's share of strong-model calls. An adversarial prefix pushed one router to the strong model 73 to 100 percent of the time.	P3

Cut tokens the model never needed

In one study of 2,848 runs, 74.7 percent of billed cost was the harness base prompt and tool definitions. Tool outputs were 3.3 percent. That ranking decides which controls come first.

Control	Default Token Economics ships
Schema deferral	On above about 20 tools, with a lookup path so the model can fetch a schema it needs.

Output cap	25,000 tokens interactive, 12,000 for Codex, 6,000 batch. Never truncate tracebacks or edit anchors.
Observation masking	Only above about 10 events. At 7 or fewer it lost money.
Clearing	Trigger at 30 to 40k tokens, keep 5 results, clear at least 3,000 to 5,000.
Compaction	Last resort. Recommend a restart at the third compaction.
`max_tokens`	Hard cap on every call, since effort settings do not cap tokens.
Output compressors	Off. One study cut tool-output tokens 38.4 percent and raised cost 6.8 percent.

History edits break the exact-prefix cache, so edit history only at a trigger, never every turn. A timestamp at the top of a prompt dropped cache reuse from 87 to 3 percent in one report.

Ship an optimisation only after a paired test

The headline metric is cost per successful task (CPS). It divides all attempt cost, failed runs included, by the number of successes. Token counts mislead. 106 of 336 price comparisons reversed the sticker-price ranking, and token reduction correlated with cost reduction at r = 0.154.

1. Run each task on both arms with several repeats, and pair the results by task.
2. Declare the non-inferiority bound for success before the test starts.
3. Isolate the cache between arms. The median gap between arms in one study was 10.8 seconds, inside the 5 minute TTL.
4. Charge router and classifier cost to the arm that uses them.
5. Use a private task set with hard cases. Public sets saturate at 96 to 98 percent.
6. Roll out as shadow, then a per-tenant canary, with a kill-switch per optimisation and automatic rollback.

Detect waste on its own, apply a fix only after a test

Token Economics finds waste automatically and does not fix everything on day 1. It reads detectors off the ledger, measures each finding, and applies a fix by class. Anything that

changes what the model sees waits for a paired win on that harness, because savings are harness-specific. The Headroom compressor raised cost 48.4 percent in one setting and cut it 12.49 percent in another (C:024).

The metric list comes from `anvesx/hermes-engine`, a Claude Code plugin with 50 ranked leak rows and 103 detectors. It only observes. It never caps spend or blocks a tool, and it reads CLI transcripts that API traffic does not have. Its costs are list-price estimates that overlap, and no saving from any fix is measured. Token Economics uses it as a detector library, not as a forecast.



Detection makes a fix eligible. It never switches the fix on.

Fixes fall into four classes.

Class	What it does	Examples	When it applies
0 Alert	Changes nothing	Cache hit below baseline, unused tool schemas, failure loops	Always, from P0
1 Zero-risk guard	Bounds a worst case	`max_tokens` cap, stale-price block, budget alerts	On detection, from P1, shadow-logged first
2 Behaviour change	Changes what the model sees or which model runs	Schema deferral, output cap, masking, clearing, routing, effort cap, loop redirect	After a paired cost-per-success win for that harness and version, then canary
3 Human action	Needs a person	Disable an unused server, trim `CLAUDE.md`, retire stale skills	Never automatic. Token Economics files the recommendation with the estimated saving

Metrics, detectors and open-source options

The 50 hermes rows fold into 13 families. Open-source names beyond the reuse table are DevX proposals that need a P0 spike. Full table with row ids: `plan/11-detect-measure-apply.md`.

Family	What Token Economics detects	Runs from	Open-source option	Class
Cache	Hit rate below the caller's own baseline, writes never read, rewrite cause	Gateway	LiteLLM cache injection, Langfuse	0, then 2
Starting context, tool schemas	Large first request, tools sent and never called	Gateway	Deferred tool loading with a lookup path	0, 2, 3
History growth	Context over a size line, stale results, compaction count	Gateway, hook	Harness clearing and masking	0, then 2
Tool output	Oversized results, repeated reads, dependency folders	Gateway	Hook output cap. Compressors stay off.	0, then 2
Retries and loops	Same call failing in a row, spend after first repeat	Gateway	Loop guard built in Token Economics	0, then 2
Agents	Helper context, sibling re-reads, long reports	Hook	Subagent settings per harness	0, 3
Model fit	Strong-model share on low-difficulty tasks	Gateway	LiteLLM router, RouteLLM	0, then 2
Output and thinking	Output beyond visible text, truncation	Gateway	`max_tokens`, effort cap	1, 2
Images and PDFs	Image tokens, repeat screenshots	Gateway	Pillow, `pdftotext`	0, then 2
Web and retrieval	Repeated fetches, overlapping results	Gateway	Response cache, only after a test	0
Instructions and skills	Oversized instruction files, stale skills	Hook	A linter in Token Economics's CLI	0, 3
Idle and repeat work	Turns with no typed prompt, repeated first requests	Hook	Memory or handoff file	0, 3
Cost accounting	Client estimate against invoice	Gateway	ccusage, Langfuse	0

- **API agents.** API traffic carries no session id, so Token Economics builds its own conversation key from a prefix hash and idle gap.
- **CLI agents.** Token Economics joins gateway rows to hook and OpenTelemetry rows on `session_id` and `prompt_id`. A harness with no `baseUrl` flag gets alerts only.
- **Gap closed.** Hermes cannot measure 29 rows. Token Economics's position on the request path should cover 12 of them: 9 from request bodies and model-call logs, 3 from

its eval harness. The other 17 need a labelled review and stay estimates.

Day 1 ships the detectors and Class 0 alerts. P1 adds Class 1 guards. P2 runs the paired test for each Class 2 fix. P3 adds automatic canary and rollback drills. Thresholds are placeholders until P0 measures them, because hermes gives none a basis.

The 50 problems and how to fix each one

Every row below is one of the 50 token-leak rows in hermes-engine. The table gives the fix for API agents, which Token Economics applies at the gateway, and for CLI agents, which Token Economics applies through hooks, settings and flags. Named techniques carry source ids (`A:nnn` API corpus, `C:nnn` CLI corpus). A fix with no corpus source says DevX proposal and waits for a P0 spike.

- **Classes.** 1 row is an alert, 3 are zero-risk guards, 33 change behaviour and 13 need a human action.
- **Evidence.** The API fix rests on a DevX proposal in 21 rows and the CLI fix in 8. Techniques were read by model agents. All 423 sources are covered in `12b-source-coverage.md`: 86 cited here, 112 papers read in full text, and 225 vendor docs, blogs and pages read at abstract and key-section depth. Each name and number is only as reliable as that read.
- **Conflict.** C:024 puts conversation history under 1 percent of billed cost. Hermes reports 10.5 percent for row 6. Token Economics measures billed cost and decides.

Terms: Model Context Protocol (MCP) is the tool-server standard. Time to live (TTL) is how long a cache entry lasts. Cost per successful task (CPS) is Token Economics's main metric. Language Server Protocol (LSP) gives code navigation.

Retrieval-augmented generation (RAG) feeds retrieved text to a model. Environment variables that start with Claude Code (CLAUDE) belong to Claude Code. RTK is a command-line proxy tool (RTK) named in C:024, and `RTK-ML` is a variant of it. The source does not expand either name. Software Engineering (SWE) bench tasks, Generative Pre-trained Transformer (GPT) models and the Tool Attention paper's intent-schema overlap (ISO) score appear as the sources name them.

Key-value (KV) is the attention cache a model keeps per token. Small language model (SLM) and language model (LM) name model size. Reinforcement learning (RL), group relative policy optimization (GRPO) and supervised fine-tuning (SFT) are training methods. Total cost of ownership (TCO) is lifetime cost. JSON Lines (JSONL) is a file format with one JSON record per line.

Humanity's Last Exam (HLE) and Massive Multitask Language Understanding (MMLU) appear in source titles. Named-entity recognition (NER) and time to first token (TTFT) appear as the sources use them. United States dollar (USD) is the currency unit.

#	Problem	Class	Phase	Fix for API agents	Fix for CLI agents	Named techniques
1	Heavy starting context	2 Behaviour change	P2	Cache the stable prefix with cache_control on the last tool and system block, and defer rarely used tools behind a lookup path (compressed MCP proxy). Class 0 alert first, then class 2.	Keep deferred tool loading on (ENABLE_TOOL_SEARCH auto/default), `CLAUDE.md` under 200 lines, disable-model-invocation on rare skills; remove unused MCP servers (class 3).	Deferred tool loading (tool search) (C:079, C:019); mcp-compressor (MCP proxy with get_tool_schema / invoke_tool) (C:022); Tool Attention (ISO score plus lazy schema loader) (C:021)
2	Cache-write premium	2 Behaviour change	P2	Use 1h only where reuse gaps fall between 5 minutes and an hour; drop breakpoints nobody reuses. Class 0 alert, TTL change is class 2.	Set promptCacheTtl / CLAUDE_CODE_PROMPT_CACHE_TTL for main, subagentPromptCacheTtl for helpers (v2.1.242+); FORCE_PROMPT_CACHING_5M=1 or ENABLE_PROMPT_CACHING_1H=1.	`TTL-tier` matching (5m vs 1h by reuse gap) (A:199, C:052); Targeted 1h write on the dispatch turn (C:052); Per-caller by TTL hit-rate logging (C:052)
3	Cold cache at session and agent start	2 Behaviour change	P2	Keep the prefix byte-stable across sessions (static head, volatile content after it) and pre-warm shared prefixes. Class 2.	Use forks (inherit and hit) instead of fresh subagents where possible; keep startup git status stable; sessions share cache only by machine and directory.	Pre-warming with max_tokens: 0 (A:199); Persistent per-type static prefix (C:052); Fork and fan-out hold (C:055)
4	Cache expired after a break	2 Behaviour change	P2	Choose TTL by gap histogram; keepalive only if the gap fits the crossover. Class 0 alert, TTL change class 2.	promptCacheTtl, idleCompaction=false (v2.1.290+); compact at natural breaks (a cold /compact reprocesses history uncached).	Keepalive crossover rule (C:052); TTL settings and idleCompaction (C:055); 1h block for long gaps (A:199)
5	Many agents	2 Behaviour change	P2	Gateway cap on concurrent derived conversations per parent; route helper-shaped requests to a cheaper model (orchestrator pattern, cheap workers). DevX proposal for the gateway rule.	Set CLAUDE_CODE_SUBAGENT_MODEL or model: haiku on helper definitions; Codex agents.default_subagent_model, reasoning_effort, agents.max_concurrent_threads_per_session and rollout_budget.	Orchestrator pattern with cheap workers and fan-out budget cap (C:012); Subagent model routing and concurrency settings (C:072, C:052); Heavier harness with subagents inflates cost (A:051)
6	Long history replay	2 Behaviour change	P2	Add `context_management` edits: tool result clearing first (trigger 30-40k, keep 5, clear_at_least 3-5k), compaction as last resort near the window. Both are Class 2, applied only after a paired CPS win.	Claude Code: let auto-compact run, steer it with Compact Instructions in `CLAUDE.md` or /compact focus; Codex: set model_auto_compact_token_limit and experimental_compact_prompt_file; start a new thread between phases with a handoff file.	Production clearing settings (trigger 30-40k, keep 5, clear_at_least 3,000-5,000) (C:066, C:065); Server-side compaction (compact_20260112) (A:271, C:065); Compact within a phase, new thread between phases (handoff file) (C:035, C:112)

7	Interrupted turns	0 Alert	P0	No corpus source; DevX proposal: alert on abort rate.	Esc keeps everything; /rewind to restore; queue corrections instead of interrupting; Codex agents.interrupt_message and Interrupt hook.	Esc interrupt semantics (C:080); /rewind checkpoint restore (C:087); Queued corrections (C:089, C:104)
8	Sessions that never end	2 Behaviour change	P2	Gateway cannot start a new task itself. Alert with the carried tokens; if a tag or gap marks a boundary, apply tool result clearing at that point (Class 2) or tell the client to reset the thread.	Claude Code: /clear between unrelated tasks (re-loads `CLAUDE.md` and project context); Codex: /new between phases; recommend a fresh session after three compactions with a structured handoff.	Phase-boundary reset with handoff file (C:035); Three-compactions rule (C:112, C:106); Sub-agent architecture (A:084, C:112)
9	No clear direction	3 Human action	P0	Gateway cannot supply direction. Alert only, then DevX proposal: route read-only exploration turns to a cheaper model (Claude Code does this with the Explore subagent on Haiku, C:072) after a paired test.	Class 3: specific prompts, plan mode (Shift+Tab), an LSP code-intelligence plugin so one go-to-definition replaces grep plus reads, and a codebase-overview skill. Keep the built-in Explore subagent so reads stay out of the main context.	Specific task specification (full spec vs bare user story) (C:027); Code intelligence (language server) plugins plus codebase-overview skill (A:215, C:083); Plan mode and Explore subagent isolation (A:215, C:072, C:055); Aider tree-sitter repo map (--map-tokens) (C:040, C:127)
10	Restarting too often	3 Human action	P0	Class 0 alert. Where the client supports it, keep the cache warm (TTL choice is a Class 2 change).	Resume the session instead of starting anew (the whole conversation resends, unchanged prefix still reads from cache within the TTL); persist state in auto memory or a handoff file; use idleCompaction=false if idle compaction itself triggers rewrites.	Resume a session (cache-aware) (C:055); Auto memory (`MEMORY` .md index + topic files) (C:077); Handoff file between threads (C:035)
11	Stale tool results carried forward	2 Behaviour change	P2	Inject context_management clear_tool_uses_20250919 with trigger 30-40k, keep 5, clear_at_least 3-5k and exclude_tools for results that cannot be re-fetched; or gateway-side masking with a rolling window. Both Class 2.	Claude Code microcompact/tool trimming already runs before LLM summary (C:112); Aider: /drop files; Codex: compact within phase. Masking with a tuned M needs harness support.	Tool result clearing (clear_tool_uses_20250919) (C:065, A:271); Observation masking (rolling window M) (C:130, C:133); Observation masking vs LLM summarisation hybrid (C:130); Garbage-collector view: liveness and write barriers (C:067)
12	Bigger model than needed	2 Behaviour change	P2	Class 2 after a paired CPS win: RouteLLM matrix-factorization router or FrugalGPT cascade at the gateway; escalate only flagged hard tasks.	Set subagent model (frontmatter or CLAUDE_CODE_SUBAGENT_MODEL), the opusplan alias (Opus plans, Sonnet executes), and availableModels allowlists.	RouteLLM matrix-factorization router (A:001); FrugalGPT LLM cascade (A:021); Hybrid LLM router (A:003); Classifier routing, Cost per task (A:052)
13	Fork context duplication	2 Behaviour change	P2	Keep prefix byte-identical so forks read cache; avoid forks after the 5-minute TTL lapses. Prefix-stability guard, class 1.	CLAUDE_CODE_FORK_SUBAGENT=0 to use a fresh subagent, with omitClaudeMd and a short brief; keep the warm cache alive.	Fork subagent reads the parent prompt cache (C:055, C:052); Cache TTL study (stable prefix, dynamic content last) (C:052); Sub-agent context isolation (A:084)

14	Large tool-call arguments	2 Behaviour change	P2	Use the provider text_editor tool and token-efficient tool use where the model supports it; once the context passes a trigger, clear old tool calls with clear_tool_inputs=true in context editing.	Aider: use edit format diff or udiff, not whole. Claude Code: skill or rule to persist and revise scripts instead of regenerating variants (DevSkills principle). Context editing is `API-only`, no CLI flag in corpus.	Aider edit formats (diff / udiff vs whole) (C:127); Token-efficient tool use beta and text_editor tool (A:068); Context editing clear_tool_inputs (C:065, A:271); DevSkills principle: persist and revise scripts (SimScrpt) (A:059)
15	Agent startup cost	1 Guard	P1	Identical prefix across helpers with cache_control breakpoints so startup is cache-read priced.	omitClaudeMd for helpers, skip skill preloads, use Explore/Plan agent types, keep description tokens under the 15k warning.	Subagent startup context contents (C:072); Stable per-type prefix (C:052); Prompt caching with cache_control (A:199)
16	Model switch rewrote the cache	2 Behaviour change	P2	No corpus source for a gateway guard; DevX proposal: sticky model per conversation key with a switch guard that warns when the cache is warm. Class 2.	Pick model and effort at session start; use a PreModelSwitch hook to confirm; avoid opusplan toggles and skills whose frontmatter names a model.	PreModelSwitch hook and model confirmation (C:055); Cache diagnostics reason model_changed (A:204); DevX proposal: sticky model plus switch guard
17	Old replies replayed	2 Behaviour change	P2	Compaction (compact_20260112) rewrites dialogue; masking does not touch assistant text. Treat as low priority: history is small in billed cost.	Compact Instructions in `CLAUDE.md` or /compact focus so the summary keeps decisions and drops narration; Codex experimental_compact_prompt_file.	Server-side compaction (compact_20260112) (A:271, C:065); `LLM-Summary` condenser (C:130, C:133); `LLMLingua-2` token-classification compression (A:081, C:024)
18	Clarification loops	2 Behaviour change	P2	Add persistence prompt text via system prompt injection at the gateway (class 2).	Bias-to-action prompts, plan mode, interview-then-`SPEC.md` pattern.	Persistence / bias-to-action prompting (A:208, A:205, A:249); Plan mode (C:095, A:053); Interview-then-`SPEC.md` (C:080)
19	Screenshot accumulation	2 Behaviour change	P2	Class 0 alert first. Then, after a paired CPS win, mask images older than M turns with a placeholder (observation masking). DevX proposal: downscale before forwarding (no corpus source).	Rely on Claude Code's oldest-image eviction past its cap; DevX proposal: a PostToolUse hook that downsizes screenshots. Context editing clear_tool_uses can clear old tool results.	Observation masking (C:130); Claude Code oldest-image eviction (C:055, A:071); Context editing: clear_tool_uses_20250919 (C:065, A:268)
20	Visual iteration loops	2 Behaviour change	P2	Class 0 alert, then mask old screenshots after each cycle (observation masking). DevX proposal: cap cycles at N and force a text summary.	Eviction of oldest images (Claude Code) bounds the cost; DevX proposal: a hook that warns on the 4th cycle. Lower effort for subagents doing the loop (A:282).	Observation masking (C:130); Claude Code oldest-image eviction (C:055); Effort parameter (A:282)
21	Cache not reused	2 Behaviour change	P2	Inject cache_control at the system message or second-to-last user turn; lengthen sub-minimum prompts. Class 0, then 2.	Ensure gateways forward cache_control and the anthropic-beta header; read /usage hit ratio and the likely cause.	cache_control_injection_points (C:060); Automatic caching and minimum-length rules (A:199, C:051); Prompt lengthening to cross the threshold (A:202)

22	Correction churn	3 Human action	P0	Alert only (class 0).	Specific first prompt; /clear after two failed corrections; repeated corrections into `CLAUDE.md`; /rewind.	Two-failed-corrections then /clear (C:080); /rewind restore (C:087); TokenEconomizer micro-skills (C:028); DevSkills (A:059)
23	Agents racing the cache	2 Behaviour change	P2	DevX proposal: single-flight lock; the first request goes alone, the rest wait for its first response. No corpus source for the lock itself. Class 2.	Prefer forks; workflow fan-out already holds all but the first agent up to 5s.	Wait for first response before parallel requests (A:199); Workflow fan-out hold (C:055); Batch caching: 1h block plus one shared-prefix request first (A:199)
24	Too much tool output	2 Behaviour change	P2	Cap and paginate in the tool server (25,000 token default is Anthropic's own), and set clear_tool_uses with a trigger, keep, clear_at_least and exclude_tools. Gateway truncation of tool_result is a DevX proposal and must be applied identically every turn to keep the prefix stable.	Codex: tool_output_token_limit (default 12,000; 8,000 and 6,000 in examples). Claude Code: PostToolUse updatedToolOutput hook to cap results; never truncate tracebacks or edit anchors. Observation masking only above about 10 events.	Tool-response pagination, range selection, filtering, truncation (A:284); Observation masking (rolling window M) (C:130, C:133); Server-side tool-result clearing (clear_tool_uses_20250919) (C:065, A:271); Codex tool_output_token_limit (C:104, C:106)
25	Noisy and repeated terminal output	2 Behaviour change	P2	Gateway cannot rewrite a command. Use clear_tool_uses for old results, a tool-server cap, or a subagent for noisy commands. Output compressors stay off by default.	Claude Code PreToolUse hook that rewrites test commands to pipe through grep for `FAIL / ERROR` with head -100 (updatedInput). Delegate verbose runs to a subagent. Codex tool_output_token_limit. Never pipe compressed output into shell counting.	PreToolUse hook rewriting the command (updatedInput) (A:215, C:090, C:024); `RTK` deterministic CLI proxy and Headroom API proxy (measured, off by default) (C:024, C:028); Preservation-first filter rules with marker recall gate (C:024); Delegate verbose operations to subagents (A:215, C:055)
26	Repeated tasks across sessions	3 Human action	P0	Class 0 alert only. A semantic response cache (Portkey simple/semantic cache) is a Class 2 candidate but unproven for multi-turn agent tasks, so it stays off.	Turn recurring tasks into a skill or `CLAUDE.md` note, or let auto memory save them; TokenEconomizer-style micro-skills are the closest measured method.	TokenEconomizer experience-grounded micro-skills (C:028); Semantic cache (Portkey simple and semantic modes) (A:240, A:113, A:115); Auto memory / `CLAUDE.md` (C:077, A:271)
27	Failed, retried and polling calls	2 Behaviour change	P2	Rate-based circuit breaker and loop guard at gateway.	Percentile turn budget, redirect then abort; maxTurns; fix error messages; constrained decoding for schema failures.	TokenEconomizer failure-loop guard (C:028); Turn-control (fixed p75, dynamic limits) (C:025, C:132); Rate-based circuit breaker (C:062); Actionable tool errors and constrained decoding (A:284, A:098, A:097, A:059, C:130)
28	Repeated screenshots	2 Behaviour change	P2	Class 0 alert. DevX proposal: perceptual-hash dedup returning the earlier image reference (no corpus source); mask older copies (observation masking).	Oldest-image eviction (Claude Code); DevX proposal: a PreToolUse hook that blocks a screenshot repeated with no intervening edit.	Observation masking (C:130); Claude Code oldest-image eviction (C:055, A:071); Image input pricing (A:146)

29	Progress narration	2 Behaviour change	P2	Class 1/2: set text.verbosity or response style default, and drop preamble prompting. Cap with max_tokens as a ceiling (guard, not a fix).	Output styles (outputStyle setting) or prompt text to remove preambles; Codex advice is to remove preamble prompting. Lower effort gives terser tool calls.	text.verbosity (A:205); Remove tool preambles (A:249, A:208); Effort parameter (A:282)
30	Whole-file reads	2 Behaviour change	P2	Alert, then Class 2 DevX proposal: return a path reference or the first N lines with a steering note for an oversized unsliced read (A:284 truncation guidance). Clearing handles file reads already in history.	Claude Code: Read deny rules for generated and vendored paths; LSP plugin for symbol lookup; PreToolUse hook to block oversize unsliced reads is a DevX proposal. Codex: tool_output_token_limit . After compaction Claude Code returns files over 5,000 tokens as a path reference.	Code intelligence plugin (go to definition) instead of reading files (A:215, C:083); Path-reference stub for oversized files (C:079); Tool-response range selection and truncation guidance (A:284); Large-file re-read after compaction (failure case) (C:106)
31	Duplicate reading across agents	2 Behaviour change	P2	No corpus source for gateway side. DevX proposal: cache identical read results across helpers.	Use fork when parent context is needed; ask helpers to return paths and line ranges (DevX proposal); apply DevSkills guidance.	DevSkills (reuse context, justify re-reads) (A:059); Fork versus fresh subagent choice (C:072); Structure-aware retrieval (CodeGraph) (A:059)
32	Agent coordination chatter	2 Behaviour change	P2	No corpus source; DevX proposal only.	CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=0; drop roster via disallowedTools on SendMessage; TeammateIdle hook.	Mailbox messaging (one message per recipient) (C:081); Drop roster by disabling SendMessage (C:072); Delegation floor cost and brief granularity (C:012)
33	Large images and `PDFs`	2 Behaviour change	P2	Class 0 alert, then DevX proposal: downscale oversized images and extract PDF text first (no corpus source). Clear after use via clear_tool_uses.	Prefer pdftotext or `OCR` text (named in hermes); Claude Code evicts oldest images/ `PDFs` past its cap. DevX proposal: a hook that downsizes images on Read.	Context editing: clear_tool_uses_20250919 (C:065, A:268); Claude Code image/PDF eviction (C:055, A:071); Observation masking (C:130)
34	Broad or unbounded searches	2 Behaviour change	P2	Alert. DevX proposal: cap search tool results in the tool server and return a steering message to narrow the query (A:284). Clearing for old results.	Claude Code: language-server plugin for symbol lookup; built-in searches already respect .gitignore; Read deny rules also drop denied paths from Grep and Glob (best effort). Hook to cap output. Aider: repo map with --map-tokens.	Structural retrieval with ast-grep hybrid (`RTK-ML`) (C:024); Code intelligence (LSP) navigation (A:215, C:083); DevSkills: state a hypothesis before retrieval, justify re-reads (A:059); Aider repo map with --map-tokens (C:040, C:127)
35	Bloated JSON and query results	3 Human action	P0	Fix at the tool or MCP server: return high-signal fields and add a response_format enum (concise/detailed). Gateway-side field projection is a DevX proposal, needs per-tool schema and a recall test.	Codex tool_output_token_limit and enabled_tools filtering; Claude Code PostToolUse updatedToolOutput hook to project fields with jq (DevX proposal). Prefer CLI tools such as gh over MCP servers per the doc.	response_format enum (concise vs detailed) and high-signal fields (A:284); Tool-result clearing for re-fetchable payloads (A:271, C:065); `LLMLingua` coarse-to-fine prompt compression (untested on JSON) (A:079, A:081, C:024)

36	Wordy and repeated output	1 Guard	P1	Class 1: set max_tokens on every call (C:073 guard) and track stop_reason; class 2: verbosity default and concise response_format.	Prefer edit-style tools over whole-file Write; output style setting for concise replies. Claude Code caps tool responses at 25,000 tokens.	Constrained-CoT (`CCoT`) (A:103); text.verbosity (A:205); response_format concise (A:284); max_tokens ceiling (A:052)
37	Web pages and search results	2 Behaviour change	P2	Class 0 alert; class 2 after a test: query-aware pruning of results (Provence, LongLLMLingua), cap web_search with max_uses, paginate and filter tool output. Response cache is a DevX proposal with correctness risk.	Claude Code caps tool responses at 25,000 tokens; design MCP tools with pagination and a concise response_format. Subagent for fetches keeps pages out of the main context.	Provence (A:087); LongLLMLingua (A:080); Self-Route (A:117); Tool-response design (A:284)
38	Copied history	3 Human action	P0	Class 0 alert; no corpus algorithm. DevX proposal: hash paragraphs at the gateway and warn with a suggested `CLAUDE.md` or memory note.	Move repeated background into `CLAUDE.md` or path-scoped rules (target under 200 lines); fork with /fork, whose first request reads the parent's cache; use a handoff file instead of pasting transcripts.	Fork inherits parent prefix (C:055); `CLAUDE.md` under 200 lines + path-scoped rules (C:077); Handoff file (C:035)
39	Compaction cost	2 Behaviour change	P2	Set compact trigger high enough that it fires rarely (default 150K, minimum 50K) and supply `instructions` so one pass keeps the right facts; prefer clearing for re-fetchable results. Class 2.	Codex: model_auto_compact_token_limit (clamped to 90% of window) and experimental_compact_prompt_file; Claude Code: PreCompact hook, Compact Instructions, idleCompaction=false; count compactions and restart at three.	Server-side compaction (compact_20260112) (A:271, C:065); Cache-warm delay rule (C:112, C:055); Codex compaction tuning (C:106)
40	Git and GitHub output	2 Behaviour change	P2	No corpus technique. DevX proposal: tool-server or gateway caps on diffs, with the agent steered to git diff --stat, --name-only, pathspec excludes for lockfiles and git log --oneline -n.	Claude Code PreToolUse updatedInput hook rewriting git diff/log commands to bounded forms (mechanism from A:215, command list DevX proposal). Subagent for review. Beware skills that inject full git diff `HEAD` or gh pr view --comments with the dynamic-context syntax.	PreToolUse command rewrite (updatedInput) (A:215, C:090); Deterministic CLI output compression (vendor claim, unverified) (C:024); Dynamic context injection and subagent delegation (C:075)
41	Dependency, vendor and generated files	3 Human action	P0	No gateway technique in corpus. DevX proposal: strip dependency-path results from tool output in the tool server and alert on first hit.	Claude Code: permissions.deny Read rules for dist, build, generated and vendor paths in .claude/settings.json (searches already respect .gitignore); a Read deny rule does not stop a Bash grep -r. Blog also claims a .claudeignore, not in the official doc.	permissions.deny Read rules (C:083); Respect .gitignore in content search (C:083); .claudeignore exclusion list (blog claim) (C:056); Cline strict path filtering (failure description) (C:126)
42	Verbose agent reports and briefs	1 Guard	P1	Truncate or summarize helper results at gateway (observation masking); cap output tokens.	Prompt cap such as Max 30 lines; short summaries back; maxTurns; tool_output_token_limit (25k default).	Report length cap in prompt (C:035, C:080, A:084); tool_output_token_limit (C:110, A:284); Observation masking (M=10) (C:130)

43	Repeated reads of unchanged files	2 Behaviour change	P2	Alert. Repeat-read stub ('unchanged since turn N') at a gateway is a DevX proposal and edits history, so apply at ingestion only. For old copies use clear_tool_uses; keep reads out of the main thread with subagents.	Claude Code: DevSkills principle (reuse context, justify re-reads) in a skill or `CLAUDE.md` ; durable facts in `CLAUDE.md` or `AGENTS.md` so they survive compaction; Codex lower tool_output_token_limit . Claude Code appends a system-reminder on file change.	DevSkills anti-repetition principles (SubRetrv, ReTest) (A:059); Observation masking (C:130, C:133); TokenEconomizer micro-skill injection (C:028); Persistent project memory (`CLAUDE.md` , `AGENTS.md`) (C:106, C:036)
44	Re-learning the codebase	3 Human action	P0	Class 0 alert; recommend a memory file. Gateway cannot save notes.	Enable auto memory (autoMemoryEnabled) and `CLAUDE.md` ; Aider repo map (tree-sitter + PageRank, --map-tokens budget) gives a ranked symbol map instead of re-exploring; TokenEconomizer micro-skills.	TokenEconomizer experience micro-skills (C:028); Memory tool and notes (memory_20250818) (A:271, A:269, C:077); Aider repo map (C:041, C:127)
45	High effort on trivial turns	2 Behaviour change	P2	Class 2 after a paired win: set output_config.effort per request (low for simple turns); per-message effort keeps the cache. max_tokens as hard limit with adaptive thinking.	effortLevel setting, CLAUDE_CODE_EFFORT_LEVEL env var, --effort flag; maxEffortLevel as managed cap. Codex: model_reasoning_effort.	Effort parameter (A:282, A:200); Prompt-Induced Waste (effort arm) (C:026); Overthinking evidence (A:089)
46	Tool discovery and skill loading	3 Human action	P0	Keep a stable wrapper (get_tool_schema/invoke_tool) so discovery is one step. Class 0.	Trim skill descriptions (capped at 1,536 characters), set skillOverrides to name-only, or disable-model-invocation on rare skills. Class 3.	skillOverrides and disable-model-invocation (C:075, C:079); Skill re-attach budget (C:075, C:079); On-demand schema access (C:022)
47	Usage while idle	3 Human action	P0	Class 0 alert with budget alerts. DevX proposal: per-key schedule rate limit.	Class 3: stop or schedule the loop; Claude Code idleCompaction setting (v2.1.290+) compacts idle one- `hour-TTL` conversations. Handoff file moves a task between threads.	Idle compaction (C:055); Handoff file (C:035); Budget alerts (A:213)
48	MCP tool schema bloat	3 Human action	P0	Replace schemas with a compressing MCP proxy (get_tool_schema/invoke_tool). Class 2.	Disable rarely used servers (class 3); in Codex use enabled_tools or disabled_tools allow/deny lists.	mcp-compressor (C:022); Server disable and Codex tool allow/deny lists (C:104, C:107); Code execution mode and mcp2cli (C:019); Tool consolidation and curation (A:284, A:084)
49	Unused skills/plugins	3 Human action	P0	Not applicable at the gateway; alert only. Class 0.	Retire stale skills, set skillOverrides off or name-only, disableBundledSkills. Class 3.	skillOverrides states and disableBundledSkills (C:075, C:079); /doctor prompt-audit and `CLAUDE.md` linting (C:077); Plugin skills are appended (C:055)

50	No visibility	3 Human action	P0	Require a virtual key per developer/team and a metadata header per request; ship events to Langfuse as a sidecar.	Claude Code: CLAUDE_CODE_ENABLE_TELEMETRY=1 with OTEL_METRICS_EXPORTER/OTEL_LOGS_EXPORTER in managed settings (locked); hooks via allowManagedHooksOnly; Codex: [otel] exporter keys and hooks in config.toml (user-level, project-local otel is ignored).	Claude Code OpenTelemetry (A:217, C:002); Gateway tagging with session header (C:001, C:088); Managed hooks and Langfuse coding-agent tracing (C:074, A:234)

Technique detail by row

1. Heavy starting context

The first-call prefix (system prompt, tool schemas, `CLAUDE.md`, skill listings) is re-read on every call; $248 of the $430 is never-called MCP tool definitions.

API detection: First request tokens minus the user prompt; flag a prefix above 10k tokens, and split it by tools[] size versus system size. CLI detection: OpenTelemetry/transcript first-call input tokens; /context breakdown; tools[] from prompt_snapshot.

Technique	How it works	Effect	Loses when	Source
Deferred tool loading (tool search)	Tool schemas load only when needed; ENABLE_TOOL_SEARCH=auto loads upfront until deferrable schemas reach 10% of the window; deferred is the default.	Example context shows MCP tools deferred at 120 tokens; auto-defer saved about 13.2k tokens in one self-measured setup.	Needs a lookup step; ENABLE_TOOL_SEARCH=false loads everything upfront.	C:079, C:019: Anthropic, Explore the context window; bswen, How MCP Tool Definitions Inflate Your `AI` Agent Token Costs
mcp-compressor (MCP proxy with get_tool_schema / invoke_tool)	Proxy replaces a server's tools with get_tool_schema, invoke_tool and optional list_tools, at verbosity levels full/brief/minimal/none.	94-tool GitHub server: 17,600 to 3,900 (low), 2,200 (strong), 500 (max) tokens; quality impact 'almost no' per Atlassian's own evaluation.	Self-reported by vendor; needs on-demand schema access kept; namespacing prefixes such as mcp__atlassian__*.	C:022: Atlassian, MCP Compression: Preventing tool bloat in `AI` agents
Tool Attention (ISO score plus lazy schema loader)	Embedding Intent-Schema Overlap score gates tools; a compact summary pool is kept and full JSON schemas are promoted only for top-k tools.	Simulated 120-tool, six-server benchmark: tool tokens per turn 47.3k to 2.4k (-95%).	Projection from simulation, not measured on live agents.	C:021: arXiv 2604.21816, Tool Attention Is All You Need (Sadani, Kumar)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Empirical token-consumption analysis and self-prediction of cost on `SWE-bench` Verified	No new algorithm; empirical study. OpenHands on `SWE-bench` Verified (500 tasks x 4 runs) across 8 models, phase analysis (Setup, Explore, Fix, Validate, Closeout), round-level cost decomposition with Sonnet 4.5, and agent self-prediction of its own token use.	Agentic coding ~1000x more tokens than code reasoning and chat; input/output ratio 153.85 vs 1.33 and 0.16; cost $1.857 vs $0.016 and $0.023; Most expensive run ~2x cheapest on average (abstract says up to 30x); accuracy peaks at intermediate cost; Cache reads dominate tokens and dollars in every phase; output ~80x pricier per token than cache reads; Self-prediction correlation up to 0.39; every model underestimates; Kendall tau-b 0.32 vs difficulty	8 models, one framework (OpenHands), one benchmark; self-prediction noisy and underestimates.	C:023

Layered `AGENTS.md` discovery with project_doc_max_bytes cap	Codex concatenates global then root-to-cwd `AGENTS.md` files, stopping at 32 KiB default.	not measured	Large files truncated at cap.	C:099

Test before enabling: Paired CPS test: same task set with and without deferral or proxy; track task success and tool-call errors next to starting tokens (C:022 evaluates quality with on-demand access kept). Not covered: No measured live-agent numbers for Tool Attention. Hermes' $182 `non-MCP` share (system prompt, `CLAUDE.md`) has no specific corpus technique beyond the 200-line target.

2. Cache-write premium

Cache writes cost 1.25x (5m) or 2x (1h) base input, so every write pays a premium that only reads repay.

API detection: Sum cache_creation_input_tokens times price times (write multiplier minus 1); split ephemeral_5m versus ephemeral_1h fields. CLI detection: Same usage fields via OpenTelemetry cache creation tokens per session; claude -p 'hello' --output-format json shows usage.cache_creation.

Technique	How it works	Effect	Loses when	Source
`TTL-tier` matching (5m vs 1h by reuse gap)	One read repays a 5m write (1.35x vs 2x); 1h needs two reads. 1h fits side-agents running longer than 5 minutes but under an hour.	Blanket 1h made one deployment's bill 8.6% worse because 98% of reuse landed within about 34 seconds.	Self-reported blog, one deployment.	A:199, C:052: Claude prompt caching docs; dev.to, Your cache hit rate is lying to you. It's the caller mix
Targeted 1h write on the dispatch turn	Write 1h only on the turn that spawns long-running children; dynamic content goes after the stable prefix.	Dispatch-turn 1h -6.0%; dynamic content after stable prefix -7.6%; all three fixes combined -13.6%.	Blog measurement, depends on caller mix.	C:052: dev.to, Your cache hit rate is lying to you
Per-caller by TTL hit-rate logging	Log hit rate per caller times TTL tier; a breakpoint nobody reuses costs 25% extra on 5m and 100% on 1h.	Not measured beyond the examples above.	Needs a caller label Token Economics must derive.	C:052: dev.to, Your cache hit rate is lying to you

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Explicit cache breakpoints with per-breakpoint TTL (30m / 12h) and stable-prefix rules	GPT-5.6+: prompt_cache_breakpoint with ttl; write 1.25x for 30m, 2.5x for 12h, reads 0.1x; min prefix 1,024 tokens; dynamic content goes last; effort changes via appended config update.	Cache read 0.1x uncached input (0.05x for GPT-6.1 Sol/Terra).	Compaction or rewriting history resets reuse; 12h write costs 2.5x.	A:201
Gemini implicit and explicit context caching at 10% input price	Implicit caching is automatic; explicit caching declares content so the discount is guaranteed; minimum 2,048 tokens.	Cached tokens cost 10% of standard input on Gemini 2.5 and above; caches deleted within 24 hours (implicit).	Explicit cache adds storage cost and management; minimum length applies.	A:143

Cache-aware cost comparison with TCO model and hybrid routing counterfactual replay	Longitudinal case study: Langfuse and vLLM telemetry plus Git mining. Fix Commit Ratio via regex over commit messages; difficulty-stratified Mantel-Haenszel odds ratio; TCO = M x (compute + ops + T_iter x wage); hybrid-routing offline counterfactual replay by difficulty tier.	99.3% cache hit cut API cost 88.6% to $0.57/M tokens vs $2.83/M shared on-prem; `FCR` 74.9% vs 45.9%; Mantel-Haenszel `OR` 3.61; Shared on-prem saves 40.1% true TCO; dedicated costs 43.8% more; High-to-Claude hybrid saves 26.3%	One developer, non-randomized, tasks differ between periods; keyword-classified fixes; Period A fix count inflated by CI work; replay assumes tier exchangeability.	C:053
prompt_cache_key and cache_control breakpoints with 5m/1h TTL via gateway	OpenAI auto-caches over 1024 tokens with prompt_cache_key for hit control; Anthropic allows up to 4 breakpoints with 5m or 1h TTL.	Up to 90% saving on cached tokens.	Gateway-specific; some providers 5-minute TTL only.	A:237
prompt_cache_key, explicit cache breakpoints, 30m/12h TTL option	Stable prompt_cache_key per shared prefix, up to four explicit breakpoints per request, ttl 30m default or 12h; explicit mode avoids write charges when no breakpoint.	Cache writes charged only on GPT-5.6 and later; key beyond about 15 requests per minute may miss cache.	Write charges appear on newer models; 12h writes billed from 2026-11-01.	A:148
caching:'auto' two-marker placement and x-session-affinity header	Gateway places cache_control on the last message and the message before the last user turn; session affinity header keeps a conversation on the provider holding its cache.	Anthropic write 1.25x, read 0.1x; one follow-up read breaks even.	One-shot traffic pays write premium; affinity does not guarantee a hit.	A:247
Per-message effort change without cache invalidation; cache price tiers	Fable 5.1 supports mid-conversation effort changes (beta) and preserved thinking across model switches; cache read $0.25 vs 5m write $12.50 and 1h write $20 per `MTok`.	Cache read is a quarter of the earlier price; input $10 vs Opus 5.5 $4 per `MTok`.	Forced tool use errors; editing earlier turns invalidates thinking blocks.	A:285
Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072
Implicit versus explicit cache checkpoints; TTL resets on hit	Explicit checkpoints need minimum prefix tokens per model; static content first; TTL (many models 5 minutes) refreshes on each hit.	not measured in the pages read	Implicit caching is best effort; minimums vary by model.	A:157
Peak/off-peak pricing; cache-hit price tier	Off-peak rates are half of peak; cache hit input is about 2% of miss price on flash ($0.003 vs $0.15 off-peak).	Price list only.	Time-of-day pricing only helps schedulable work.	A:181
caching:'auto', cache_anchor_items, cache_ttl, session affinity	Gateway adds cache markers per provider, pins a conversation to the provider holding its cache, and selects 5-minute or 1-hour lifetime.	not measured	Gateway lock-in; affinity limits failover.	A:248
Cached-input discount, sparse attention indexer and speculative decoding at serving level	Cached input priced at $0.14 per million (90% off); sparse attention picks 2,048 prior tokens; speculative decoding verified by the full model.	Peak 446 tok/s; typical 90k prompt with 600 turns mostly cached.	Vendor claim for one provider/model.	A:258
Cache-read at 5% of input on Opus 5.5 and Sonnet 5.5; Haiku 5.5 as cheap subagent	Price table: Opus 5.5 $4/$20 with cache read $0.20 and write $5 (5m) / $8 (1h); Haiku 5.5 steps price above 100K prompt tokens.	Haiku 5.5 over 100K still a quarter of Sonnet 5.5 rate.	Third-party blog; prices may change.	C:010
/usage breakdown and ENABLE_PROMPT_CACHING_1H	/usage shows parallel sessions, subagents, cache misses, long context as percent of last 24 hours; PreCompact hooks can block compaction; env var opts into 1h TTL.	Not measured.	Release notes only.	C:092

Test before enabling: Replay a week of traffic and compute cost under 5m versus 1h per caller using the 1.25x/2x/0.1x multipliers (A:199); ship only where CPS falls. Not covered: Doc and transcripts disagreed on subagent TTL (C:052); verify with the usage.cache_creation check before trusting it.

---

3. Cold cache at session and agent start

The first call of the main session and of each helper writes the whole prefix, because a subagent's first request does not read the parent cache.

API detection: First call per derived conversation key with cache_creation_input_tokens above 0 and cache_read 0; gateway cannot see session ids; Token Economics derives a conversation key (prefix hash plus idle gap). CLI detection: session_id/agent start events plus first-call usage; /usage Prompt cache (main) line.

Technique	How it works	Effect	Loses when	Source
Pre-warming with max_tokens: 0	Send a request with max_tokens 0 and an explicit breakpoint on the shared block, same thinking and effort config; billed as a write with zero output.	Not measured in corpus.	Costs a write; wasted if nothing follows.	A:199: Claude prompt caching docs
Persistent per-type static prefix	Give each agent type a stable prefix so children read a warm cache; cwd, date and branch go after it.	-1.0% alone; combined -13.6% with the other two fixes.	Small alone; cron/scheduled sessions still ran about 0% hit.	C:052: dev.to, Your cache hit rate is lying to you
Fork and fan-out hold	A fork inherits the parent cache; a workflow fan-out holds all but the first agent up to 5s so they can read its cache.	Not measured in corpus.	Subagents (not forks) still start cold.	C:055: Claude Code docs, How Claude Code uses prompt caching

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
`KV-aware` routing, agent hints (nvext) and selective KV retention	Harness sends priority, output-length and prefill hints; global index routes to workers with highest cache overlap; ephemeral subagent KV marked for early eviction.	Claude Code calls hit 85-97% cache; agent teams 97.2% aggregate hit rate; 11.7x read/write ratio.	Needs a Dynamo-based serving stack, not hosted `APIs`.	A:167
Session affinity for cache hits; cache and difficulty-aware routing; batch tier	Keeps an agent on the same server so its context stays cached; routing is cache- and difficulty-aware; Batch is 50% off.	Cache hit rates up to 99.6% in long agent sessions; cached input at half price or less; cache writes free.	Vendor marketing claims; a customer quote of $4M to $800K is anecdotal.	A:260
Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072

Automatic on-disk prefix cache with persisted prefix units	Each request persists prefixes at the end of user input and output; a hit needs full match of a persisted unit; common prefixes auto-detected after the second request.	Cache hit and miss reported per request; hits may need several seconds to build.	Best effort; no guaranteed hit; cleared in hours to days.	A:182
Prefix prompt cache with x-grok-conv-id routing header	Matching prefix is reused at reduced price; x-grok-conv-id steers requests to the same server to raise hit rate.	Cached tokens billed at a reduced rate (rate not in extract).	Not guaranteed; evictions and routing can miss.	A:223
Provider sticky routing and session_id for cache hits	After a cached request, later requests go to the same provider; session_id sets the sticky key; Anthropic needs explicit cache_control.	Sticky sessions expire after 60 minutes idle; only active when cache read price is lower.	Manual provider order disables stickiness; fallback loses cache.	A:231

Test before enabling: Compare first-call cw tokens per helper before and after a stable-prefix change on matched tasks; keep success rate and CPS as the guard. Not covered: No corpus source for a gateway-level conversation key; DevX proposal: derive it from prefix hash plus idle gap.

4. Cache expired after a break

After an idle gap beyond the TTL (5m on API keys, 1h on subscriptions) the entire history is rewritten at the write premium.

API detection: Rewrite (cw above max(5000, 0.5*ctx)) after a gap at or above the TTL on the same conversation key. CLI detection: Timestamps between calls in the transcript plus ttl; /usage likely-cause text.

Technique	How it works	Effect	Loses when	Source
Keepalive crossover rule	Keepalive pings pay off only below a 62.5 minute break (5 x 1.25 / 0.10); TTL runs from request start.	Rule derived in the source; no field measurement.	Self-reported blog arithmetic.	C:052: dev.to, Your cache hit rate is lying to you
TTL settings and idleCompaction	Main-conversation TTL is 1h on subscriptions, 5m on API key; promptCacheTtl and idleCompaction control it.	Not measured in corpus.	Needs v2.1.242 or later.	C:055: Claude Code docs, How Claude Code uses prompt caching
1h block for long gaps	Use 1h when prompts come less often than every 5 minutes but more often than every hour.	Not measured in corpus.	2x write cost.	A:199: Claude prompt caching docs

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Explicit cache breakpoints with per-breakpoint TTL (30m / 12h) and stable-prefix rules	GPT-5.6+: prompt_cache_breakpoint with ttl; write 1.25x for 30m, 2.5x for 12h, reads 0.1x; min prefix 1,024 tokens; dynamic content goes last; effort changes via appended config update.	Cache read 0.1x uncached input (0.05x for GPT-6.1 Sol/Terra).	Compaction or rewriting history resets reuse; 12h write costs 2.5x.	A:201

Implicit caching with 90% discount plus durable tier (24h max)	Prefixes are cached automatically with a 90% discount, falling to durable storage after eviction; put common content first.	90% discount on cached tokens; expiry no later than 24h.	Explicit cache adds storage fees; Gemini-specific.	A:280
Implicit versus explicit cache checkpoints; TTL resets on hit	Explicit checkpoints need minimum prefix tokens per model; static content first; TTL (many models 5 minutes) refreshes on each hit.	not measured in the pages read	Implicit caching is best effort; minimums vary by model.	A:157
Idle-session thinking clearing (clear_thinking_20251015, keep:1) and effort-default / length-limit regression lessons	Clearing old thinking after 1h idle was meant to shrink the uncached resume; a bug repeated it every turn, causing cache misses; a 25-word between-tool limit cost quality.	Prompt ablation showed a 3% eval drop; medium default effort felt less intelligent and was reverted to high/xhigh.	Pruning history or verbosity limits can silently hurt quality and cause cache misses; need per-change evals.	A:054
/usage breakdown and ENABLE_PROMPT_CACHING_1H	/usage shows parallel sessions, subagents, cache misses, long context as percent of last 24 hours; PreCompact hooks can block compaction; env var opts into 1h TTL.	Not measured.	Release notes only.	C:092

Test before enabling: Histogram of inter-call gaps per key; simulate TTL choices with the multipliers; A/B on a subset. Not covered: No corpus measurement of subscription 1h behaviour beyond the docs.

5. Many agents

Each helper agent is a separate model loop with its own context, so fan-out multiplies billed turns and prefix tokens.

API detection: No session or helper id in API traffic. Derive a conversation key from prefix hash plus idle gap, then count distinct short-lived keys per parent and their cost share. CLI detection: SubagentStart/SubagentStop hooks and transcript sidechains: helper count, turns and cost per parent session.

Technique	How it works	Effect	Loses when	Source
Orchestrator pattern with cheap workers and fan-out budget cap	A strong planner delegates to cheaper worker models, and total fan-out is capped by a budget.	84-98% cost reduction at the worker rate, with a floor cost per delegation and a brief-granularity optimum.	Delegation floor cost makes tiny tasks net negative.	C:012: Orchestrator / cheap-worker cost analysis (C:012)
Subagent model routing and concurrency settings	Pin helper agents to a small model and limit concurrent threads and rollout budget through harness settings.	Not measured in corpus as a multiplier; no measured subagent multiplier exists.	Weaker helper may need retries, erasing the saving.	C:072, C:052: Claude Code subagent docs; cache TTL study
Heavier harness with subagents inflates cost	Paired comparison of harness configurations on the same tasks shows added subagent machinery raised cost.	Cost increased versus the lighter harness (see source for setting).	Result is one benchmark setting.	A:051: A:051 harness cost comparison

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source

Empirical token-consumption analysis and self-prediction of cost on `SWE-bench` Verified	No new algorithm; empirical study. OpenHands on `SWE-bench` Verified (500 tasks x 4 runs) across 8 models, phase analysis (Setup, Explore, Fix, Validate, Closeout), round-level cost decomposition with Sonnet 4.5, and agent self-prediction of its own token use.	Agentic coding ~1000x more tokens than code reasoning and chat; input/output ratio 153.85 vs 1.33 and 0.16; cost $1.857 vs $0.016 and $0.023; Most expensive run ~2x cheapest on average (abstract says up to 30x); accuracy peaks at intermediate cost; Cache reads dominate tokens and dollars in every phase; output ~80x pricier per token than cache reads; Self-prediction correlation up to 0.39; every model underestimates; Kendall tau-b 0.32 vs difficulty	8 models, one framework (OpenHands), one benchmark; self-prediction noisy and underestimates.	C:023
Decoupled controller model approves extra token budget from workspace snapshot	Worker freezes at a 50,000-token limit; a separate controller model scores approve-vs-deny with expected value p>1/2 using task, files, tests snapshot.	Injected budget counter referenced 0 times in 14,000+ messages; request_more_budget used 0 of 5,000 turns; approval 97% default, 79% neutral.	Controller models biased to approve (four of six); 100-task evaluation set.	C:136
Session `AI-credit` cap (/limits set, --max-ai-credits)	A soft cap on credits per session (min 30; 1 credit = $0.01) stops the agent cleanly at the limit and lets you raise it.	not measured	Soft limit: in-progress response completes so spend can exceed it; preview feature.	A:153
Execution-verified downgrade-and-cascade step-level routing labels	Step-level router mapping a trajectory prefix to a tier (low, mid, mid_high, high). Labels come from a greedy sequential-locking downgrade search: from a successful strong trajectory, try tiers cheapest-up at each step and accept if the full trajectory still passes with no more `LLM-call` steps. Three-model tier cascade verification; failure-aware `COST SAVE` ; bill = API cost + (1-resolved)*$0.60.	Dynamic: trained router resolved 75/100 at $25.66 vs unrouted Opus 4.6 74/100 at $54.73 (-53.1% API cost); Rule-based variant cost $172.56 (6.7x more than trained router); Opus 4.6 as a router predicted high for only 7 of 147 verified-high SWE steps; Cascade recovered 28 of 33 downgradeable labels vs 15 of 33 for single-model probing	Small static corpus; greedy search may miss cross-step interactions; labels tied to a fixed pool and price snapshot; dynamic track `SWE-bench` Verified only (Astropy and Django); English only.	C:030
Code Review run-frequency setting (once after creation vs every push vs manual)	Cost multiplies by pushes under After every push; choose Once or Manual and track Average cost per repo.	not measured	Default for new repos is the costliest setting.	C:082
Mixture-of-Agents (layered proposers and aggregators)	Layered proposers and aggregators: y_i = concat_j A_ij(x_i) + x_1, each layer's models see all previous outputs through an Aggregate-and-Synthesize prompt. Default 6 open models, 3 layers, Qwen1.5-110B-Chat final aggregator. MoA-Lite is 2 layers with Qwen1.5-72B aggregator.	MoA 65.1+-0.6% `LC` win; MoA w/ GPT-4o 65.7%; MoA-Lite 59.3%; GPT-4o 57.5%; MoA-Lite matches GPT-4o cost with +1.8 points; MoA-Lite beats GPT-4 Turbo (55.0%) by ~4% at over 2x cost-effectiveness; 6 proposers 61.3% vs 1 proposer 47.8%	High time-to-first-token (last layer must finish). First aggregation layer gives most gain. LLM ranker much worse than aggregation.	A:031
`Comp-LLM` (sub-query dependency graph with embedding-similarity expert assignment)	`Comp-LLM` : decomposer (SFT Llama 2 7B on 1000 GPT-4o examples) → expert router (mean-pooled embedding, cosine, sq_sim 0.7 else base model) → `DAG` scheduler running ready sub-queries in parallel → aggregator; LoRA experts.	`Comp-LLM` (35B) F1 0.78 vs Llama-2 70B 0.79, Mixtral 0.74, `OLMoE` 0.52. Decomposer F1 0.87/0.86 vs few-shot 0.48/0.42. 108.78s vs 892.07s for offloaded Llama-2-70B on one A40; latency 1.1x-1.7x vs sequential.	Needs experts per domain, static reasoning only, measures size not dollars.	A:039
`RLM-Cascade` (complexity router, cheap draft model, Opus verify or skip)	Local proxy (ANTHROPIC_BASE_URL) with a rule-based O(1) router, no model calls. Requests with a non-empty tools field go straight to Opus. Simple prefixes go to the cheap model only (`SKIPPED`). Complex keywords or input over 1,500 chars get cheap draft then Opus verify (USE_DRAFT = `ACCEPTED` , or `ENHANCE`).	Savings 45.8% (service, N=125), 47.4% (architecture set, N=39), 32% (current run, N=12); model predicted 67.2% vs 47.4% observed. p50 latency 2,026 vs 3,698 ms. Quality 100% vs 95% pass (N=20).	Tool-selection turns (70-80% of Claude Code turns) bypass to Opus, so savings apply to the ~20-30% text turns. Router is a keyword classifier. One cost inversion (+$0.01189). TTFT 2.1x worse. Learned router, parallel draft+verify, local tier are future work.	C:031

Harness-level cost and Pass@1 benchmark with a cost-calibrated Lite-80 subset	Adapter protocol (create_agent, send_task, backup_session, delete_agent, get_docker_args); patch exported from git diff; history after base_commit removed to stop leakage. Lite-80 subset picked by quartile-quota local search with ranking hinge and cost-parity filter.	Bare adapter 19.3% vs full adapter 71.2% resolved; largest harness gap at fixed model 24.3 pp; Best Pass@1 Hermes Agent 81.7% at $36; Lite-80 vs full: `MAE` 0.81 pp at 25.3% of cost	Meta-Harness evolved and evaluated on same instances; Lite is a proxy; appendix F is single-run.	A:060
`MTRouter` turn-level outcome-estimator routing	`MTRouter` : `MLP` outcome estimator on [history embedding (frozen Qwen3-Embedding-0.6B, 8192 tokens); model embedding (8-d metadata + 16-d residual → 64-d)]. Target = terminal score minus progress-weighted error penalties (1.0/0.8/0.2). Argmax per turn, $2/episode cap.	ScienceWorld 53.8 vs GPT-5 48.4 at -58.7% cost. HLE 26.0 vs 23.8 at -43.4%. `OOD` ScienceWorld +5.0 at -65.8%; HLE `OOD` cost -52.3%. ~5 switches vs ~20 (Router-R1). Stays on same model after error 90.2%/80.9% vs 38.3%/66.4%.	Offline only; switching can lose prompt cache (authors note).	A:008
`SLM-first` heterogeneous agents with `LLM-to-SLM` conversion algorithm	Six-step `LLM-to-SLM` conversion: secure logging of non-human-facing calls; curate and filter data (10k-100k examples rule of thumb, `PII` removal); task clustering; SLM selection; specialised fine-tuning (LoRA/ `QLoRA` , optional distillation); iterate and retrain including the router.	7B SLM 10-30x cheaper than 70-175B LLM in latency/energy/ `FLOPs` (cited). Replaceable query share 40-70% across three agents (authors' estimates, not measured). SLM defined as under 10B params.	Authors list opposing views: LLM generalists win on general language, centralisation economies of scale ('jury is still out'), equally possible worlds. Barriers: infrastructure investment, generalist benchmarks, awareness.	A:041
Budget Tracker and `BATS` budget-aware test-time scaling	Budget Tracker: plug-in that shows remaining and used per-tool budgets each ReAct iteration. `BATS` : structured tree plan (exploration vs verification clues), self-verification with `SUCCESS/CONTINUE/PIVOT` decisions, trajectory summarization, LLM judge picks final answer. Unified cost = token cost (input/output/cache) + tool-call cost.	Budget Tracker at budget 10 matches ReAct at 100 (12.8% vs 12.6%) with 40.4% fewer search calls, 19.9% fewer browse calls, 31.3% lower cost.; `BATS` 24.6% BrowseComp, 46.0% `BrowseComp-ZH` , 27.0% `HLE-Search` , training-free.; Ablation: removing verification drops BrowseComp 18.7% to 15.4%.	Evaluated on web search agents only; tool-call budget, not token budget.	C:131
Cache-read at 5% of input on Opus 5.5 and Sonnet 5.5; Haiku 5.5 as cheap subagent	Price table: Opus 5.5 $4/$20 with cache read $0.20 and write $5 (5m) / $8 (1h); Haiku 5.5 steps price above 100K prompt tokens.	Haiku 5.5 over 100K still a quarter of Sonnet 5.5 rate.	Third-party blog; prices may change.	C:010
opusplan alias, subagent model setting, effort levels, organization effort caps	opusplan uses Opus in plan mode and Sonnet in execution; CLAUDE_CODE_SUBAGENT_MODEL sets subagent model; effort sets adaptive reasoning.	Not measured.	Risk of mismatch with effort after fallback model.	C:073

Test before enabling: Paired A/B on identical tasks: subagents on versus capped or cheap-model, compare total cost per resolved task and pass rate. Not covered: No measured subagent multiplier. C:013's 5-10x claim is a vendor blog. Gateway-side detection has no corpus source (DevX proposal).

6. Long history replay

Every call resends the whole conversation, so a 100k-plus history is re-read (at cache-read price) on every later turn.

API detection: Gateway: per request, input+cache_read+cache_creation tokens minus the first request of the same conversation key, flagged above 100k (hermes A1 constant;

set from P0 baseline). CLI detection: gateway signal only; join to session via the x-claude-code-session-id header the gateway receives (C:001) or `OTel` session.id (A:217).

Technique	How it works	Effect	Loses when	Source
Production clearing settings (trigger 30-40k, keep 5, clear_at_least 3,000-5,000)	Cookbook advises lowering the trigger to 30-40k input tokens for production, keeping 5 tool uses and forcing at least 3,000-5,000 tokens freed per clear so the cache rewrite is worth it.	Guidance only; not measured as a saving in the corpus.	Defaults of 100,000 / keep 3 fire late; the 30-40k advice has no head-to-head test.	C:066, C:065: Memory & context management with Claude (cookbook); Context editing on the Claude Developer Platform
Server-side compaction (compact_20260112)	Beta context edit that fires at `trigger` (default 150K, minimum 50K), summarises the conversation into a typed compaction block, accepts custom `instructions` and `pause_after_compaction`.	Cookbook: peak 335,279 to 169,164 tokens with a 2,783-token summary; high-level facts kept 3/3, obscure specifics 0/3. Single run. Figures are from A:271.	Loses verbatim detail; next turn rebuilds the cache; on short sessions the summary call costs more than it saves.	A:271, C:065: Context engineering: memory, compaction, and tool clearing (Claude cookbook)
Compact within a phase, new thread between phases (handoff file)	Compact only inside one verifiable phase; at a phase boundary have the agent write a handoff file (docs/plans/*.md), start a clean thread and pass only that file.	not measured in corpus	Costs one write turn; loses anything not written to the file.	C:035, C:112: Context Management Across Codex Surfaces (Developer Toolkit); Context Compaction Showdown (Vaughan)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Time-at-end-of-message placement, prompt_cache_key, encrypted-reasoning carry-over, compaction	Keep timestamps out of the system prompt prefix, route each chat with prompt_cache_key, pass tolnput() reasoning items so cache covers prior answers, compact with responses.compact().	Time at top of system prompt made the same chat cost 2.2-2.9x as much; run cost 56 cents total.	Turn after compaction misses cache; single example app, Grok pricing.	A:224
Copilot CLI background auto-compaction at ~80% (dynamic to 90%) plus /context, /usage, /compact	Compresses history in background near 80% of token limit, waiting until 90% when static context is 75% or more; /compact manual, /usage and /context for visibility.	Threshold behavior only; no savings measured.	Compaction reclaims less when static context is large.	A:154
Retained reasoning plus native compaction, programmatic tool calling, low reasoning effort, small-tier model (Luna) selection	Persists reasoning across turns with compaction, runs tool orchestration in code outside context, and uses cheaper tiers or lower effort.	`ARC-AGI-3` 13.3% to 38.3% with about 6x fewer output tokens; BrowseComp 84.04% at $1.33 versus 84.36% at $33.27 for GPT-5.5; PlayerZero cost -64%.	Vendor and customer testimonials; harness-specific.	A:207
Length-aware `AUC` scoring and harmonic-mean `ATLAScore` over an 8K-1M grid	`ATLAS` benchmark. 3+5 taxonomy (foundational: retrieval, aggregation, reasoning; application: `QA` , `ICL` , code, memory, holistic). Length-aware trapezoidal `AUC` per dimension; `ATLAScore` is the harmonic mean of three category aggregates; delta-method confidence intervals.	Gemini-3.1-Pro-Preview leads at 128K; Claude-Opus-4.6 leads at 1M (`ATLAScore` 70.55). 20 of 26 models change rank between 128K and 1M. Mean relative decay 24.3%; Opus-4.6 loses 8.5%, `GLM-4` .7 non-reasoning 60.5%. GPT-5.2 retrieval 88.21->47.31 at 1M.	English only, holistic at original lengths, model-generated `AMemBench` transcripts, no summarisation, contamination risk.	A:048
OpenCode prune-then-summarise; Claude Code three-layer trim, cache-friendly tail trimming, 9-section summary	OpenCode hides old tool outputs only if it frees over 20,000 tokens and keeps the last 40,000; Claude Code trims the tail to preserve the cache prefix and auto-compacts at window minus 13,000.	Worked example on a 15,400-token session; no cost figures.	Codex handoff summary is all-or-nothing; source based on reverse engineering.	C:007

Model natively trained for compaction across multiple context windows	The model auto-compacts near the window limit and repeats; adds xhigh reasoning effort.	30% fewer thinking tokens at medium effort on `SWE-bench` Verified versus GPT-5.1-Codex.	Vendor claim; Codex-specific.	A:250
Codex compaction backends (default summarization, remote, remote_v2) and two-phase memory consolidation	Auto-compaction at model_auto_compact_token_limit ; memory phase 1 extracts raw memories, phase 2 consolidates into `MEMORY` .md and summaries.	Not measured.	Reverse-engineered reading of one Codex version.	C:047

Test before enabling: Paired replay on sampled long sessions: arms raw, clearing, compaction; metric cost per successful task with caching on, plus a probe set of high-level and obscure facts after the edit (the A:271 probe design). Sweep clearing trigger 30k/60k/100k. Not covered: No measured dollar saving for history edits in a coding-agent bill. C:024 puts conversation history under 1% of billed cost and caps all visible-token compression near 5%, which conflicts with hermes's 10.5% estimate for this row.

7. Interrupted turns

Interrupting a response wastes the output already generated and often forces a re-ask.

API detection: No corpus source. DevX proposal: streams closed early by client plus follow-up near-duplicate request. CLI detection: Interrupt hook and Esc events in transcripts, followed by a similar prompt.

Technique	How it works	Effect	Loses when	Source
Esc interrupt semantics	Esc stops the turn and keeps all context.	Not measured.	Kept partial output still costs tokens.	C:080: Claude Code best practices
/rewind checkpoint restore	Restore conversation or code to an earlier checkpoint.	Not measured.	Loses later work.	C:087: Claude Code checkpointing docs
Queued corrections	Queue a message instead of interrupting; Codex Interrupt hook and agents.interrupt_message.	Not measured.	Delays the correction.	C:089, C:104: Claude Code and Codex docs

Test before enabling: Count aborts per session and tokens discarded; compare with queued corrections. Not covered: Billing of aborted streams has no corpus source.

8. Sessions that never end

A new task starts inside an old session, so 20k+ tokens of unrelated prior context ride along on every call.

API detection: Gateway: derive a conversation key (prefix hash plus idle gap, 30 min per hermes); flag a first-of-task request whose context exceeds prefix plus new prompt by 20k+ tokens. CLI detection: Hook SessionStart/UserPromptSubmit (C:074) plus transcript

compaction count; task boundary from tag or 30 min gap; SessionEnd reason shows clear vs resume.

Technique	How it works	Effect	Loses when	Source
Phase-boundary reset with handoff file	Compact within a phase, open a new thread at the next phase, pass only a handoff file the user reads line by line.	not measured in corpus	The handoff is hand-written state; omissions are lost.	C:035: Context Management Across Codex Surfaces (Developer Toolkit)
Three-compactions rule	Each compaction summarises a summary, so loss compounds; after three or more, start a fresh session with a structured handoff. /context shows depth.	not measured in corpus	Heuristic from one blog; no threshold study.	C:112, C:106: Context Compaction Showdown: Codex CLI, Claude Code and 5 Other Agents; Mastering Context Compaction in Codex CLI
Sub-agent architecture	Give each focused subtask a sub-agent with a clean window; the lead keeps a plan and receives a condensed result.	not measured in corpus here	Sub-agents repeat startup context (rows 5/13).	A:084, C:112: Effective context engineering for `AI` agents (Anthropic)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Small single-task threads and 20% context notice	Amp warns at 20% of context window; advises small scoped threads because requests above 200k tokens cost roughly twice per token.	Over-200k requests about 2x per-token price.	Short post, no measurement.	C:123
One chat per coherent outcome with fork, compact, subagents and `AGENTS.md`	Plan mode before hard tasks, short `AGENTS.md`, /fork only when work branches, subagents for bounded work, one chat per outcome.	not measured	Qualitative guidance only.	C:098
Model natively trained for compaction across multiple context windows	The model auto-compacts near the window limit and repeats; adds xhigh reasoning effort.	30% fewer thinking tokens at medium effort on `SWE-bench` Verified versus GPT-5.1-Codex.	Vendor claim; Codex-specific.	A:250

Test before enabling: Mine sessions that crossed a task boundary and replay them with a reset at the boundary vs without; compare cost per successful task and first-attempt success on the follow-up task. Not covered: Corpus gives no task-boundary detector and no measured saving from /clear; hermes's 20k line has no stated basis.

---

9. No clear direction

The agent reads and searches broadly before its first edit because the task or target files were not stated, and every result stays in history and is re-billed each turn.

API detection: Per derived conversation key, count Read/Grep/Glob tool_use blocks in assistant responses before the first Edit/Write tool_use; flag above 15 (hermes constant, no stated basis; set from the P0 baseline). CLI detection: PostToolUse hook or `OTel` claude_code.tool_result events (tool_name, prompt.id) give the same count per prompt.id; join to the first edit-tool event.

Technique	How it works	Effect	Loses when	Source
Specific task specification (full spec vs bare user story)	Give the agent a structured spec with the target files and expected behaviour instead of a plain problem description. The study varies spec detail on `SWE-bench` Verified tasks at three thinking efforts and measures token spend.	Cutting a full spec to a plain prose problem description raised spend 29.7% and turns 16.4%, 2,700 runs, Kimi K3.	Prompt sensitivity is task-dependent (13% to 115%); one model; run-to-run spread of x1.34 is unchanged by the prompt.	C:027: Can your `AI` agent be cheaper? Investigating the effects of task specifications on token spend in agentic coding tasks (Smekal, Stanford)
Code intelligence (language server) plugins plus codebase-overview skill	A language server answers definitions and references directly, so one call replaces a grep followed by reading several candidate files. A skill describing architecture and key directories gives the agent the map up front.	not measured in corpus (doc states the mechanism only).	Needs the language server binary on each machine; typed languages only per the doc.	A:215, C:083: Manage costs effectively (Claude Code), Anthropic; Set up Claude Code in a monorepo or large codebase, Anthropic
Plan mode and Explore subagent isolation	Plan mode has the agent explore and propose an approach before editing, so a wrong direction is caught early. The Explore subagent runs read-only search in its own context and returns a summary, and can be pinned to model: haiku.	not measured in corpus.	Subagents do not share the parent prompt cache (C:055) and agent teams use about 7x tokens in plan mode (A:215).	A:215, C:072, C:055: Manage costs effectively (Claude Code); Create custom subagents, Anthropic; How Claude Code uses prompt caching, Anthropic
Aider tree-sitter repo map (--map-tokens)	Builds a dependency graph from tree-sitter definitions and ranks symbols with a graph-ranking algorithm to fit a token budget (default 1k), so the model sees structure without reading files.	not measured in corpus.	Budget is a target not a ceiling: issue reports show 16,000 tokens at --map-tokens 1024; map is doubled when no files are added.	C:040, C:127: How to Manage the Aider Context Window and Token Budgets; How to Reduce Aider Token Usage (Tokenade blogs)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
One chat per coherent outcome with fork, compact, subagents and `AGENTS.md`	Plan mode before hard tasks, short `AGENTS.md`, /fork only when work branches, subagents for bounded work, one chat per outcome.	not measured	Qualitative guidance only.	C:098

Test before enabling: Paired A/B on matched tasks: vague prompt vs specific prompt (C:027 design: repeat each arm, report mean and spread), with and without LSP plugin. Metric: billed cost per successful task, plus reads before first edit. Run 3 repeats per arm because run spread is about x1.34. Not covered: No corpus source measures the cost of exploration itself or a read-count threshold. Kiro has no source in the CLI corpus. No billed saving for LSP plugins or plan mode.

---

10. Restarting too often

A fresh session soon after the last one rebuilds the same context (re-reads files, re-issues setup) at cold-cache prices.

API detection: Gateway: new conversation key within 3600 s of the previous one for the same caller and project, with cache_creation high and overlapping prefix; cost = calls before the first edit. CLI detection: Hook SessionStart source

(startup/resume/clear/compact/fork) and SessionEnd reason (C:074); gap under RESTART_GAP 3600 s.

Technique	How it works	Effect	Loses when	Source
Resume a session (cache-aware)	Resume resends the whole conversation; the unchanged prefix still hits the cache within the cache lifetime, one hour on a Claude subscription by default, five minutes otherwise.	Resume benefit not quantified in corpus.	After a break longer than the TTL nothing is cached and resume reprocesses history uncached; /compact then costs the most.	C:055: How Claude Code uses prompt caching (Anthropic)
Auto memory (`MEMORY` .md index + topic files)	Claude writes four kinds of notes itself to ~/.claude/projects/<project>/memory/; the first 200 lines or 25KB of `MEMORY` .md load each session. Toggle autoMemoryEnabled, CLAUDE_CODE_DISABLE_AUTO_MEMORY, autoMemoryDirectory.	not measured in corpus	Loaded every session so it is a standing token cost; a wrong note repeats in every session.	C:077: How Claude remembers your project (Anthropic)
Handoff file between threads	Write docs/plans/<task>.md before ending the thread; start the next thread from it.	not measured in corpus	Hand-maintained.	C:035: Context Management Across Codex Surfaces (Developer Toolkit)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Small single-task threads and 20% context notice	Amp warns at 20% of context window; advises small scoped threads because requests above 200k tokens cost roughly twice per token.	Over-200k requests about 2x per-token price.	Short post, no measurement.	C:123

Test before enabling: Compare matched task pairs started as new session vs resumed/handoff: cost per successful task and tokens before first edit. Not covered: No measured saving for restart avoidance; resume economics beyond the TTL rule are not quantified.

11. Stale tool results carried forward

File reads and command output stay in context 30+ calls after use, and are re-read on every call.

API detection: Gateway: tool_result blocks that remain in the messages array 30 calls after first appearing; tokens (plus image tokens) times later cache-read price until compaction (hermes A5). CLI detection: gateway signal only (transcript tool_use_id gives the same).

Technique	How it works	Effect	Loses when	Source

Tool result clearing (clear_tool_uses_20250919)	Server replaces the oldest tool results with placeholder text once input exceeds `trigger` (default 100,000), keeps the last `keep` (default 3) pairs, skips if `clear_at_least` tokens cannot be freed, and honours `exclude_tools`.	Cookbook run, one synthetic task: peak context 335,279 to 173,137 tokens. No dollars, single run. Figures are from A:271.	Invalidates the cached prefix on every clear, so each clear pays a cache write; cleared content is gone unless re-fetched.	C:065, A:271: Context editing on the Claude Developer Platform (Anthropic); Context engineering: memory, compaction, and tool clearing (Anthropic cookbook)
Observation masking (rolling window M)	Replace tool observations older than the last M turns with a short placeholder (for example 'Previous 8 lines omitted'); the agent's own reasoning and actions stay.	`SWE-agent`, M=10: cost per instance cut by more than half in most setups (Gemini 2.5 Flash $0.41 to $0.18); solve rate not significantly lower. Qwen3-Coder solve rate 54.8% masked vs 53.4% raw.	Needs re-tuning per scaffold; masking loses at 7 events or fewer (-45% to -149% savings); only +8.6% net on a 9-event average task set.	C:130, C:133: The Complexity Trap: Observation Masking vs LLM Summarization (JetBrains Research); Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
Observation masking vs LLM summarisation hybrid	Combine masking of old observations with LLM summarisation of the turn history rather than either alone.	Cost down a further 7% vs masking alone and 11% vs summary alone (`SWE-agent/OpenHands`).	Adds an LLM call and one more knob set.	C:130: The Complexity Trap (JetBrains Research)
Garbage-collector view: liveness and write barriers	Clearing is garbage collection with no liveness knowledge; protect results the agent builds on via exclude_tools and write state to memory before clearing.	not measured in corpus	Opinion piece; exclude_tools is described as crude.	C:067: Context Editing Looks Like a Feature. It's Actually a Garbage Collector Without Write Barriers

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
OpenCode prune-then-summarise; Claude Code three-layer trim, cache-friendly tail trimming, 9-section summary	OpenCode hides old tool outputs only if it frees over 20,000 tokens and keeps the last 40,000; Claude Code trims the tail to preserve the cache prefix and auto-compacts at window minus 13,000.	Worked example on a 15,400-token session; no cost figures.	Codex handoff summary is all-or-nothing; source based on reverse engineering.	C:007

Test before enabling: Arms raw/clearing/masking on sampled traffic; billed cost with caching on, count of cache rewrites caused by clears, and re-fetch rate of cleared files (agent re-reads). Not covered: Cache-aware break-even for clearing (yield vs rewrite) is not measured in the corpus; masking effect at 8-10 events is mixed (C:133).

12. Bigger model than needed

Simple tasks and helper calls run on the most expensive model when a cheaper one gives the same result.

API detection: Classify difficulty per call (simple categories, helper calls) and compare the model used with sonnet-5-5 price delta; gateway needs a difficulty classifier. CLI detection: Transcript model field per turn and subagent; flag Opus/Fable on simple-category turns and helper subagents.

Technique	How it works	Effect	Loses when	Source

RouteLLM matrix-factorization router	Learns P(win) between strong and weak model from preference data using query embeddings; threshold sets the strong-model share.	Over 2x cost reduction without quality loss; 95% of GPT-4 score at 50% calls (`MT` Bench).	`BERT` and `causal-LLM` routers near random on low-data training.	A:001: RouteLLM (arXiv 2406.18665)
FrugalGPT LLM cascade	A scoring function rates each answer in [0,1]; query models in order and stop at threshold tau.	Matches GPT-4 with up to 98% lower cost.	Needs a trained scorer and thresholds.	A:021: FrugalGPT (Chen, Zaharia, Zou)
Hybrid LLM router	Predicts query difficulty and routes small or large; threshold tunable at test time.	Up to 40% fewer large-model calls, no quality drop.	Depends on difficulty predictor.	A:003: Hybrid LLM (Microsoft Research)
Classifier routing, Cost per task	Cheap yes/no complexity classifier routes between Flash-Lite and Flash; escalate flagged tasks only.	Router solved 50/50; regex plus length heuristic fell to 70.6% held-out.	50 hand-labelled prompts, single runs; routing and caching pull apart.	A:052: Cost per task (Google blog)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
RouterBench evaluation framework with `AIQ` metric and Zero router baseline	Routers are points in the cost-quality plane. Mixing two routers with probability t gives any point on their segment; extrapolation adds cost without quality. The non-decreasing convex hull is the Zero router; `AIQ` is hull area over [cmin, cmax] divided by the cost span. `KNN` and `MLP` routers pick argmax of lambda times predicted quality minus cost; the cascade calls models cheapest first until a judge passes.	Oracle: MMLU 0.957 at $0.297 vs GPT-4 0.828 at $4.086; HellaSwag 0.994 at $0.86 vs GPT-4 0.923 at $24.29. Predictive `AIQ` vs Zero: MMLU 0.773/0.769 vs 0.763; `ARC` 0.871/0.875 vs 0.877; `MBPP` 0.644/0.642 vs 0.660; HellaSwag 0.588 all. Cascade at zero error: MMLU 0.901, `MBPP` 0.862, `ARC` 0.923, `GSM8K` 0.680 (Zero 0.628), HellaSwag 0.469 vs Zero 0.607. RAG `AIQ : KNN` 0.695, `MLP` 0.675, cascade 0.657, Zero 0.632.	Cost and quality only, no latency. Mostly academic multiple-choice tasks, 2023-era models, no coding agents or long contexts. Cost for predictive routers approximated by per-token price. Cascade judge is simulated with perfect knowledge, so results are an upper bound. Three tasks graded by GPT-4. RAG has few models and one client's queries. In-domain train and test split; out-domain test only plotted.	A:002
Preference-aligned routing via Domain-Action Taxonomy with a 1.5B generative router	Qwen2.5-1.5B fully fine-tuned to generate a route-policy id given the conversation plus all policy (name, `NL` description) pairs in the prompt. A separate policy-to-model mapping is edited without retraining. Domain-Action taxonomy with fallback to domain if action is vague.	Overall 93.17 vs Claude-sonnet-3.7 92.79 and GPT-4o 89.74; turn 96.05, span 94.98, conversation 88.48. Authors cite +7.71% average over the compared models.	Bounded by precision of policy descriptions; quality bounded by user's model-to-policy assignment; all policies sit in the prompt; no cost or answer-quality measurement.	A:011
Frozen-backbone `SLO-decoded` router heads with GRPO reward quality - lambda*cost - mu*latency	One forward pass of a 0.6B model gives task class, routing distribution over a model catalog and embedding, decoded under cost/latency/quality constraints; trained SFT then GRPO.	Holdout n=1,941: route_acc 0.791 (+25.8 pts over 53.3% majority), task_acc 0.723; top3 0.990. Experimental.	Labels are what models actually served, quality proxy fixed at 1.0; Opus-dominated data; frozen backbone cut accuracy from 0.961.	A:019
Conformal Cascade: defer when conformal prediction set is not a single answer	Per tier sample N=16 at T=0.7; nonconformity s=1-freq(a); split-conformal threshold from calibration; accept if prediction set size is between 1 and kappa, else defer to next tier. Cost c1+c2*Pr[|C1|!=1]; beats always-strong if Pr[|C1|=1]>c1/c2.	Wins 49 of 72 pairs, ties 6, loses 17 vs oracle-tuned best heuristic; Llama/MMLU kappa 1 to 3 at alpha 0.10: cost 2.86 to 1.34 (-53%) at 5.3 pp accuracy cost; Tier-1 acceptance 0% (`MMLU-Pro` , `GPQA`) to 95.6% (Belebele-Eng)	Finite answer sets only; marginal guarantee; K=2 only; needs 16 samples per tier and at least 200 labelled calibration examples.	A:028
Anthropic-compatible endpoint with claude-opus mapped to deepseek-v4-pro, others to deepseek-flash	Point ANTHROPIC_BASE_URL at DeepSeek; claude-haiku/sonnet names route to the cheaper flash model.	not measured	Many Anthropic fields ignored (mcp_servers, container); behaviour differs from Claude.	A:183
Cursor Router modes (Cost, Balance, Intelligence) and pooled usage billing	Cost mode routes to cheapest-model pool; Balance/Intelligence bill at routed model list price; third-party routes incur Cursor Token Rate.	not measured	Individual plans only get Cost mode; unused usage does not roll over.	A:267

Fine-tuned 350M SLM for tool calling (SFT with `TRL`)	Supervised fine-tuning of facebook/opt-350m for 1 epoch (`HF TRL SFTTrainer`) on ToolBench converted to 187,542 Thought/Action/Action Input examples; `LR` 5e-5, effective batch 32, `FP16`, gradient checkpointing, 5,860 steps.	Pass rate 77.55% overall vs `ToolLLaMA-DFS` 30.18%, ChatGPT-CoT 26.00%, ToolLLaMA-CoT 16.27%, Claude-CoT 2.73%; 74.0-80.5% across categories.	Training data and evaluation tightly coupled; may not generalize beyond ToolBench; controlled API set; retraining needed as `APIs` evolve; still needs compute and data.	C:048
LiteLLM budget caps, Langfuse, `LLMLingua`, RouteLLM, `GPTCache` semantic cache	Roundup: LiteLLM max_budget per key; `LLMLingua` prompt compression; RouteLLM routes between two models by threshold; `GPTCache` semantic caching.	RouteLLM 85% cost reduction keeping 95% GPT-4 quality on `MT` Bench (45% MMLU, 35% `GSM8K`); LongLLMLingua +21.4% RAG with 1/4 tokens.	Vendor blog citing others' numbers; RouteLLM is binary routing only.	C:061
GraphRouter heterogeneous graph router (task/query/LLM nodes, edge prediction)	Heterogeneous graph with task, query and LLM nodes; initial embeddings from a GPT-4o description encoded by `BERT`. Edges task-query and `LLM-query` (performance, cost). 2-layer `GAT`, 32-dim hidden. Trained by edge prediction; inference picks argmax dot product. Reward = alpha*Perf - beta*Cost with (alpha,beta) = (1,0), (0.5,0.5), (0.2,0.8).	Performance-First reward 0.539 vs FrugalGPT 0.517, oracle 0.588; Balance 0.209 vs 0.196 (oracle 0.231); Cost-First 0.064 vs 0.057 (oracle 0.072); Claims >=12.28% reward gain and >=88.89% of oracle; Performance-First table gain is ~4%; New LLMs few-shot 0.207 vs C2MAB-V 0.189 (+9.52%), time cost -99.45%	Exploratory; no path or taxonomy signals; LLMs handle numeric price and context info poorly; small 4-task benchmark.	A:004
Arch-Router preference-aligned domain-action routing (1.5B model)	A 1.5B model maps each prompt to user-defined domain and action routes, each tied to a preferred model, without retraining when models change.	Claims state-of-the-art preference matching on conversational data; no cost figure in this page.	Routes by declared preference, not measured cost; accuracy numbers not on this page.	A:012
Cost-Spectrum Contrastive Routing (`CSCR`): shared embedding plus `FAISS k-NN`	Cost-Spectrum Contrastive Routing: prompts (frozen MiniLM + `MLP` head) and models embedded in one space. Models described by logit footprints (open models) or perplexity fingerprints (black-box `APIs`). Cost-Spectrum InfoNCE: banded positives (K=5), band-specific temperatures, cost-weighted negatives (gamma=0.2). Routing is one `FAISS k-NN` lookup.	Best `AUDC` on all three benchmarks; up to 25% better accuracy-cost trade-off. `OOD AUDC` 0.5146 vs next-best 0.4863. Cost-aware loss lifts `AUDC` 0.342 to 0.495 and peak accuracy 36% to 54% (ablation).	Did not test large pools. Perplexity descriptor is at least 2x slower. Cost-aware training raises cost versus vanilla variant.	A:020
Two-stage cascade: confidence deferral policy then abstention policy, online-learned	Two-tier cascade Mbase → Mlarge → human. Verification confidence from surrogate token probability `YES /( YES + NO )` (also self-verification, consistent self-verification, `MC-dropout STP`), calibrated by Bayesian logistic regression on 100 samples; posterior mean is confidence, posterior std is uncertainty. Defer if confidence <= phi_base; abstain if uncertainty > xi. Thresholds tuned online by Adam on a soft-masked system risk 1-E[Correct]+lc*E[Cost]+la*P(abstain).	Cascade has lower cumulative regret than either single model on 4 of 5 datasets; MedQA unclear; Verifying with the large model gives better `delta-IBC`; on `ARC2-simple delta-IBC` up to 258.4 +-39.1 with large-model verification vs about -2 to -37 with base; Calibration set size 50-500 samples makes no significant difference; Label flipping in calibration raises regret	Sensitive to cost and abstention weights (trivial solutions); initialisation affects convergence; needs human feedback; MedQA no gain; multiple-choice `QA` only.	A:029
RouterEval benchmark and oracle-router analysis (model-level scaling)	RouterEval: router as m-way classifier over an input embedding predicting a one-hot or multi-hot `best-LLM` vector. Splits 8:1:1; simple m in {3,5}, hard m in {10,100,1000}.	A router with p>=0.5 of matching the oracle improves quickly with candidates; about 3-10 candidates most cost-effective, can approach GPT-4 reference with weak models. Existing routers have `VR` <=1 and `VB` <=1. Some routers have Ep 0.00-0.03 (collapse to one model).	Accuracy only; cost objective deferred; most records not open source; data volume too small for a strong router.	A:045

Inference production frontier (cost-quality Pareto, optimal concurrency zone)	Cost per test = hourly `GPU` cost x wall time. Hourly cost = depreciation + power + maintenance (about $0.79/h per A800; cloud $2.82-5.64/h). Sweep serving concurrency to find the cheapest setting that still meets throughput (>20 tok/s) and TTFT (<1s) limits.	WiNGPT-3.5 concurrency 8→48: total time 2034s→774s; 48 optimal. WiNGPT-3.5 76.2 at $0.34 vs `Seed-OSS` -36B 72.2 at $0.55. Thinking model WiNGPT-3.0 emits 3.44M output tokens vs about 0.35-1.4M for others.	No training cost or capex, hardware-stack dependent, proxy benchmark, no confidence intervals, weak link to API billing.	A:065
Auto Router: eligibility filter, difficulty classification, quality-vs-cost scoring, session affinity	Filters eligible models, classifies task type and difficulty, ranks by quality versus cost, falls back down the list; session-id header keeps one model per turn for caching.	not measured; sticky session lets requests reuse prompt cache.	Without session id, requests may switch models and lose cache; no savings number.	A:192
Session affinity for cache hits; cache and difficulty-aware routing; batch tier	Keeps an agent on the same server so its context stays cached; routing is cache- and difficulty-aware; Batch is 50% off.	Cache hit rates up to 99.6% in long agent sessions; cached input at half price or less; cache writes free.	Vendor marketing claims; a customer quote of $4M to $800K is anecdotal.	A:260
Cascade routing (unified routing plus cascading, proven optimal)	Routing is a linear program. Theorem 1: the optimum is gamma*s_MIN + (1-gamma)*s_MAX at the lambda that meets the budget, found by binary search over lambda. Cascading is sequential routing over supermodels M_1:i (Theorem 2). Cascade routing picks among all supermodels containing the models already run (Theorem 3), with supermodel quality E[max member quality] and cost summed. Lemma 1 prunes negative-marginal-gain models.	Table 1 0-shot `AUC` at 11 models Low/Med/High: routing 83.25/74.63/72.67, cascade baseline 84.48/73.64/69.79, cascade ours 84.47/75.10/70.26, cascade routing 87.24/77.57/74.40. Gains up to 8% over cascading and 12% over routing. `SWE-Bench` : 54.12 vs routing 40.47 (10 models). Realistic noisy estimators: 1.2% mean gain. Runtime 15 ms; Greedy 1.4 ms with 0.5-1.3 point loss.	Gains depend on estimate quality; with poor estimators cascade routing only matches routing. Quality estimators are trained per model and per computed-model history. Up to 2^k supermodels; Normal handles 40 models in 1 s, Max-Depth 3 handles 80. Hyperparameter search is harder for cascades; in 5-shot low-noise cascade ours trails the baseline in two of three cases.	C:029
`LLM-to-SLM` agent conversion algorithm (S1-S6)	S1-S6 conversion recipe: log `non-HCI` agent calls; curate and filter (10k-100k examples, remove `PII`); cluster tasks; select SLM; fine-tune (LoRA/ `QLoRA/full` , optional distillation); iterate and retrain `SLMs` and router.	Cited from other work: 7B SLM 10-30x cheaper than 70-175B LLM. Authors estimate replaceable LLM queries: MetaGPT about 60%, Open Operator about 40%, Cradle about 70%.	No measured savings; estimates are the authors'; acknowledges centralized LLM inference may still be cheaper per token (jury still out).	C:049
`SLM-default` , `LLM-fallback` with uncertainty routing, verifiers, Cost per Successful task (CPS)	`SLM-default` routing: small model with guided decoding and validators handles the task; uncertainty or validation failure escalates to a large model. Track Cost per Successful task (CPS) and schema validity; rollback thresholds trigger fallback.	Body claims 10-30x lower cost; abstract says 10x-100x (inconsistent).; Table `II` is placeholder data.	Not a controlled experiment; headline factors unmeasured.	C:063
Premium request multipliers and monthly allowance	Premium models consume allowance with multipliers; overages billed per request; budgets can be set.	Reasoning models may consume 5x or 20x the standard rate.	Legacy billing for annual plans only after June 2026.	C:116
Router-R1 `RL-trained` multi-round router with cost reward	Router is a 3B LLM policy trained with `PPO` that interleaves think with search calls to a pool of 6 LLMs (max 4 routing steps). Reward = format + (1-alpha)* `EM` outcome + alpha*cost; hierarchical (format failure zeroes others); cost normalized with sliding-window percentiles. New LLMs added via descriptors without retraining.	Average `EM` 0.416 (Qwen router) and 0.409 (Llama router) above baselines.; Adding two unseen LLMs: average `EM` 0.458 to 0.463 without retraining.; Average API calls about 1.01-1.36.; Alpha 0.6-0.9: performance falls as cost reward rises; emergent small-first then escalate.	`QA` only with Exact Match; cost reward studied only for `QA` ; router itself costs 3B inference per step; no cost-weighted Pareto in the main table.	A:005

Causal (CATE-corrected) router training mixing gold and preference labels	Pairwise router (premium vs cheap). Bias between preference-based (PB) and gold-standard (GS) labels is a CATE , estimated with R- or DR-learners ; pseudo-GS labels y+bias train a regressor on GS plus corrected PB . Route to premium if predicted gain exceeds cost factor w. GS scale normalised to PB scale.	PB-GS difference mean significantly below 0 (p<2.2e-16); Meta-routers gain most when GS data is scarce; PRBench : DR-learner best at small n	GS labels from an LLM judge not experts; pairwise only; needs overlapping query support; no headline cost number in text.	A:013
Learned post-hoc deferral rule on token-level uncertainty quantiles (LM cascade)	Two-model cascade (FLAN-T5 Base to Large). Deferral score from token-level uncertainty: Chow-Sum, Chow-Average, Chow-Quantile (quantiles of per-token probabilities), and a learned post-hoc MLP (5 layers, 32 units) on quantile features, optionally plus small-model and large-model intermediate embeddings.	Chow-Sum defers long outputs, Chow-Average defers short ones (length bias). Post-hoc rule is consistently best or near-best on AUC-DF . Using large-model first-token embedding helps most on classification.	Only FLAN-T5 encoder-decoder; decoder-only and RLHF-calibration effects untested. Intermediate large-model embeddings add non-trivial cost. No single best quantile across tasks.	A:022
RouterArena router leaderboard (accuracy vs cost metrics, 12 routers)	RouterArena: Arena score= (1+b) AC /(bA+C), log2-normalised cost, b=0.1, cmin 0.0044, cmax 200 per 1k queries.	MIRT-BERT 66.9, Azure-Router 66.7, NIRT-BERT 66.1, GPT-5 64.3, vLLM-SR 64.3, CARROT 63.9, RouterDC 33.8; oracle 90.89%. NIRT-BERT ~378% of best model cost; vLLM-SR / CARROT ~35% cheaper under 2% loss; MIRT-BERT ~5x optimal.	One-shot queries, not agents.	A:046
Execution-verified downgrade-and-cascade step-level routing labels	Step-level router mapping a trajectory prefix to a tier (low, mid, mid_high, high). Labels come from a greedy sequential-locking downgrade search: from a successful strong trajectory, try tiers cheapest-up at each step and accept if the full trajectory still passes with no more LLM-call steps. Three-model tier cascade verification; failure-aware COST SAVE ; bill = API cost + (1-resolved)*$0.60.	Dynamic: trained router resolved 75/100 at $25.66 vs unrouted Opus 4.6 74/100 at $54.73 (-53.1% API cost); Rule-based variant cost $172.56 (6.7x more than trained router); Opus 4.6 as a router predicted high for only 7 of 147 verified-high SWE steps; Cascade recovered 28 of 33 downgradeable labels vs 15 of 33 for single-model probing	Small static corpus; greedy search may miss cross-step interactions; labels tied to a fixed pool and price snapshot; dynamic track SWE-bench Verified only (Astropy and Django); English only.	C:030
Premium request model multipliers (code review = 13)	Each prompt uses one premium request times the model's rate; Copilot code review consumes 13 from June 1 2026.	not measured	Applies to legacy annual plans only.	C:117
LLMRouterBench unified routing evaluation (Best Single and Oracle baselines, Pareto analysis)	Benchmark and framework with Collector, Evaluator, Adaptor. 21 datasets, 33 models, 391,645 instances, 23,945 prompts, ~1.8B tokens, $2,771.84 API cost, ~1K GPU hours. Two settings: performance-oriented (20 ~7B models, 15 datasets) and performance-cost (13 flagship models, 10 datasets). 10 routers compared; metrics AvgAcc, Gain@R/B, Gap@O, PerfGain, CostSave, ParetoDist.	Best routers: up to +4% AvgAcc and up to 31.7% cost reduction vs GPT-5 (Best Single) at matched accuracy; OpenRouter -24.7% PerfGain, fails to match; Model-recall failure: when <=3 experts are right (410 queries, 11.9%), Avengers 24.6%, EmbedLLM 23.2%; Latency: Qwen3-Thinking vs GLM-4 .6 similar cost and accuracy but 262.1s vs 32.4s	Not all routers covered; excludes long-context, multimodal, vertical domains; latency approximate via OpenRouter throughput.	A:006
BayesianRouter (offline multi-task router prior plus online Thompson sampling)	Offline: SmolLM2-135M encoder over (prompt, response pair) with MLP on [e+e'; |e-e'|]; Bradley-Terry head on reward-model disagreement pairs plus auxiliary per-RM classification head, L = L_bt + lambda L_cls. Online: Bayesian linear Thompson sampling per query, Sigma^-1 += hh^T/sigma^2, offline BT embedding as prior mean; reward is negative DPO loss, quantile-normalised.	AlpacaEval-2 63.23 vs best single RM 61.86 and LASER 60.50; GSM8K 75.66; MMLU 57.39; Offline router OOD accuracy 87.92 vs ~87.65 best single (RewardBench 2, 2,939 pairs); LinUCB collapses to a single arm	DPO only (PPO untested); gap to oracle remains; slightly slower than the fastest single RM ; gains modest.	A:014

Speculative cascades (deferral rule implemented via speculative execution)	Speculative cascades. Drafter q proposes gamma tokens; verifier p scores them in parallel. A token-level deferral rule (Chow, Diff, `OPT`, or token-specific V1-V3) sets target (1-delta)q + delta*p. GenSpecSample (Algorithm 4) generalizes speculative sampling to that target. Lossy `SD` is a special case.	Table 2 (T=1, gamma=5, Small to Large) latency reduction at large-model quality, `WMT/XSum/CNNDM`: SpecDecode[Lossy] 1.61/1.10/1.57x; SpecCascade[`OPT`] 1.95/1.17/1.80x; Token 1.85/1.18/1.89x. Best quality at large-model latency, `WMT` `BLEU`: Lossy 17.26, `OPT` 18.33, Token 22.50.	Two-model setup only. Higher total compute than sequential cascades. Plug-in estimators need calibrated models. Objective is local per position. Batch size 1, `TPU`. Evaluation protocol differs from standard `SD`. No fairness analysis. Paper claims no new state of the art. `OPT` weaker on Gemma than T5.	A:023
Mixture-of-Agents (layered proposers and aggregators)	Layered proposers and aggregators: y_i = concat_j A_ij(x_i) + x_1, each layer's models see all previous outputs through an Aggregate-and-Synthesize prompt. Default 6 open models, 3 layers, Qwen1.5-110B-Chat final aggregator. MoA-Lite is 2 layers with Qwen1.5-72B aggregator.	MoA 65.1+-0.6% `LC` win; MoA w/ GPT-4o 65.7%; MoA-Lite 59.3%; GPT-4o 57.5%; MoA-Lite matches GPT-4o cost with +1.8 points; MoA-Lite beats GPT-4 Turbo (55.0%) by ~4% at over 2x cost-effectiveness; 6 proposers 61.3% vs 1 proposer 47.8%	High time-to-first-token (last layer must finish). First aggregation layer gives most gain. LLM ranker much worse than aggregation.	A:031
`Comp-LLM` **(sub-query dependency graph with embedding-similarity expert assignment)**	`Comp-LLM`: decomposer (SFT Llama 2 7B on 1000 GPT-4o examples) → expert router (mean-pooled embedding, cosine, sq_sim 0.7 else base model) → `DAG` scheduler running ready sub-queries in parallel → aggregator; LoRA experts.	`Comp-LLM` (35B) F1 0.78 vs Llama-2 70B 0.79, Mixtral 0.74, `OLMoE` 0.52. Decomposer F1 0.87/0.86 vs few-shot 0.48/0.42. 108.78s vs 892.07s for offloaded Llama-2-70B on one A40; latency 1.1x-1.7x vs sequential.	Needs experts per domain, static reasoning only, measures size not dollars.	A:039
Application-specific capability profiling testbed (cost-aware model selection per application)	Eight-step profiling method, Stage 1 built as testbed: sparse versioned model profile where missing never equals zero; comparability only for same attribute, context and dataset; evidence levels L0-L4; Pareto set with evidence counts; plan gate with `SHA-256` plan hash and baseline arm; null-agent check; range noise rule with three-state verdict; fail-closed reporting. Constrained model-to-task-part selection (Eq. 11) is planned, not built.	Migration fully-correct 98 to 108 of 216 (45.4 to 50.0%, Fisher p=0.39); accepted-but-wrong or unverifiable 9.7% to 21.9% (p=0.023); about 78% of ok verdicts proved nothing. Breadth: agent 91.9% vs cheap 90.5% (p=0.21); agent used 3.0% fewer tokens. Automation: model effect p=0.007, framework p=0.40; tokens 1,953-9,465; CrewAI about 4x. Judge churn 16.2% at T=0; full run $3-4.	Stage 1 only; model-vs-framework comparisons are non-causal separate sweeps. Migration inference unpaired, no multiplicity correction. Only P1 and chatbot integrated as probes; Grafana JSON never imported. Adapter-generation hook unevaluated. 15 automation samples unreadable. Caller key crosses a process boundary unprotected. Range rule loses power as repeats grow.	A:047
`LLM-assisted` **labeling then fine-tuned small NER model (** `GLiNER`**, SpanMarker)**	Uses a large LLM to label data, then fine-tunes a compact model for the task and deploys it on a cheap endpoint.	Up to 6.4% accuracy gain and up to 80x cheaper inference than large LLMs for financial NER.	Needs labeled data and training; narrow task, not general agent work.	A:129
Retained reasoning plus native compaction, programmatic tool calling, low reasoning effort, small-tier model (Luna) selection	Persists reasoning across turns with compaction, runs tool orchestration in code outside context, and uses cheaper tiers or lower effort.	`ARC-AGI-3` 13.3% to 38.3% with about 6x fewer output tokens; BrowseComp 84.04% at $1.33 versus 84.36% at $33.27 for GPT-5.5; PlayerZero cost -64%.	Vendor and customer testimonials; harness-specific.	A:207
OpenRouter provider routing (effective price with real cache hit rate, sort, max_price, :floor/:nitro, fallbacks)	Picks a provider per request by measured effective price reflecting its cache hit rate, throughput, latency and reliability; supports sort, max_price and fallbacks.	Not measured; reacts to provider changes within minutes.	Provider choice not model choice; bypassed when order or own keys set.	A:227
Rule-first classifier plus decision model with probability threshold (0.85) routing fast versus frontier tier, per-tier API key budgets	Explicit rules first, then a decision model picks tier; fast tier accepted only if probability >= 0.85, else frontier; `AI` Gateway fallbacks and spend caps per tier.	Not measured; threshold 0.85 given as an example.	Experimental Decision API; threshold needs tuning.	A:245

`RLM-Cascade` (complexity router, cheap draft model, Opus verify or skip)	Local proxy (ANTHROPIC_BASE_URL) with a rule-based O(1) router, no model calls. Requests with a non-empty tools field go straight to Opus. Simple prefixes go to the cheap model only (`SKIPPED`). Complex keywords or input over 1,500 chars get cheap draft then Opus verify (USE_DRAFT = `ACCEPTED` , or `ENHANCE`).	Savings 45.8% (service, N=125), 47.4% (architecture set, N=39), 32% (current run, N=12); model predicted 67.2% vs 47.4% observed. p50 latency 2,026 vs 3,698 ms. Quality 100% vs 95% pass (N=20).	Tool-selection turns (70-80% of Claude Code turns) bypass to Opus, so savings apply to the ~20-30% text turns. Router is a keyword classifier. One cost inversion (+$0.01189). TTFT 2.1x worse. Learned router, parallel draft+verify, local tier are future work.	C:031
Cache-aware cost comparison with TCO model and hybrid routing counterfactual replay	Longitudinal case study: Langfuse and vLLM telemetry plus Git mining. Fix Commit Ratio via regex over commit messages; difficulty-stratified Mantel-Haenszel odds ratio; TCO = M x (compute + ops + T_iter x wage); hybrid-routing offline counterfactual replay by difficulty tier.	99.3% cache hit cut API cost 88.6% to $0.57/M tokens vs $2.83/M shared on-prem; `FCR` 74.9% vs 45.9%; Mantel-Haenszel `OR` 3.61; Shared on-prem saves 40.1% true TCO; dedicated costs 43.8% more; High-to-Claude hybrid saves 26.3%	One developer, non-randomized, tasks differ between periods; keyword-classified fixes; Period A fix count inflated by CI work; replay assumes tier exchangeability.	C:053
`PORT` : training-free online routing via `ANN` query-feature estimation plus one-time dual optimisation	`PORT` : kNN (5 neighbours) estimate perf and cost; random-route first eps=0.025 queries; solve dual gamma* once; route to argmax(alpha*d - gamma*_i*g); queue when budget exhausted. Competitive ratio 1-O(eps+delta).	Avg 3.55x performance, 1.85x cost efficiency, ~4.25x throughput. Vs BatchSplit +33% perf, +38% cost efficiency. Reaches 75.99-84.66% of approximate optimum.	Only perf and cost modelled; needs labelled history; assumes similar queries behave alike and random order.	A:007
RouterDC: encoder plus LLM embeddings trained with `sample-LLM` and sample-sample contrastive losses	RouterDC: encoder plus learnable LLM embeddings, trained with `sample-LLM` and sample-sample contrastive losses; route a query to the LLM whose embedding is most similar. Clustering similar queries stabilises training.	Average 58.54 vs 54.56 (+3.98) in-distribution; `OOD` 45.85 vs 43.95 (+1.90).; About 6x faster than voting.	Needs scored training data per candidate LLM. Limitations section only says scenarios are left to future work.	A:015
Self-MoA and sequential Self-MoA (aggregate samples from one top model)	Self-MoA samples N outputs from the single best model (temperature 0.7); one aggregator synthesises them. Self-MoA-Seq aggregates with a sliding window of 6 responses. Regression of mixture performance on quality and Vendi-score diversity.	AlpacaEval `LC` : Self-MoA 65.7 vs Mixed-MoA 59.1 (+6.6); Quality coefficient exceeds diversity coefficient (MMLU 2.558 vs 1.841); Self-MoA with TaskBest averages +3.8%	Mixed-MoA wins when models are similar in quality and complementary; gains not monotonic in samples; aggregator context limit.	A:032
Bayesian aggregation of `multi-LLM` likelihoods with expected-utility and value-of-information gating	Elicit p(evidence|state) from M LLMs via contrastive 'assume state s' prompts, aggregate by median, apply Bayes with deployment prior, choose action by minimum expected cost, gather more evidence if value of information exceeds its cost or models disagree.	$294,000 (34%) lower total cost vs best `single-LLM` ; parity gap 22 to 5 points. Ablation shares: `multi-LLM` 51%, sequential 43%, disagreement-triggered screening 20%. 20 LLM queries per observation (M=5, K=4).	Synthetic data and simulated screens; latency of 20 queries per step; cost matrix assumed; model `IDs` and versions inconsistent between abstract and appendix.	A:040
Harness-level cost and Pass@1 benchmark with a cost-calibrated Lite-80 subset	Adapter protocol (create_agent, send_task, backup_session, delete_agent, get_docker_args); patch exported from git diff; history after base_commit removed to stop leakage. Lite-80 subset picked by quartile-quota local search with ranking hinge and cost-parity filter.	Bare adapter 19.3% vs full adapter 71.2% resolved; largest harness gap at fixed model 24.3 pp; Best Pass@1 Hermes Agent 81.7% at $36; Lite-80 vs full: `MAE` 0.81 pp at 25.3% of cost	Meta-Harness evolved and evaluated on same instances; Lite is a proxy; appendix F is single-run.	A:060
Shapley-value cost attribution of thinking tokens and turns; price reversal	Cost c = p_in*n_in + p_out*n_out (+ cache terms for agents). Shapley-value attribution of the cost gap between two models over eight multilinear factors (turns, input, output, cache, thinking tokens, and three prices), with a closed form. Cost treated as a distribution with quantile-level reversal analysis.	106 of 336 pairs (32%) show price reversal; 11% on ArenaHard to 57% on `MMLU-Pro` ; Gemini 3 Flash: 80% lower listed price than GPT-5.4 but 38% higher total cost ($705 vs $509); Single-turn: thinking tokens over 95% of the gap; agentic: turns over 80% (Kimi vs GPT-5.4); Repeat-run cost on the same query varies up to 9.7x	Costs only, no quality; one price snapshot (May 2026); cost prediction left open; no limitations section.	A:069

Cost-of-pass and frontier cost-of-pass metric	cost-of-pass v(m,p) = C_m(p)/R_m(p), the expected cost of one correct answer (assumes independent trials). Frontier cost-of-pass V_p(M) is the minimum over models plus a human-expert baseline. Gain is the reduction when a model is added; family essentialness is the relative frontier increase when a family is removed. Exponential decay fit gives a halving time.	Frontier cost-of-pass halves every 2.6 months (`MATH-500`) and 7.1 months (`AIME24`); Essentialness: lightweight 93.5% (two-digit addition); reasoning 74.4% (`MATH500`) and 81.0% (`AIME24`); large models matter on knowledge tasks (`GPQA` 33.3%, `BBQ` 44.7%); Majority voting yields 0% relative gain everywhere; self-refine 24.9% on `GPQA` ; `TALE-EP` 66.6% on `GSM8K`	Binary pass/fail and expectation only (no variance); ignores verification, retry and tool costs; contamination possible; single prompt setting; i.i.d. retry assumption is contested (A:123); only 16 Tau-bench tasks.	A:122
Task-based auto model routing along cache boundaries with 10% credit discount	Scores the task on reasoning, code complexity, bug difficulty and tool needs plus model health, then picks a model; routes along cache boundaries.	Paid users get 10% fewer `AI` credits; vendor says token efficiency improves with no quality regression (no numbers).	Vendor claim; model changes over time.	A:155
Flex service tier (service_tier: flex)	Tokens priced at Batch API rates, plus caching discounts, in exchange for slower responses and occasional resource unavailability.	Batch-rate pricing; no single percentage stated in the part read.	Beta, limited models; 15-minute timeout advised; resource-unavailable errors.	A:195
Decision-model request rating with cost_tier policy and cache-aware model switching	A fast model rates eleven aspects of the request, then picks model and effort against live prices and cache status within a low/medium/high cost tier.	Not measured here; says the new default costs more per task than the old router, and low tier restores prior cost.	Model switches cost a cache rebuild, so the router prefers effort changes over model changes.	A:228
`ACRouter` : Context-Action-Feedback loop with orchestrator, verifier and memory	Routing as a Context-Action-Feedback loop (contextual bandit with cumulative regret vs a per-task oracle, reward = score - 0.1*cost). `ACRouter` : Orchestrator (fine-tuned Qwen3.5-0.8B plus heuristics, DimensionBest prior and top-10 kNN memory neighbours, weighted vote), sandbox Verifier (`AST` , execution, tests, self-consistency, judge), `FIFO` memory of 20K entries.	Router diagnosis: adding per-dimension performance stats to Sonnet 4.6 router lifts 41.41 to 47.74 (+15.3%); `ID` : `ACRouter` 49.98% AvgPerf, cumulative regret 205.5, vs DimensionBest 47.50%; `OOD` : `ACRouter` 62.50% vs Always-Opus 57.14%; lightweight classifiers fall to 8.93-21.43%, below Random 31.25%; Perf/$ `ID` 3.79 vs Always-Opus 1.29; Opus costs about 12x Kimi-K2.5 on the probing set	Provider cache hit rates unobservable so dollar costs are secondary; agentic `OOD` uses a 40-step limit, 176 tasks; `ACRouter` pays more for Memory and Verifier than static routers; LLM policy plus kNN memory is one instantiation; routing is per task, not per step.	C:032
Cursor Router modes (Cost, Balance, Intelligence) and Cursor Token Rate	Router picks Cursor or third-party models per request by mode; third-party use is billed at API list price from prepaid credit.	Max Mode on legacy plans costs API rate plus 20%.	Pricing doc, not measured.	C:119
Architect/editor model split and per-task token budgets	R1 as architect with Sonnet as editor; notes code generation is 5-15% of tokens and Aider's tree-sitter repo map saves 20,000+ tokens per turn.	Aider polyglot: DeepSeek R1 + Sonnet $13.29 for 225 tasks, 14x cheaper than o1; Gemini 2.5 Pro $0.028/task at 72% pass.	Blog aggregation; real sessions 3-5x benchmark cost.	C:129
`MTRouter` turn-level outcome-estimator routing	`MTRouter` : `MLP` outcome estimator on [history embedding (frozen Qwen3-Embedding-0.6B, 8192 tokens); model embedding (8-d metadata + 16-d residual → 64-d)]. Target = terminal score minus progress-weighted error penalties (1.0/0.8/0.2). Argmax per turn, $2/episode cap.	ScienceWorld 53.8 vs GPT-5 48.4 at -58.7% cost. HLE 26.0 vs 23.8 at -43.4%. `OOD` ScienceWorld +5.0 at -65.8%; HLE `OOD` cost -52.3%. ~5 switches vs ~20 (Router-R1). Stays on same model after error 90.2%/80.9% vs 38.3%/66.4%.	Offline only; switching can lose prompt cache (authors note).	A:008
EmbedLLM encoder-decoder model embeddings with matrix-factorization router	EmbedLLM: model-embedding network maps each model to a vector; question embedded with all-mpnet-base-v2 plus projection; linear classifier on the Hadamard product gives a correctness score (`BCE` loss). Router picks the model with highest predicted correctness; linear regression on embeddings predicts benchmark accuracy.	Correctness forecasting 0.7409 (`MF`) vs 0.7152 (`KNN`) at 29K training questions. Routes 3,000 questions in 3.80 s on A100 (over 750/s, 15x the `causal-LLM` router). Training under 1GB `GPU` memory.	Only 112 models so embeddings are sparse; new model needs retraining; correctness labels only; not agentic.	A:016

Agent Forest sampling-and-voting	Agent Forest: sample N outputs from the same LLM, then majority-vote. Code tasks vote by `BLEU` similarity between samples; closed-ended tasks by answer frequency. Combinable with CoT, debate and other prompting. Hierarchical variant votes within sub-groups, then across groups.	Gains: `GSM8K` 12-24%, `MATH` 6-10%, Chess 1-4%, MMLU 5-11%, HumanEval 4-9%. Llama2-13B `GSM8K` .35 to .59; GPT-3.5 .73 to .85. Hierarchical: 21% to 31% homogeneous, 35% to 47% GPT-3.5+GPT-4. Table 5 single-run tokens 138-495.	Cost scales with N: every sample is a full generation. Curves stop at N=40. Gain peaks at mid inherent difficulty (steps 100-200) then tapers at 400. Debate on Llama2 HumanEval gave 0 to 0. Code voting relies on `BLEU`.	A:033
`SLM-first` heterogeneous agents with `LLM-to-SLM` conversion algorithm	Six-step `LLM-to-SLM` conversion: secure logging of non-human-facing calls; curate and filter data (10k-100k examples rule of thumb, `PII` removal); task clustering; SLM selection; specialised fine-tuning (LoRA/ `QLoRA` , optional distillation); iterate and retrain including the router.	7B SLM 10-30x cheaper than 70-175B LLM in latency/energy/ `FLOPs` (cited). Replaceable query share 40-70% across three agents (authors' estimates, not measured). SLM defined as under 10B params.	Authors list opposing views: LLM generalists win on general language, centralisation economies of scale ('jury is still out'), equally possible worlds. Barriers: infrastructure investment, generalist benchmarks, awareness.	A:041
Token price half-life and cost decomposition analysis	Fit exponential price-decay half-lives per price tier, test structural break (Chow), compute Malmquist productivity and `DEA` efficiency, and regress inference price on training cost.	Half-lives: economy 1.10y (R2 0.192), mid 1.55y (R2 0.307), flagship 3.59y but R2 0.031 so not exponential. Reasoning premium averages 31.5x, range 0.51x to 83.3x. GPT-3 $60/M (2020) to Gemini 2.0 Flash $0.10/M (2025-02) is 600x.	Only 62 milestone observations, no transaction volume, fast-moving market, low R2 in each tier.	A:061
Dollar-denominated economic evaluation (cost of mistake, latency, abstention)	Reward r = -(C + lambda_L*L + lambda_E*1_error + lambda_A*1_abstain): prices of error, latency and abstention. Pick the model or system maximising expected reward, for a standalone LLM, a cascade, or a router with abstention. Cascade error reduction `CER` = Cov(deferral, small-model error), related to `AUARC` .	Reasoning models cost 10-100x more per query, up to 10x slower, but win once error price exceeds about $0.01 (abstract); Single Qwen3-235B beats a cascade once error price exceeds $0.10; Llama3.1-405B to Qwen3-235B cascade beats the big model up to error price $10,000 and latency $10/minute; Cost and latency are linear in deferral rate	`MATH` only; possible contamination; judge model may inflate 405B's self-verification; no separate limitations section.	A:123
Total-cost-of-ownership framing: tokens per task, calls per action, retries, routing	Argues cost per task depends on tokens consumed, model calls, routing, context size and retries, not token price.	Cites Gartner `AI` spend $2.52 trillion in 2026, +44%.	Vendor marketing; no technique numbers.	A:180
Premium request multipliers per model	Each prompt costs one premium request times a model multiplier; autonomous tool calls do not count.	Copilot code review multiplier 13 from 2026-06-01.	Legacy plan billing only.	A:210
Realized price-latency-throughput-reliability provider routing, cache-aware	Scores each endpoint on realized price, wait time and error rate, shifts traffic from failing endpoints, stays cache-aware.	TTFT -16.9%, throughput +16.3%, capacity errors -46%, cost +6.0%.	Costs rose 6% for speed and reliability.	A:229
Asynchronous batch API at up to 50% discount	Submit a JSONL file of independent requests for 24h completion, separate rate-limit pool.	Up to 50% off serverless; under 1,000 requests usually finish in minutes.	Not for interactive or stateful work; one model per batch.	A:275
`FLARE` length-aware multi-objective router	`FLARE` length-aware router. Two `XGBoost` predictors give accuracy p^ and output length L^ from MiniLM-384 query embeddings plus model features. Latency = k_m*L^ + b_m, cost = pi_in*l_in + pi_out*L^. Filter models with p^ >= tau, then pick argmin alpha*T^ + beta*C^.	Up to 68% lower latency and 75% lower cost (Efficient mode, 5-20% accuracy sacrifice). `FLARE-Acc` keeps accuracy with up to 59% latency and 69% cost cut. Latency vs output length Pearson r=0.96. Routing overhead 18.12ms.	Short benchmarks, open models only, three preset modes.	A:009

`Model-SAT` aptitude-test router	Capability instruction = model capability representation + user instruction + Yes/No prompt. The representation is a natural-language aptitude profile from 50 MMLU tasks x 20-shot per candidate. E5-Large encoder, `MLP` connector, Phi-3-Mini 3.8B; score is P('Yes'). In-batch negatives, two-stage tuning, data refinement. Router never runs candidates.	Table 2 (`MMLU/ARC-C/TruthfulQA/mean`): `Model-SAT` 79.86 / 72.53 / 65.12 / 72.50 vs encoder-less Phi-3 74.71 / 70.58 / 61.70 / 68.83; High-performance zoo reaches 85.64 MMLU; Union of 5 small models covers 90% of GPT-4o's correct instructions; A new model needs only 50×20 inferences to onboard	Benchmark-style multiple-choice instructions, not agent traces; requires training; needs an aptitude test per new model; no limitations section.	A:017
`BELLA` critic-based skill profiling and budget-constrained selection	Four stages: benchmark models on cost and performance; a critic LLM extracts demonstrated skills, missing skills and criticality per task; skills are embedded, hierarchically clustered and labelled into a binary skill vector; select. Matrices C (model proficiency), R (task requirements), cost vector c. Pick M_capable = models with C_ms >= tau for all required skills, then max predicted performance subject to c_m <= B.	No empirical results; Motivating price gap: GPT-5 $10/M tokens vs Llama-3.3-70B $0.88/M (11x cheaper)	Not empirically evaluated (paper says it requires validation); critic-call overhead, clustering stability and critic variance untested.	A:042
Trained model router with Balanced/Cost/Quality quality-band modes	Router model scores prompt complexity and picks the cheapest model within a quality band: about 1-2% of best in Balanced, 5-6% in Cost.	Claims cost and latency savings at comparable quality; no figures given.	Context window limited by smallest pool model; auto-update changes pool and cost.	A:149
Dynamic route flow with conditional, percentage, rate-limit and budget-limit nodes	Versioned `JSON/visual` route sends segments to different models and falls back when a key's budget or rate quota is exceeded.	not measured	Gateway configuration only.	A:189
Premium-request model multipliers; 10% auto-selection discount	Each model consumes allowance by multiplier: Haiku 4.5 0.33, Sonnet 4.6 9, Opus 4.8 27; auto model selection earns 10% off.	Multiplier table only.	Legacy plans; subject to change.	A:211
Spend-share routing with cost_tier and sticky model reuse	Classifier assigns about 30 task types; picks models by 7-day community spend share within a cost band; keeps the conversation model to protect the cache.	Cost at default tier: MMLU Pro $141 vs $393 old, `SWE-Atlas` QnA $297 vs $464; accuracy about equal in most domains.	Vendor benchmark; some tasks regress or cost more, and switching rebuilds cache.	A:230
Confounder-gadget attack on LLM routers (adversarial robustness of routing)	Confounder gadget: a query-independent adversarial token prefix prepended to a query to push a binary router's score above its threshold so the query goes to the strong model. Hill climbing: 100 iterations, 32 candidate token swaps, 10-token gadget, early abort after 25 stagnant iterations. Black-box via surrogate router transfer; low-perplexity variant adds a perplexity penalty.	White-box upgrade rates mostly 100%, worst 88%; black-box lower but high; Unify: 49% strong-model share, upgrade 79-91%, cost $0.13 to about $0.22 (1.7x); OpenRouter: some gadgets rerouted all queries to GPT, about $0.25 vs $0.03 (over 8x); Perplexity filtering catches plain gadgets (97% `TPR` at 3% `FPR` for `RSW`) but low-perplexity gadgets drop `ROCAUC` to about 0.66-0.71	Binary routers and small or older models; mainly `MT-bench` for commercial routers; cost inflation measured on small query sets; no test against multi-model or agentic routers; one calibration scheme.	A:010
Distilled encoder classifier router (mmBERT-small, 7 heads: category, difficulty, stakes, needs_tools/vision/long_context)	One `CPU` pass classifies a request on seven questions to pick a model; priced at USD 0.001 per million input tokens.	Engine ~14 ms p50 on 4 cores; category accuracy 0.898 vs 0.845 for Jev hosted (vendor test, own data).	Vendor self-comparison; synthetic training data; input truncated to 512 tokens; text only.	A:018
Bedrock intelligent prompt routing (predicted response quality per model, with fallback model)	A router predicts each model's response quality per request within one family and picks the best cost/quality option, falling back to a default model.	Not measured in the page.	Same model family only; criteria and fallback must be configured.	A:158
Serverless vs dedicated endpoint break-even	Compare per-token serverless price against `GPU-hour` dedicated cost to find the crossover volume.	Example: H100 at $3/hour vs $1 per million tokens crosses at ~72M tokens/day.	Illustrative numbers; ignores idle and utilization variance.	A:277

Provider selection policy suffixes :fastest, :cheapest, :preferred	Appending a policy suffix to the model id picks the provider by throughput, lowest output price, or user order; no markup on provider rates.	Not measured.	Only provider choice, not model choice.	A:287
Cache-read at 5% of input on Opus 5.5 and Sonnet 5.5; Haiku 5.5 as cheap subagent	Price table: Opus 5.5 $4/$20 with cache read $0.20 and write $5 (5m) / $8 (1h); Haiku 5.5 steps price above 100K prompt tokens.	Haiku 5.5 over 100K still a quarter of Sonnet 5.5 rate.	Third-party blog; prices may change.	C:010
opusplan alias, subagent model setting, effort levels, organization effort caps	opusplan uses Opus in plan mode and Sonnet in execution; CLAUDE_CODE_SUBAGENT_MODEL sets subagent model; effort sets adaptive reasoning.	Not measured.	Risk of mismatch with effort after fallback model.	C:073

Test before enabling: Use `PGR/CPT` (x%) from RouteLLM: minimum strong-model share to reach a target quality, plus paired CPS on your own tasks. Not covered: Price Reversal (A:069) warns cheaper listed models can cost more (32% of pairs). Router attack risk (A:010). Corpus has no gateway difficulty classifier specific to coding.

13. Fork context duplication

A forked subagent inherits the parent prefix, so every fork re-sends the whole parent context.

API detection: Requests sharing a long identical prefix hash with a parent but different tails; cache_read versus cache_creation tokens in usage. CLI detection: SubagentStart hook with fork flag; transcript sidechain input tokens versus parent prefix size.

Technique	How it works	Effect	Loses when	Source
Fork subagent reads the parent prompt cache	Fork shares the parent prefix so the excess is cache-read priced; a non-fork subagent starts its own cache.	Excess is cheap unless the cache is dead.	After the 5-minute cliff the fork pays full creation price.	C:055, C:052: Claude Code subagent docs; prompt cache TTL analysis
Cache TTL study (stable prefix, dynamic content last)	Measured waits against the 5-minute TTL; 96% of waits fall in the cliff. Stable per-type prefix and dynamic content after it.	Combined fixes -13.6%; stable prefix -1.0%; dynamic after prefix -7.6%; blanket 1h TTL +8.6% worse.	Blanket 1h TTL costs more.	C:052: C:052 prompt cache analysis
Sub-agent context isolation	Run exploration in a fresh context and return a 1,000-2,000 token summary.	Net saving not measured in corpus.	Isolation cost can exceed the benefit for small tasks.	A:084: A:084 context engineering guide

Test before enabling: Compare fork on versus off on matched tasks measuring cache-read share, total cost and pass rate. Not covered: Net saving from subagent isolation is not in the corpus.

14. Large tool-call arguments

Long plans, whole-file writes and big shell heredocs are billed as output tokens at the higher output rate and then stay in history as input on every later turn.

API detection: In assistant response content, tokenise json of each tool_use input (non-edit, non-agent tools); flag above 1,000 tokens (hermes A66). Count against usage.output_tokens. CLI detection: Gateway signal only for size; `OTel` tool_result gives tool_name and OTEL_LOG_TOOL_DETAILS=1 adds parameters (C:002).

Technique	How it works	Effect	Loses when	Source
Aider edit formats (diff / udiff vs whole)	diff returns search/replace blocks and udiff a simplified unified diff, so output scales with the change, while whole returns a full copy of every changed file.	not measured in corpus (blog asserts the whole default can outweigh all other costs).	Weaker models fail diff formats; blog says use whole only when the model cannot do better.	C:127: How to Reduce Aider Token Usage (Tokenade blog)
Token-efficient tool use beta and text_editor tool	Header token-efficient-tools-2025-02-19 makes tool calls shorter in output tokens. The text_editor tool makes targeted edits to portions of a file instead of rewriting it.	Up to 70% output-token cut, 14% average, vendor-reported, Claude 3.7 Sonnet only.	Limited to one model generation in the source; vendor numbers, no method.	A:068: Token-saving updates on the Anthropic API, Anthropic
Context editing clear_tool_inputs	With clear_tool_uses, setting clear_tool_inputs=true clears the tool call parameters as well as results once the trigger is passed; the API replaces them with placeholder text.	not measured in corpus.	Invalidates the cached prefix, so set clear_at_least to make the rewrite worthwhile; cleared content is unrecoverable.	C:065, A:271: Context editing on the Claude Developer Platform, Anthropic; Context engineering: memory, compaction, and tool clearing (Claude cookbook)
DevSkills principle: persist and revise scripts (SimScrpt)	One developer-written principle tells the agent to persist and revise scripts instead of creating similar variants. The paper counts similar script generation as a cost-inefficient behaviour.	SimScrpt in up to 68% of tasks and up to 9.57% of task cost; all 7 principles together cut cost 7.88-41.73% in 6 of 8 settings.	One setting lost 1.5 pp Pass@1; the authors own the winning method.	A:059: Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Edit-format and reasoning-effort cost/accuracy benchmark on polyglot tasks	Reports pass rate, edit format and total cost per model across 225 Exercism tasks.	gpt-5 high 88.0% at $29.08 vs medium 86.7% at $17.69; o3-pro high 84.9% at $146.32.	Single benchmark; cost shown at benchmark run time.	C:038

Test before enabling: Replay sampled traffic with the treatment, then a paired run per harness. Metric: billed cost per success, output tokens per task, Pass@1. For diff formats also track failed-edit rate, since a failed diff costs a retry. Not covered: Corpus has no measured share for plan-sized tool arguments beyond 1.4% of billed cost (C:024). No Codex or Kiro setting that limits argument size.

15. Agent startup cost

Each subagent starts by loading `CLAUDE.md`, git status, skills and a roster, paying that fixed cost per spawn.

API detection: First request of a derived helper conversation: system-prompt and tool-block token count, repeated across spawns. CLI detection: SubagentStart hook plus first-turn input tokens in the sidechain transcript.

Technique	How it works	Effect	Loses when	Source
Subagent startup context contents	Helper loads `CLAUDE.md`, git status, preloaded skills, sibling roster and tool descriptions; description tokens above 15k warn.	Size not measured per helper in corpus.	Dropping `CLAUDE.md` loses project rules.	C:072: Claude Code subagents documentation
Stable per-type prefix	Keep one fixed prefix per agent type and put dynamic content after it.	-1.0% stable prefix; -7.6% dynamic content after prefix (C:052).	Blanket 1h TTL is +8.6% worse.	C:052: C:052 prompt cache analysis
Prompt caching with cache_control	Mark the static prefix as cacheable so repeat spawns read it.	Not measured here for spawns.	Prefix change invalidates cache.	A:199: A:199 prompt caching documentation

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
--bare mode and --max-budget-usd cap for claude -p	--bare skips hooks, skills, plugins, MCP servers, auto memory and `CLAUDE.md` for faster startup; budget cap stops background work.	not measured (startup time claim only)	Bare mode drops project context and `OAuth`; only `API-key` auth.	C:094

Test before enabling: Measure first-turn tokens before and after omitting each startup item; check helper task success. Not covered: No per-helper startup size measurement in corpus.

16. Model switch rewrote the cache

Each model has its own cache, so a mid-session model change rewrites the whole prefix on the next request.

API detection: Rewrite where the model field differs from the previous call on the same key (cascade step 2 after compaction). CLI detection: Model per call in the transcript; PreModelSwitch hook events; /model events.

Technique	How it works	Effect	Loses when	Source
PreModelSwitch hook and model confirmation	A hook can require or skip confirmation; /model confirms only while the cache is warm.	Not measured in corpus.	Confirms but does not block intended routing.	C:055: Claude Code docs, How Claude Code uses prompt caching
Cache diagnostics reason model_changed	Providers report why a miss happened (model_changed, tools_changed, reasoning_effort_changed and others) against a baseline response id.	Not measured in corpus.	OpenAI GPT-5.6+ only.	A:204: OpenAI, Prompt cache diagnostics
DevX proposal: sticky model plus switch guard	Gateway pins model per conversation key and routes only at session boundaries. No corpus source.	Not measured.	Blocks cheap-model downgrades mid-task.	: DevX proposal (C:055 supplies the invalidation facts)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Auto Router: eligibility filter, difficulty classification, quality-vs-cost scoring, session affinity	Filters eligible models, classifies task type and difficulty, ranks by quality versus cost, falls back down the list; session-id header keeps one model per turn for caching.	not measured; sticky session lets requests reuse prompt cache.	Without session id, requests may switch models and lose cache; no savings number.	A:192
Task-based auto model routing along cache boundaries with 10% credit discount	Scores the task on reasoning, code complexity, bug difficulty and tool needs plus model health, then picks a model; routes along cache boundaries.	Paid users get 10% fewer `AI` credits; vendor says token efficiency improves with no quality regression (no numbers).	Vendor claim; model changes over time.	A:155
Decision-model request rating with cost_tier policy and cache-aware model switching	A fast model rates eleven aspects of the request, then picks model and effort against live prices and cache status within a low/medium/high cost tier.	Not measured here; says the new default costs more per task than the old router, and low tier restores prior cost.	Model switches cost a cache rebuild, so the router prefers effort changes over model changes.	A:228
Gateway failover and caching:'auto' with cache-invalidation warning	Gateway reissues failed requests to another provider and applies provider-specific cache markers; warns abstraction layers can silently invalidate cache hits.	3.5% of requests succeed only through failover, carrying 4.9% of spend; zero token markup.	A documented AutoGPT bug put a per-call random token in the cached prefix, voiding the 90% discount.	A:246
Per-message effort change without cache invalidation; cache price tiers	Fable 5.1 supports mid-conversation effort changes (beta) and preserved thinking across model switches; cache read $0.25 vs 5m write $12.50 and 1h write $20 per `MTok`.	Cache read is a quarter of the earlier price; input $10 vs Opus 5.5 $4 per `MTok`.	Forced tool use errors; editing earlier turns invalidates thinking blocks.	A:285
Fast mode with separate cache and switch cost	Fast mode is up to 2.5x faster at $8/$40 on Opus 5.5; first enable pays uncached full price and fast and standard keep separate caches.	Up to 2.5x speed at higher per-token cost.	Each switch rereads context uncached.	C:086
Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072
Spend-share routing with cost_tier and sticky model reuse	Classifier assigns about 30 task types; picks models by 7-day community spend share within a cost band; keeps the conversation model to protect the cache.	Cost at default tier: MMLU Pro $141 vs $393 old, `SWE-Atlas` QnA $297 vs $464; accuracy about equal in most domains.	Vendor benchmark; some tasks regress or cost more, and switching rebuilds cache.	A:230
caching:'auto', cache_anchor_items, cache_ttl, session affinity	Gateway adds cache markers per provider, pins a conversation to the provider holding its cache, and selects 5-minute or 1-hour lifetime.	not measured	Gateway lock-in; affinity limits failover.	A:248

Test before enabling: Compare CPS with and without a pinned model on paired tasks; count rewrites caused by switches. Not covered: No corpus source for a gateway-side guard.

17. Old replies replayed

The model's earlier prose answers are re-read on every later call though rarely needed.

API detection: Gateway: sum of text blocks in earlier assistant messages, times later cache-read price (hermes A6). CLI detection: gateway signal only.

Technique	How it works	Effect	Loses when	Source
Server-side compaction (compact_20260112)	Beta context edit that fires at `trigger` (default 150K, minimum 50K), summarises the conversation into a typed compaction block, accepts custom `instructions` and `pause_after_compaction`.	Cookbook: peak 335,279 to 169,164 tokens with a 2,783-token summary; high-level facts kept 3/3, obscure specifics 0/3. Single run. Figures are from A:271.	Loses verbatim detail; next turn rebuilds the cache; on short sessions the summary call costs more than it saves.	A:271, C:065: Context engineering: memory, compaction, and tool clearing (Claude cookbook)
`LLM-Summary` condenser	Every N turns an LLM summarises older turns into one block, keeping the last M verbatim (N=21, M=10 in the study).	Qwen3-Coder solve rate 53.8% vs 53.4% raw; summary cost up to 7.2% of instance; runs lengthen (52 vs 44 turns).	Costs more than masking; trajectory elongation.	C:130, C:133: The Complexity Trap (JetBrains Research); Memory Condensation Strategies (arXiv)
`LLMLingua-2` token-classification compression	A trained classifier keeps or drops each token of the prompt, with no LLM generation step.	Abstract: 3x-6x faster than prior compressors, 1.6x-2.9x end-to-end latency gain at 2x-5x compression. Cost saving not measured on agents.	Compressors saved nothing or lost in the one large agent study; keep off by default (C:024).	A:081, C:024: `LLMLingua-2`: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression (Tsinghua/Microsoft)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Learned gist tokens inserted at sentence ends	Add learned gist tokens (Ng in {1,2,4,8}) after every sentence-ending punctuation mark. Gist tokens see their own sentence plus earlier gist tokens; regular tokens see their sentence plus gist tokens. Standard LM loss. Three stages: gist embeddings only, full fine-tune, large-batch cooldown. About 4B FineWeb-Edu tokens.	Ng=4: about 6x average long-context compression vs about 2x for Activation Beacon; Ng=4 `HELMET` recall 96 vs base 100; icl 63.2 vs 68.2; Ng=1: recall drops to 1.0, `ARC` 49.77 vs 59.21	Rule-based placement, fixed budget per segment, punctuation sensitivity, does not fully recover base, full mask memory-heavy at 128K, only 3B tested.	A:086
Gist tokens via modified attention masks	Gist tokens: insert k gist tokens between prompt and input; mask so later tokens cannot see the prompt; trained by instruction finetuning at no extra cost. Meta-distillation L_G = `KL` (pLM(y|t,x)||pG(y|G(t),x)).	Up to 26x prompt compression, up to 40% `FLOPs` saving (vs no cache). k=1 sufficient. Win rate vs control `LLaMA` 48.6/49.7/45.8, `FLAN-T5` 50.8/46.2/42.5. vs instruction caching only 0.11% `FLOPs`, 1% wall time, but 26x more prompts cached per storage.	Loses nuance; needs finetuning; small models; short (~20-token) prompts.	A:082
Selective Context (self-information pruning)	A base causal LM (`LLaMA-7B`; GPT-3 curie for OpenAI models) computes self-information -log2 P(x_t|x_<t) sentence by sentence. Tokens are merged into lexical units (tokens, noun phrases via spaCy, sentences). Units at or above the p-th percentile of self-information are kept.	At ratio 0.5: `BERTScore-F1` drop .023, unfaithfulness .038. At ratio 0.2: `BERTScore-F1` .931 vs .899 random. Phrase-level units best. One vicuna-13b case: 36% memory (77,695 to 61,885 `MB`), 1.32x speed.	Needs phrase-boundary detection (no verb phrases). Best percentile varies by task and context. Quality drops noticeably from ratio 0.5 up. Never tested on code or tool output.	A:083

Gist-token context compression with fine-grained autoencoding and segment-wise token importance	Segment-wise gist compression: L tokens into t gist tokens (ratio L/t); compares memory location (recurrent vs KV) and granularity (coarse vs fine): Coarse-Rec, `Coarse-KV`, `Fine-KV`. Adds fine-grained autoencoding and segment-wise token importance estimation (`TIE`) to fix failures.	`Fine-KV` best, near-lossless on RAG, LongQA, summarization, `ICL` at low ratios; weak on rerank, synthetic recall and `BBH`. Autoencoding reconstruction under 20% at high ratios. At ratio 4 synthetic recall improved 52.7% and 33.7% by the two fixes.	Models up to 7/8B, 16K window; only compared with full attention; requires continued training and KV access.	A:085

Test before enabling: Paired replay: raw vs compaction with a recall probe of decisions made in earlier assistant turns; cost per successful task. Not covered: No corpus measurement isolates assistant-text replay; C:024 puts all history under 1% of billed cost.

18. Clarification loops

Each question the agent asks adds a turn and re-sends the context.

API detection: Assistant turns ending in a question with no tool call, followed by a short user reply. CLI detection: Transcript turns with no tool call ending in a question mark; Notification or elicitation events.

Technique	How it works	Effect	Loses when	Source
Persistence / bias-to-action prompting	Tell the agent to proceed and not ask unless blocked; give stop conditions.	Cost of asking not measured in corpus.	Wrong assumptions on ambiguous tasks.	A:208, A:205, A:249: Model prompting guides
Plan mode	Plan first, approve once, then execute.	Not measured.	Extra planning turn.	C:095, A:053: Claude Code docs; A:053
Interview-then- `SPEC.md`	One structured interview writes a spec, then a fresh session executes.	Not measured.	Upfront cost.	C:080: Claude Code best practices

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
One chat per coherent outcome with fork, compact, subagents and `AGENTS.md`	Plan mode before hard tasks, short `AGENTS.md`, /fork only when work branches, subagents for bounded work, one chat per outcome.	not measured	Qualitative guidance only.	C:098

Test before enabling: A/B bias-to-action prompt versus control on matched tasks: clarification count, cost, success. Not covered: Cost of clarification is not measured.

19. Screenshot accumulation

Every screenshot stays in context and is re-billed each turn until compaction, so old images keep costing input tokens.

API detection: Count image blocks in request messages per call; estimate tokens as w*h/750 (hermes formula); flag images older than N turns still carried. CLI detection: Transcript scan for image blocks in tool_result entries; PostToolUse hook counts screenshots per session and their age in turns.

Technique	How it works	Effect	Loses when	Source
Observation masking	Rolling window of M=10 turns keeps recent observations and replaces older ones with a placeholder.	About half the cost of the raw agent at a solve rate matching `LLM-Summary` (`SWE-bench` Verified, `SWE-agent`).	Tuned per agent (M=58 for OpenHands); paper tests text observations, so images are an extrapolation.	C:130: The Complexity Trap (JetBrains, arXiv 2508.21433)
Claude Code oldest-image eviction	When the next request would pass the image or PDF cap, Claude Code removes a batch of the oldest images and `PDFs` .	Costs one slower turn per batch while the cache reprocesses from the earliest changed message.	Claude can no longer see removed images.	C:055, A:071: How Claude Code uses prompt caching
Context editing: clear_tool_uses_20250919	Clears old tool results past a trigger (example 30000 input tokens), keeps the last N tool uses, and clear_at_least ensures each clear is worth the cache write.	not measured in corpus	Clearing invalidates the cache prefix; extracts do not say whether images are cleared.	C:065, A:268: Context editing on the Claude Developer Platform

Test before enabling: Paired A/B on the same tasks: baseline vs masking or downscale, metric cost per successful task, check task success does not drop before enabling. Not covered: no corpus source for image downscale or the w*h/750 formula. DevX proposal: Pillow downscale to a max edge before forwarding. Corpus does not confirm context editing clears images.

20. Visual iteration loops

Each edit-then-screenshot cycle adds a fresh image and a model turn, so a long loop multiplies image tokens.

API detection: Per session, count edit-tool-call then image-bearing tool-result cycles; flag more than 3 per task. CLI detection: Transcript or hook sequence: Edit/Write followed by a screenshot tool result; count cycles per prompt.

Technique	How it works	Effect	Loses when	Source
Observation masking	Keep the last M=10 turns of observations and mask the rest, so each loop iteration drops the prior screenshot.	About half the cost of the raw agent, matching `LLM-Summary` solve rate (`SWE-bench` Verified).	Masking loses at 7 events or fewer (C:133 as noted); text-only evidence.	C:130: The Complexity Trap (JetBrains, arXiv 2508.21433)
Claude Code oldest-image eviction	Removes a batch of the oldest images when the cap is near.	One slower turn per batch.	Earlier screenshots become invisible to the model.	C:055: How Claude Code uses prompt caching
Effort parameter	output_config.effort low to max changes tool-call count and verbosity; lower effort gives fewer tool calls.	not measured in corpus for visual loops	Behavioural signal, not a budget; run a sweep on your own evals.	A:282: Effort parameter (Claude)

Test before enabling: Replay recorded `UI` tasks with and without the cycle cap; compare success rate and cost per successful task. Not covered: no corpus source for a

visual-loop cap or the 3-cycle threshold. DevX proposal: threshold calibrated per task type.

21. Cache not reused

Writes never read back, missing caching, or a prefix below the minimum cacheable length mean the premium or full price is paid for nothing.

API detection: cw never followed by cr (A35); uncached input above 10k (A21); prefix below minimum with both cache fields 0 (A24). CLI detection: Same usage fields; headers show a gateway stripping cache_control if cw is always 0.

Technique	How it works	Effect	Loses when	Source
cache_control_injection_points	LiteLLM injects the cache marker at the system message or second-to-last user turn without client edits.	Cache hit costs about 10% of fresh input.	Needs a LiteLLM proxy; vendor blog.	C:060: LiteLLM, 5 ways to cut Claude Code costs with LiteLLM
Automatic caching and minimum-length rules	Top-level cache_control caches automatically; below the model minimum both cache fields are 0 with no error.	Minimums 512 to 4,096 tokens depending on model.	Silent failure; verify with usage fields.	A:199, C:051: Claude prompt caching docs; Start Debugging, Add Prompt Caching to an Anthropic SDK App
Prompt lengthening to cross the threshold	Extend a 900-token prompt to 1100 tokens so it can hit.	50% hit rate saves 33%, 70% saves 55%.	OpenAI case; extra tokens cost if hit rate stays low.	A:202: OpenAI, Prompt Caching 201

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Explicit cache breakpoints with per-breakpoint TTL (30m / 12h) and stable-prefix rules	GPT-5.6+: prompt_cache_breakpoint with ttl; write 1.25x for 30m, 2.5x for 12h, reads 0.1x; min prefix 1,024 tokens; dynamic content goes last; effort changes via appended config update.	Cache read 0.1x uncached input (0.05x for GPT-6.1 Sol/Terra).	Compaction or rewriting history resets reuse; 12h write costs 2.5x.	A:201
Time-at-end-of-message placement, prompt_cache_key, encrypted-reasoning carry-over, compaction	Keep timestamps out of the system prompt prefix, route each chat with prompt_cache_key, pass toInput() reasoning items so cache covers prior answers, compact with responses.compact().	Time at top of system prompt made the same chat cost 2.2-2.9x as much; run cost 56 cents total.	Turn after compaction misses cache; single example app, Grok pricing.	A:224
Gemini implicit and explicit context caching at 10% input price	Implicit caching is automatic; explicit caching declares content so the discount is guaranteed; minimum 2,048 tokens.	Cached tokens cost 10% of standard input on Gemini 2.5 and above; caches deleted within 24 hours (implicit).	Explicit cache adds storage cost and management; minimum length applies.	A:143
Session affinity for cache hits; cache and difficulty-aware routing; batch tier	Keeps an agent on the same server so its context stays cached; routing is cache- and difficulty-aware; Batch is 50% off.	Cache hit rates up to 99.6% in long agent sessions; cached input at half price or less; cache writes free.	Vendor marketing claims; a customer quote of $4M to $800K is anecdotal.	A:260
Implicit caching with 90% discount plus durable tier (24h max)	Prefixes are cached automatically with a 90% discount, falling to durable storage after eviction; put common content first.	90% discount on cached tokens; expiry no later than 24h.	Explicit cache adds storage fees; Gemini-specific.	A:280

Cache-aware cost comparison with TCO model and hybrid routing counterfactual replay	Longitudinal case study: Langfuse and vLLM telemetry plus Git mining. Fix Commit Ratio via regex over commit messages; difficulty-stratified Mantel-Haenszel odds ratio; TCO = M x (compute + ops + T_iter x wage); hybrid-routing offline counterfactual replay by difficulty tier.	99.3% cache hit cut API cost 88.6% to $0.57/M tokens vs $2.83/M shared on-prem; `FCR` 74.9% vs 45.9%; Mantel-Haenszel `OR` 3.61; Shared on-prem saves 40.1% true TCO; dedicated costs 43.8% more; High-to-Claude hybrid saves 26.3%	One developer, non-randomized, tasks differ between periods; keyword-classified fixes; Period A fix count inflated by CI work; replay assumes tier exchangeability.	C:053
prompt_cache_key and cache_control breakpoints with 5m/1h TTL via gateway	OpenAI auto-caches over 1024 tokens with prompt_cache_key for hit control; Anthropic allows up to 4 breakpoints with 5m or 1h TTL.	Up to 90% saving on cached tokens.	Gateway-specific; some providers 5-minute TTL only.	A:237
Gateway failover and caching:'auto' with cache-invalidation warning	Gateway reissues failed requests to another provider and applies provider-specific cache markers; warns abstraction layers can silently invalidate cache hits.	3.5% of requests succeed only through failover, carrying 4.9% of spend; zero token markup.	A documented AutoGPT bug put a per-call random token in the cached prefix, voiding the 90% discount.	A:246
Longitudinal gateway-log cost analysis of nine coding agents	Computes cost per active user and cached_tokens/input_tokens across a year of gateway traffic.	$92/month per active user, Claude Code $108 (P95 $291); 86% cache hit overall, 92% Claude Code; input per call grew 50K to 200K.	Single gateway's users; correlational.	C:054
prompt_cache_key, explicit cache breakpoints, 30m/12h TTL option	Stable prompt_cache_key per shared prefix, up to four explicit breakpoints per request, ttl 30m default or 12h; explicit mode avoids write charges when no breakpoint.	Cache writes charged only on GPT-5.6 and later; key beyond about 15 requests per minute may miss cache.	Write charges appear on newer models; 12h writes billed from 2026-11-01.	A:148
cached_tokens usage field for verifying cache hits	Read usage.prompt_tokens_details.cached_tokens: 0 is a miss, otherwise tokens reused; multi-turn shows it rising each turn.	Example 98 of 125 prompt tokens cached.	Verification only; no pricing figure read.	A:221
caching:'auto' two-marker placement and x-session-affinity header	Gateway places cache_control on the last message and the message before the last user turn; session affinity header keeps a conversation on the provider holding its cache.	Anthropic write 1.25x, read 0.1x; one follow-up read breaks even.	One-shot traffic pays write premium; affinity does not guarantee a hit.	A:247
Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072
Implicit versus explicit cache checkpoints; TTL resets on hit	Explicit checkpoints need minimum prefix tokens per model; static content first; TTL (many models 5 minutes) refreshes on each hit.	not measured in the pages read	Implicit caching is best effort; minimums vary by model.	A:157
prompt_cache_key for cache affinity; cached tokens at 10% price	Sending the same prompt_cache_key on prefix-sharing requests raises hit chance.	Cached input billed at 10% of standard price.	Hit not guaranteed; short or changing prefixes gain little.	A:173
Automatic prefix caching with x-grok-conv-id affinity header	Exactly matching leading messages are cached; setting x-grok-conv-id routes a conversation to maximise hits.	not measured in the page read	Exact prefix needed.	A:222
caching:'auto', cache_anchor_items, cache_ttl, session affinity	Gateway adds cache markers per provider, pins a conversation to the provider holding its cache, and selects 5-minute or 1-hour lifetime.	not measured	Gateway lock-in; affinity limits failover.	A:248

Prefix caching with x-session-affinity replica routing	Cache works only on one replica, so a per-user or session `ID` routes similar prompts to the same replica.	TTFT cut up to 80%; serverless default discount 50%.	Exact prefix only; per-replica.	A:257
Automatic prefix cache discount; batch inference up to 50% off	Cache is automatic, prefix-based and best effort; asynchronous batch jobs are discounted up to 50%.	Batch up to 50% lower cost.	No configurable retention; async only.	A:276
Automatic token caching for `API-key` and Vertex users; /stats cached-token display	CLI reuses system instructions and context under API key or Vertex; `OAuth` users get no caching.	not measured	Unavailable for `OAuth` personal accounts; CLI later replaced.	C:121
Automatic on-disk prefix cache with persisted prefix units	Each request persists prefixes at the end of user input and output; a hit needs full match of a persisted unit; common prefixes auto-detected after the second request.	Cache hit and miss reported per request; hits may need several seconds to build.	Best effort; no guaranteed hit; cleared in hours to days.	A:182
Prefix prompt cache with x-grok-conv-id routing header	Matching prefix is reused at reduced price; x-grok-conv-id steers requests to the same server to raise hit rate.	Cached tokens billed at a reduced rate (rate not in extract).	Not guaranteed; evictions and routing can miss.	A:223
Provider sticky routing and session_id for cache hits	After a cached request, later requests go to the same provider; session_id sets the sticky key; Anthropic needs explicit cache_control.	Sticky sessions expire after 60 minutes idle; only active when cache read price is lower.	Manual provider order disables stickiness; fallback loses cache.	A:231
Cached-input discount, sparse attention indexer and speculative decoding at serving level	Cached input priced at $0.14 per million (90% off); sparse attention picks 2,048 prior tokens; speculative decoding verified by the full model.	Peak 446 tok/s; typical 90k prompt with 600 turns mostly cached.	Vendor claim for one provider/model.	A:258

Test before enabling: Hit-rate wrapper from usage fields (C:051), then paired CPS A/B with injection on. Not covered: Gateway stripping cache_control is documented only in C:055; no measured prevalence.

22. Correction churn

A correction adds turns and a wrong attempt stays in context.

API detection: Correction regex on user turns; no corpus source for the regex. DevX proposal. CLI detection: Same correction matcher over transcript user turns; /rewind use.

Technique	How it works	Effect	Loses when	Source
Two-failed-corrections then /clear	After two failed fixes, clear context and restart with a better prompt.	Not measured.	Loses context.	C:080: Claude Code best practices
/rewind restore	Roll back to before the wrong attempt.	Not measured.	Discards work.	C:087: Claude Code checkpointing
TokenEconomizer micro-skills	Short skills that encode repeated fixes.	-11.8% and -14.3% in two settings.	Benchmark only.	C:028: TokenEconomizer
DevSkills	Reusable guidance from past corrections.	See row 31.	Benchmark only.	A:059: A:059

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Trajectory test-outcome vectors by language	No new method; diagnostic study. Each intermediate solution is re-executed to get a test-outcome vector; transitions labelled breakthrough/stay/improved/regressed/churn/broke/stuck; spans labelled with 12 behaviors by GPT-5-Mini; mixed-effects model on output tokens with problem as random effect.	`OCaml` uses 1.28-1.69x Python tokens (p<0.001). Gemma on `OCaml` : 1,196 stuck self-loops, 92% of failed snapshots are compile errors. Gemma/Qwen keep working after the first correct solution; over 70% of their spans into `STAY` are bare writes with no test run.	One scaffold, competitive-programming problems only, 4 languages, 5 models; may not generalize to repo-scale work.	C:134

Test before enabling: Track correction rate before and after `CLAUDE.md` additions. Not covered: CORRECTION_RE text is not in the corpus.

23. Agents racing the cache

Parallel helpers that start within seconds each write the same prefix because the cache entry exists only after the first response starts.

API detection: Calls with the same prefix hash within 10s, both cw above 5k. CLI detection: Subagent start timestamps and first-call usage per session_id.

Technique	How it works	Effect	Loses when	Source
Wait for first response before parallel requests	The entry becomes available only after the first response begins; send the rest after.	Not measured in corpus.	Adds latency to the fan-out.	A:199: Claude prompt caching docs
Workflow fan-out hold	Holds all but the first agent up to 5s so they read its cache.	Not measured in corpus.	Beyond 5s the race returns.	C:055: Claude Code docs, How Claude Code uses prompt caching
Batch caching: 1h block plus one shared-prefix request first	Best effort; send one shared-prefix request first, using a 1h block.	Not measured in corpus.	Best effort only.	A:199: Claude prompt caching docs

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
`KV-aware` routing, agent hints (nvext) and selective KV retention	Harness sends priority, output-length and prefill hints; global index routes to workers with highest cache overlap; ephemeral subagent KV marked for early eviction.	Claude Code calls hit 85-97% cache; agent teams 97.2% aggregate hit rate; 11.7x read/write ratio.	Needs a Dynamo-based serving stack, not hosted `APIs` .	A:167
Asynchronous Batch API (JSONL, 24-hour window)	Submit a JSONL file of requests that completes within 24 hours with a separate higher rate-limit pool.	50% lower cost than synchronous calls.	Not interactive; up to 24h delay; 200 `MB` file limit.	A:111

Message Batches API and Token Counting API	Batches run asynchronously at a 50 percent cost reduction; `POST` /v1/messages/count_tokens estimates tokens before sending.	50% batch discount.	Page is mainly an index; no usage detail.	A:220
Batch API plus automatic prompt caching on batch jobs	Async JSONL jobs at 50% off serverless prices, with cached tokens an additional 50% cheaper if static content comes first.	50% off plus 50% off cached tokens.	Not every model supports batch; jobs can stay pending.	A:255
prompt_cache_key, explicit cache breakpoints, 30m/12h TTL option	Stable prompt_cache_key per shared prefix, up to four explicit breakpoints per request, ttl 30m default or 12h; explicit mode avoids write charges when no breakpoint.	Cache writes charged only on GPT-5.6 and later; key beyond about 15 requests per minute may miss cache.	Write charges appear on newer models; 12h writes billed from 2026-11-01.	A:148
caching:'auto' two-marker placement and x-session-affinity header	Gateway places cache_control on the last message and the message before the last user turn; session affinity header keeps a conversation on the provider holding its cache.	Anthropic write 1.25x, read 0.1x; one follow-up read breaks even.	One-shot traffic pays write premium; affinity does not guarantee a hit.	A:247
Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072
Automatic on-disk prefix cache with persisted prefix units	Each request persists prefixes at the end of user input and output; a hit needs full match of a persisted unit; common prefixes auto-detected after the second request.	Cache hit and miss reported per request; hits may need several seconds to build.	Best effort; no guaranteed hit; cleared in hours to days.	A:182
Provider sticky routing and session_id for cache hits	After a cached request, later requests go to the same provider; session_id sets the sticky key; Anthropic needs explicit cache_control.	Sticky sessions expire after 60 minutes idle; only active when cache read price is lower.	Manual provider order disables stickiness; fallback loses cache.	A:231

Test before enabling: Measure duplicate cw tokens before and after a hold on a fan-out workload. Not covered: No corpus source for a gateway lock; labelled DevX proposal.

24. Too much tool output

One large Read, MCP or fetch result enters history and is paid again at cache-read price on every later call until compaction.

API detection: Tool_result block tokens in the next request body for non-Bash tools; flag above 4,000, charge tokens minus 2,000 (hermes BIG_RESULT/BIG_KEEP, unvalidated). CLI detection: Gateway signal only for size; PostToolUse hook gives tool_response and `OTel` tool_result gives tool_name and duration (C:002).

Technique	How it works	Effect	Loses when	Source

Tool-response pagination, range selection, filtering, truncation	Anthropic's tool-writing guide advises combining pagination, range selection, filtering and truncation with sensible defaults, and steering the agent with the truncation message. Claude Code caps tool responses at 25,000 tokens.	Concise vs detailed Slack response: 72 vs 206 tokens in the guide's example.	Truncation can cut needed content; the guide says to give helpful steering text.	A:284: Writing effective tools for `AI` agents, Anthropic
Observation masking (rolling window M)	Replace tool observations older than the last M turns with a placeholder, keeping all reasoning and actions: o'_i = p_i if i < t - M, else o_i.	M=10 cut Gemini 2.5 Flash cost $0.41 to $0.18 (-56.1%); Qwen3-Coder solve rate 54.8% vs 53.4% raw; DiscoveryBench: +8.6% token saving, only method that saved.	At 7 events or fewer it lost 45% to 149%; window needs tuning per scaffold; `SWE-agent` only.	C:130, C:133: The Complexity Trap: Observation Masking vs LLM Summarization (JetBrains Research); Evaluating Memory Condensation Strategies for Coding Agents in Data-Driven Scientific Discovery
Server-side tool-result clearing (clear_tool_uses_20250919)	When input tokens pass a trigger (default 100,000; docs example 30,000) the API clears the oldest tool results, keeps the last N (default 3), and honours clear_at_least and exclude_tools.	Cookbook, one synthetic task: peak context 335,279 (file reads 96.3%) down to 173,137, no dollars measured.	Clearing invalidates the cached prefix; single run; reads not re-fetched are lost.	C:065, A:271: Context editing on the Claude Developer Platform; Context engineering cookbook, Anthropic
Codex tool_output_token_limit	Config key setting the token budget for storing each tool output in history; default 12,000.	not measured in corpus.	Smaller cap means less is kept and large files get re-read after compaction (issue 16812 pattern).	C:104, C:106: Codex configuration reference, OpenAI; Mastering Context Compaction in Codex CLI (Vaughan blog)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Retained reasoning plus native compaction, programmatic tool calling, low reasoning effort, small-tier model (Luna) selection	Persists reasoning across turns with compaction, runs tool orchestration in code outside context, and uses cheaper tiers or lower effort.	`ARC-AGI-3` 13.3% to 38.3% with about 6x fewer output tokens; BrowseComp 84.04% at $1.33 versus 84.36% at $33.27 for GPT-5.5; PlayerZero cost -64%.	Vendor and customer testimonials; harness-specific.	A:207

Test before enabling: Sweep the cap at 6k, 12k and 25k on replayed traffic; metric billed cost per success, re-read count and turn of first edit (C:024 design: paired, provider-billed, cache reset between arms). For masking sweep M at 5, 10, 20 per scaffold. Not covered: No corpus measurement of billed saving from a size cap. Corpus holds no Kiro setting. `LLMLingua` on agent tool output is untested in corpus (A:079 covers prose prompts).

25. Noisy and repeated terminal output

Build, install, test and lint output is mostly passing lines and repeats, but it sits in history and is re-billed every turn.

API detection: Bash tool_result above 4,000 tokens; duplicate-line share above 0.5 on 20+ normalised lines; repeated traceback key (md5 of last 5 normalised lines) or repeated linter result hash (hermes A87, A82, A84). CLI detection: Gateway signal only; PostToolUse hook receives the command and output; Aider /tokens shows the context breakdown (C:127).

Technique	How it works	Effect	Loses when	Source
PreToolUse hook rewriting the command (updatedInput)	Hook inspects tool_input.command for npm test, pytest or go test and returns an updated command piped through grep -A 5 for `FAIL / ERROR` /error: and head -100. The doc also describes a hook that greps `ERROR` from a 10,000-line log.	Doc says tens of thousands of tokens down to hundreds; no measurement.	Can drop critical lines: a pytest rule had 56.1% marker recall at 31.9% saving before a fix; only one hook may rewrite the same input.	A:215, C:090, C:024: Manage costs effectively (Claude Code); Automate actions with hooks (guide), Anthropic; Token Reduction Is Not Cost Reduction
`RTK` deterministic CLI proxy and Headroom API proxy (measured, off by default)	`RTK` is a PreToolUse hook that routes shell commands through a CLI proxy with strategy-based compression; Headroom sits at ANTHROPIC_BASE_URL and optimises payloads in flight.	`RTK` -2.7% billed cost (not significant); `RTK-ML` cut 38.4% of tool-output tokens but cost +6.8%; Headroom +48.4%; Codex Headroom -12.49%.	Sign flips by harness; compression delayed the first edit and raised repeated searches; compressed grep fed to a shell pipeline gave silently wrong counts.	C:024, C:028: Token Reduction Is Not Cost Reduction (Weinberger and Hozez, PointFive); TokenEconomizer (Li et al., University of California, Berkeley and Microsoft Research)
Preservation-first filter rules with marker recall gate	Score a filter by savings and by the share of task-critical marker lines kept. The pytest rule was changed to preserve traceback lines.	Original rule 31.9% savings at 56.1% recall; fixed rule 15.3% savings at 99.2% recall, 24 pytest observations.	Halves the nominal saving; interactive task success was not run.	C:024: Token Reduction Is Not Cost Reduction
Delegate verbose operations to subagents	Running tests or reading logs in a subagent keeps the verbose output in its context and returns a summary to the main thread.	not measured in corpus.	The subagent's own requests still bill; no shared cache with parent.	A:215, C:055: Manage costs effectively (Claude Code); How Claude Code uses prompt caching

Test before enabling: Per C:024: paired runs, billed cost per successful task, cache reset between arms, plus 99% marker recall on a labelled set of failing logs before any filter goes live. Prove the hook is active (byte-diff output) before reading results. Not covered: No corpus source for build or install output (xcodebuild, npm install) filtering. Vendor 75-80% claims for git/ls/grep are unmeasured (C:024).

26. Repeated tasks across sessions

The same request is solved again from scratch in a later session of the project, paying the whole exploration again.

API detection: Gateway: first user message of a conversation key with word overlap 60%+ against an earlier conversation of the same caller/project (hermes #15), flagged as spend-to-review. CLI detection: UserPromptSubmit hook first prompt per session (C:074) with project id; same overlap rule.

Technique	How it works	Effect	Loses when	Source
TokenEconomizer experience-grounded micro-skills	External controller condenses past trajectories into digests, an LLM proposes micro-skills with regex detectors, activates one only after confirmation on several tasks with non-overlapping edited files and a need-rate threshold, retires it when a trace contradicts it.	Token cost down 18.4-30.9% across Claude Code, Codex, Copilot CLI with comparable pass@1 (within 3 points); cumulative savings positive by task six.	Costs more on early tasks; micro-skills go stale; tasks were independent `SWE-Bench` issues.	C:028: Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer; Berkeley/Microsoft)
Semantic cache (Portkey simple and semantic modes)	Gateway returns a stored response when a new request's embedding matches a cached one; simple mode needs an exact match.	not measured for agent tasks in corpus	Wrong-answer risk when the repo changed; targets repetitive single-turn chat.	A:240, A:113, A:115: Simple and semantic cache (Portkey); GPT Semantic Cache; MeanCache

Auto memory / `CLAUDE.md`	Persist solved procedures as notes that load next session.	Memory run: session 2 peak 172,623 vs 333,977 tokens without memory (4 of 8 documents re-read).	Single synthetic run.	C:077, A:271: How Claude remembers your project; Context engineering cookbook

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
LiteLLM budget caps, Langfuse, `LLMLingua`, RouteLLM, `GPTCache` semantic cache	Roundup: LiteLLM max_budget per key; `LLMLingua` prompt compression; RouteLLM routes between two models by threshold; `GPTCache` semantic caching.	RouteLLM 85% cost reduction keeping 95% GPT-4 quality on `MT` Bench (45% MMLU, 35% `GSM8K`); LongLLMLingua +21.4% RAG with 1/4 tokens.	Vendor blog citing others' numbers; RouteLLM is binary routing only.	C:061
Exact-match edge response cache with cache seed	Hashes URL, body and headers into a key stored on Cloudflare edge; default max-age 7 days; bucket size for multiple responses.	not measured	Exact match only; any parameter change misses; aimed at dev and test.	A:238
Semantic cache eviction via combinatorial bandits (`CUCB-SC`, greedy)	Loss = sum p(q)*min{c(q), d(q,M)}; `NP-hard`, non-increasing supermodular. Reverse_Greedy (curvature bound); offline `CUCB-SC` (pessimistic cost `UCB`); online `CLCB-SC-LS` (`LCB`, stage-based low switching).	Regret O(sqrt(mT log(mT)) loglogT), switches O(m loglogT). >=11.75% regret gain (up to 54.04%), switches -90.91%, runtime -85.40%.	Synthetic only.	A:114
Session-id cost attribution, custom property segmentation, graduated budget alerts (50/80/95%), response cache	Headers tag requests by session and property; alerts at 50%, 80%, 95% of budget; Cache-Control max-age enables response caching.	Example dashboard: 73% hit rate saving $1,247 a month (illustrative).	Gateway-dependent; example figures.	A:239
Codex compaction backends (default summarization, remote, remote_v2) and two-phase memory consolidation	Auto-compaction at model_auto_compact_token_limit; memory phase 1 extracts raw memories, phase 2 consolidates into `MEMORY`.md and summaries.	Not measured.	Reverse-engineered reading of one Codex version.	C:047

Test before enabling: Cross-session sequence test as in C:028: ordered task stream per repo, cumulative cost vs default, pass@1 within 3 points, report break-even task index. Not covered: No corpus technique detects 'same task' beyond word overlap; semantic caching evidence is single-turn only.

27. Failed, retried and polling calls

Repeated failing attempts keep re-sending growing context with no progress.

API detection: Repeated near-identical tool-error results and requests in one derived conversation. CLI detection: Failure-signature normalization over transcript tool errors; repeat count.

Technique	How it works	Effect	Loses when	Source

TokenEconomizer failure-loop guard	Normalize failure signatures, set turn budget Bi = beta times Q_p, rule trigger then LLM judge, redirect then abort; forced stop at kappa=2.5.	17% of tasks and 32% of spend failed; 51% of spend after first repetition; guard alone gives single-digit savings.	36 of 50 successful runs also repeat a signature.	C:028: TokenEconomizer
Turn-control (fixed p75, dynamic limits)	Cap turns at a percentile, then grow dynamically p25 to p50.	24-68% with fixed p75, further 12-24% dynamic.	Capability wall on hard tasks.	C:025, C:132: Turn-control study
Rate-based circuit breaker	Trip when error rate exceeds a threshold.	Not measured here.	False trips.	C:062: C:062
Actionable tool errors and constrained decoding	Clear error messages; constrained decoding for schema-invalid output.	Not measured.	Summaries mask failure signals (C:130).	A:284, A:098, A:097, A:059, C:130: Tool design guide; constrained decoding papers; A:059 ReTest

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Decoupled controller model approves extra token budget from workspace snapshot	Worker freezes at a 50,000-token limit; a separate controller model scores approve-vs-deny with expected value p>1/2 using task, files, tests snapshot.	Injected budget counter referenced 0 times in 14,000+ messages; request_more_budget used 0 of 5,000 turns; approval 97% default, 79% neutral.	Controller models biased to approve (four of six); 100-task evaluation set.	C:136
`SLM-default` , `LLM-fallback` with uncertainty routing, verifiers, Cost per Successful task (CPS)	`SLM-default` routing: small model with guided decoding and validators handles the task; uncertainty or validation failure escalates to a large model. Track Cost per Successful task (CPS) and schema validity; rollback thresholds trigger fallback.	Body claims 10-30x lower cost; abstract says 10x-100x (inconsistent).; Table `II` is placeholder data.	Not a controlled experiment; headline factors unmeasured.	C:063
`OTC-PO` / `OTC-GRPO` tool-productivity reward	Reward r_final = alpha*r_tool*r_correct (multiplicative; additive was unstable because the model gained reward by calling fewer tools without a correct answer). `OTC-PPO` : r_tool = cos(m*pi/(2m+c)). `OTC-GRPO` : f(m,n)=2nm/(m+n), r_tool = sin(f*pi/(2n)), peaking at m=n. GRPO uses the global approximation of optimal tool calls.	Up to -68.3% tool calls and +215.4% `TP` (`OTC-PPO` 7B `NQ` : `TC` 3.282 to 1.040, `TP` 0.136 to 0.429); 7B `NQ` `EM` 0.446 vs 0.449; 3B `OTC-PPO` `NQ` `EM` 0.355 vs 0.403 (drop); Code 7B `AIME24` : `TP` 16.4 to 52.4 (+199.4%), `TC` 2.1 to 0.7, `EM` 26.7 to 33.3	Gains larger at 7B; 3B loses accuracy in some cells; only search and code tools on `QA` and math; GRPO less stable than `PPO` .	C:064

Restart Economics: retry priced by cost per resolved task, iteration cap as a guard, break-even in median attempt costs	Restart Economics: expected cost per incoming task, with agent spend, verification and a priced outside option (annotated engineer time). Policy ladder: no retry, retry with best attempt budget (K at most 4), retry with a per-attempt iteration cap, schedules by coordinate ascent, a receding-horizon restart rule on three state variables, and cascades across configurations. Exact replay of truncated logged trajectories scores each policy. Break-even is stated in multiples of the median attempt cost.	Automated verifier, $100 an hour: retry cuts expected cost 13.2 to 19.6% for six configurations and 4.3% for GPT-5.2; at $300 an hour 16.4 to 21.0% (GPT-5.2 8.0%). Dollar savings per task at $100 an hour: GPT-5 5.67 [3.42, 8.80], GPT-5.2 1.01 [0.45, 2.16]. The iteration cap adds nothing given retry: break-even 2.14 to 3.83 median attempt costs ($1.74 to $7.08 an hour); margin never distinct from zero on the cost side in any of 532 cells; best hindsight cap saves at most $0.03 per task. Attempts still running at 100 calls resolve 33 to 55% of the time. Only 11 to 25% of tasks have mixed outcomes. Cross-configuration outcome correlation 0.62 to 0.90. Cascades save 9.2% in point estimate at $100 an hour, interval includes zero. Review at 0.5 of the outside option: no configuration retries, the cap saves $0.75 to $3.56 per task at $100 an hour. The fitted state rule stops 42 to 78% too early and costs 7 to 32% more than the exact optimum on synthetic attempts.	Clean restarts only. One benchmark, one harness (OpenHands CodeActAgent), at most 4 attempts. A configuration is not a model. `SWE-bench` Verified contamination. The outside option is annotated time, understated for short tasks. The state rule has 8 coefficients; learned monitors are untested. Working paper v0.1, single author, not peer reviewed.	C:043
Budget Tracker and `BATS` budget-aware test-time scaling	Budget Tracker: plug-in that shows remaining and used per-tool budgets each ReAct iteration. `BATS` : structured tree plan (exploration vs verification clues), self-verification with `SUCCESS/CONTINUE/PIVOT` decisions, trajectory summarization, LLM judge picks final answer. Unified cost = token cost (input/output/cache) + tool-call cost.	Budget Tracker at budget 10 matches ReAct at 100 (12.8% vs 12.6%) with 40.4% fewer search calls, 19.9% fewer browse calls, 31.3% lower cost.; `BATS` 24.6% BrowseComp, 46.0% `BrowseComp-ZH` , 27.0% `HLE-Search` , training-free.; Ablation: removing verification drops BrowseComp 18.7% to 15.4%.	Evaluated on web search agents only; tool-call budget, not token budget.	C:131
Trajectory test-outcome vectors by language	No new method; diagnostic study. Each intermediate solution is re-executed to get a test-outcome vector; transitions labelled breakthrough/stay/improved/regressed/churn/broke/stuck; spans labelled with 12 behaviors by GPT-5-Mini; mixed-effects model on output tokens with problem as random effect.	`OCaml` uses 1.28-1.69x Python tokens (p<0.001). Gemma on `OCaml` : 1,196 stuck self-loops, 92% of failed snapshots are compile errors. Gemma/Qwen keep working after the first correct solution; over 70% of their spans into `STAY` are bare writes with no test run.	One scaffold, competitive-programming problems only, 4 languages, 5 models; may not generalize to repo-scale work.	C:134

Test before enabling: Replay logged failure loops with guard on; measure saved spend and tasks wrongly stopped. Not covered: Polling-loop specific technique has no source beyond the generic breaker and A:059 ReTest.

28. Repeated screenshots

Identical or back-to-back screenshots pay image tokens again for no new information.

API detection: Hash image bytes per request (md5 repeat, A114/A112); flag a retake after a failed browser call (A120). CLI detection: Transcript: consecutive screenshot tool calls, identical image hash, retake after a tool error.

Technique	How it works	Effect	Loses when	Source

Observation masking	Replace observations outside a 10-turn window with a placeholder.	About half the cost of the raw agent at matched solve rate (`SWE-bench` Verified).	Text observations only in the paper.	C:130: The Complexity Trap (JetBrains, arXiv 2508.21433)
Claude Code oldest-image eviction	Drops oldest images and `PDFs` past the cap.	One slower turn per batch.	Removed images cannot be seen.	C:055, A:071: How Claude Code uses prompt caching
Image input pricing	1 input image token is billed as 1 token and a cached image token at 0.1, so a repeat under a cache costs less than a fresh one.	pricing fact only	Needs a warm cache.	A:146: Vertex Provisioned Throughput

Test before enabling: Count duplicate hashes on a week of traffic, estimate savings as repeats times image tokens, then A/B the dedup on a sample. Not covered: no corpus source for perceptual-hash dedup. DevX proposal only.

29. Progress narration

Text beside tool calls ('now I will...') adds output tokens that cost the highest rate and carry no result.

API detection: Assistant text over 50 tokens in a response that also has a tool_use block (A54). CLI detection: Transcript: assistant text blocks over 50 tokens adjacent to tool calls.

Technique	How it works	Effect	Loses when	Source
text.verbosity	Sets default detail low, medium or high; task-specific length goes in the prompt.	not measured in corpus	Broad brevity prompts can make answers too short.	A:205: GPT-5.6 guide
Remove tool preambles	Delete prompting for upfront plans and status updates; preambles are steerable in frequency.	not measured in corpus	Hurts user visibility; preambles can cause early stopping when prompted.	A:249, A:208: Codex prompting guide; GPT-5 prompting guide
Effort parameter	output_config.effort affects all output, including text and tool calls; lower gives fewer, terser calls.	not measured in corpus	Behavioural signal, not a budget.	A:282: Effort parameter (Claude)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
NoWait (suppress Wait-like self-reflection tokens at inference)	Logit processor at inference sets logits of reflection keywords to a large negative value. 15 seed keywords were mined from 32 QwQ-32B runs on `AIME` 2025 (wait, alternatively, hmm, but, however, check, maybe, verify, again, now) and expanded to all vocabulary tokens containing a keyword as substring. No training or weight change.	QwQ-32B `AMC23` 95.50 (+4.25) at -30% tokens; `AIME25` 68.00 (+1.33) at -31%; Phi4-R+ `AMC23` +6.00 at -28%; Qwen3-32B -13% to -16% tokens; `Kimi-VL` -40% to -60% tokens on some sets (-7.25 accuracy on `EMMA-mini`); QwQ-32B-Preview `AIME24` 42.00 at 5764 tokens vs 8979 baseline; O1-Pruner 33.33	Distilled Qwen3-4B/8B/14B degrade (>5 points `AIME24` , >12 `AIME25`); suits `RL-trained` reasoning models; math-heavy benchmarks.	A:091

`THINK-Bench` efficiency metrics (useful tokens, reflection quality)	`THINK-Bench` : 1375 questions; metrics for thinking efficiency, useful tokens and reflection quality over reasoning-model chains.	1375 questions.; Efficiency Grok-3-mini 61.69% vs QwQ-plus 44.58%.	Benchmark only; no mitigation.	A:093
Idle-session thinking clearing (clear_thinking_20251015, keep:1) and effort-default / length-limit regression lessons	Clearing old thinking after 1h idle was meant to shrink the uncached resume; a bug repeated it every turn, causing cache misses; a 25-word between-tool limit cost quality.	Prompt ablation showed a 3% eval drop; medium default effort felt less intelligent and was reverted to high/xhigh.	Pruning history or verbosity limits can silently hurt quality and cause cache misses; need per-change evals.	A:054
Length Controlled Policy Optimization (`LCPO`) for reasoning-length control	`LCPO` : prompt adds 'Think for n_gold tokens' with n_gold ~ Uniform(100,4000); GRPO reward. L1-Exact: correctness minus 0.0003*|n_gold - n_y|. L1-Max: correctness times clipped length slack with delta 0.5, trained 120 steps after 700 of Exact.	Beats S1 by 20-25% absolute at 512/1024 tokens. L1-Max matches DeepScaleR-4K with up to 2x fewer tokens. Mean length error about 3% on math (20-40% `OOD`). Short CoT beats GPT-4o by about 2% at equal length.	May not generalize to longer requested lengths; 4K training context; 1.5B math model (7B in appendix); little tuning.	A:095

Test before enabling: A/B narration-off vs on; metric output tokens per task and task success. Not covered: no corpus source for a numeric saving from removing narration. Claude Opus 5 does not shorten with lower effort (A:200); prompt explicitly.

30. Whole-file reads

A Read with no offset or limit pulls the full file (over 5,000 tokens) into history when a slice would answer the question.

API detection: Read tool_use with no offset and limit in input, and the following tool_result above 5,000 tokens (hermes A71); charge tokens minus 2,000. CLI detection: Gateway signal only; PreToolUse hook sees tool_input without offset or limit.

Technique	How it works	Effect	Loses when	Source
Code intelligence plugin (go to definition) instead of reading files	A language server returns the definition or references directly so the agent does not read through candidate files to locate a symbol.	not measured in corpus.	Needs a language server per language; typed languages named in the doc.	A:215, C:083: Manage costs effectively (Claude Code); Set up Claude Code in a monorepo or large codebase, Anthropic
Path-reference stub for oversized files	Claude Code re-reads up to five recent files after compaction and returns any file over 5,000 tokens as a path reference, shown as Referenced file, without content.	not measured in corpus.	The agent must read again if it needs the content.	C:079: Explore the context window, Anthropic
Tool-response range selection and truncation guidance	Offer pagination and range parameters with defaults on any tool that can return a large file, and word the truncation message so the agent makes small targeted reads.	72 vs 206 tokens for concise vs detailed response in the guide's example.	Can hide needed content.	A:284: Writing effective tools for `AI` agents, Anthropic
Large-file re-read after compaction (failure case)	Codex issue 16812: a 93 `KB` analysis, 58 `KB` plan and 610 `KB` OpenAPI spec had to be re-read after every compaction. Mitigation given: lower tool_output_token_limit and put durable facts in `AGENTS.md` .	not measured in corpus.	Smaller limit keeps less context.	C:106: Mastering Context Compaction in Codex CLI (Vaughan blog)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Text-as-file-system navigation with coding agents (ripgrep, scripts)	Place the corpus as a directory of txt files (one JSONL for `NQ`) and give an off-the-shelf coding agent (Codex GPT-5; Claude Code Sonnet 4.5) only the path and query. It uses grep/rg, sed, nl and Python scripts iteratively. Configs: no retriever, `BM25`, Gemini embeddings.	Beats published `SOTA` by 17.3% on average; `SOTA` on 4 of 5 benchmarks; Native search commands per trajectory 14.92 (no retriever) vs 8.33 (Gemini retriever); GPT-5 full-context cost $1.421 per query on Oolong-Syn vs $0.275 on BrowseComp-Plus	Retrievers did not consistently help; optimised for coding not long context; costlier than RAG; 200-sample evals; about equal to full context on LongBench.	C:046

Test before enabling: Replay, then paired A/B on the guard: billed cost per success, extra Read calls with offset, first-edit turn (C:024 reports compression delayed first edit 0.94 turns). Count outlier reads over 5,000 tokens before and after. Not covered: No corpus source gives a read-size threshold or a measured saving for forcing slices. Hook-based read blocking is a DevX proposal.

31. Duplicate reading across agents

A helper re-reads files the parent already read because it does not share context.

API detection: Same file contents appearing in multiple derived conversations under one parent. CLI detection: Repeated Read/Grep on identical paths across sidechains in transcripts.

Technique	How it works	Effect	Loses when	Source
DevSkills (reuse context, justify re-reads)	Skill instructs the agent to state a hypothesis, reuse context and justify re-reads.	SubRetrv cost behaviour reduced 51.52% and 43.57% in two settings.	Benchmark settings only.	A:059: A:059 cost-inefficient agent behaviors (SubRetrv, SimScrpt, ReTest)
Fork versus fresh subagent choice	Fork inherits parent context so no re-read; fresh helper starts cold.	Not measured.	Fork inherits full parent prefix cost.	C:072: Claude Code subagents documentation
Structure-aware retrieval (CodeGraph)	Graph-based retrieval to avoid re-reading.	Raised cost by up to 28.14% in the source.	Cost increase, so not a fix.	A:059: A:059

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Repository-level RAG taxonomy (graph-based, non-graph, selection by Shapley, context pruning)	Survey, no algorithm. Taxonomy of `RACG` by retrieval substrate, control regime and evaluation; agent autonomy levels L0 (static pipeline), L1 (iterative feedback), L2 (autonomous agents).	Agentless (L0) cited at 32% on `SWE-bench` Lite at low cost. Long-context models can match RAG on small repos (Peng et al. 2025).	Survey only; no new measurements; cost and latency claims qualitative.	C:044

Test before enabling: Count repeated reads per task before and after DevSkills; check pass rate unchanged. Not covered: No gateway-side source.

---

32. Agent coordination chatter

Teammates exchange messages and idle-wake, and each message re-enters context.

API detection: No gateway source. DevX proposal: count derived conversations that alternate with near-identical prefixes. CLI detection: Team mailbox message count, TeammateIdle hook events, per-teammate tokens.

Technique	How it works	Effect	Loses when	Source
Mailbox messaging (one message per recipient)	Teams exchange mailbox messages, one per recipient; idle teammates wake via hook.	Not measured in corpus.	Disabling teams loses parallel collaboration.	C:081: Claude Code agent teams documentation
Drop roster by disabling SendMessage	The sibling roster only appears with SendMessage; disallowedTools removes it.	Not measured.	No peer messaging.	C:072: Claude Code subagents documentation
Delegation floor cost and brief granularity	Each delegation has a fixed cost; brief granularity has an optimum.	84-98% at worker rate for orchestrator case.	Too fine briefs net negative.	C:012: C:012 orchestrator analysis

Test before enabling: A/B teams on versus off on matched multi-part tasks; compare cost per resolved task. Not covered: No `API-side` source.

---

33. Large images and `PDFs`

One large image or PDF can cost thousands of input tokens in a single call, then keeps being re-sent.

API detection: Image over 1,200 tokens (A111), cost (itok-800) times input price; PDF read (A115); same document as page images and text (A116/A117). CLI detection: Transcript: Read of a PDF or large image, tool result token size, same file read twice in different forms.

Technique	How it works	Effect	Loses when	Source
Context editing: clear_tool_uses_20250919	Clears large tool results after a trigger, with exclude_tools and clear_at_least to protect the cache.	not measured in corpus	Clearing breaks the cache prefix; extracts do not say it clears images.	C:065, A:268: Context editing on the Claude Developer Platform
Claude Code image/PDF eviction	Removes a batch of oldest images and `PDFs` when caps would be exceeded.	One slower turn per batch.	Model loses sight of removed documents.	C:055, A:071: How Claude Code uses prompt caching
Observation masking	Rolling window of recent observations, placeholder for the rest.	About half cost at matched solve rate (text tasks).	Extrapolation to `PDFs`.	C:130: The Complexity Trap (JetBrains, arXiv 2508.21433)

Test before enabling: Compare answer quality on a document `QA` set for image vs extracted-text input; gate on equal accuracy. Not covered: no corpus source for image downscale, pdftotext or ocrmypdf choice. DevX proposal; hermes names pdftotext and ocrmypdf.

---

34. Broad or unbounded searches

A grep or find with no path, glob or type limit returns hundreds of matches that stay in history and push the agent to read more.

API detection: Grep/Glob tool_use with no path, glob or type input, or Bash grep/rg/find; result above 100 lines or 1,000 tokens (hermes A73, A79). CLI detection: Gateway signal only; PostToolUse hook gets tool_input and the result.

Technique	How it works	Effect	Loses when	Source
Structural retrieval with ast-grep hybrid (`RTK-ML`)	Query-aware ranking and ast-grep structural matches replace raw ripgrep output, grouping and ranking hits under a token budget.	ripgrep 310.9 matches, 2,905 tokens, 0.66 precision vs hybrid 170.9, 2,090 tokens, 0.995 precision (8 tasks, 38 queries); retrieval-quality proxy only.	Whole `RTK-ML` arm raised billed cost 6.8%; no task success recorded for this test.	C:024: Token Reduction Is Not Cost Reduction
Code intelligence (LSP) navigation	Go-to-definition replaces a grep followed by reading several candidate files.	not measured in corpus.	Typed languages; binary per machine.	A:215, C:083: Manage costs effectively (Claude Code); Set up Claude Code in a monorepo, Anthropic
DevSkills: state a hypothesis before retrieval, justify re-reads	Principles written by developers tell the agent to state a concrete hypothesis before retrieving, reuse available context and justify necessary re-reads; addresses subsumed retrieval.	Subsumed retrieval in 64.33-92.33% of tasks and 5.01-11.41% of cost; SubRetrv fell 51.52% and 43.57% in one config.	One setting lost 1.5 pp Pass@1; a structure-aware retrieval tool raised cost up to 28.14%.	A:059: Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents
Aider repo map with --map-tokens	Graph-ranked tree-sitter symbol map under a token budget (default 1k) lets the model find files without searching; 0 disables it.	not measured in corpus.	Budget is soft; map grows when no files are added.	C:040, C:127: How to Manage the Aider Context Window and Token Budgets; How to Reduce Aider Token Usage

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Repository-level RAG taxonomy (graph-based, non-graph, selection by Shapley, context pruning)	Survey, no algorithm. Taxonomy of `RACG` by retrieval substrate, control regime and evaluation; agent autonomy levels L0 (static pipeline), L1 (iterative feedback), L2 (autonomous agents).	Agentless (L0) cited at 32% on `SWE-bench` Lite at low cost. Long-context models can match RAG on small repos (Peng et al. 2025).	Survey only; no new measurements; cost and latency claims qualitative.	C:044
Architect/editor model split and per-task token budgets	R1 as architect with Sonnet as editor; notes code generation is 5-15% of tokens and Aider's tree-sitter repo map saves 20,000+ tokens per turn.	Aider polyglot: DeepSeek R1 + Sonnet $13.29 for 225 tasks, 14x cheaper than o1; Gemini 2.5 Pro $0.028/task at 72% pass.	Blog aggregation; real sessions 3-5x benchmark cost.	C:129

Text-as-file-system navigation with coding agents (ripgrep, scripts)	Place the corpus as a directory of txt files (one JSONL for `NQ`) and give an off-the-shelf coding agent (Codex GPT-5; Claude Code Sonnet 4.5) only the path and query. It uses grep/rg, sed, nl and Python scripts iteratively. Configs: no retriever, `BM25`, Gemini embeddings.	Beats published `SOTA` by 17.3% on average; `SOTA` on 4 of 5 benchmarks; Native search commands per trajectory 14.92 (no retriever) vs 8.33 (Gemini retriever); GPT-5 full-context cost $1.421 per query on Oolong-Syn vs $0.275 on BrowseComp-Plus	Retrievers did not consistently help; optimised for coding not long context; costlier than RAG; 200-sample evals; about equal to full context on LongBench.	C:046

Test before enabling: Paired run with the search cap or LSP on: billed cost per success, repeated-search count and match precision against test-grounded gold (C:024 method). Check that no compressed search output feeds a shell pipeline. Not covered: No corpus source on the right result-line cap; the 100-line threshold is hermes's, unvalidated.

35. Bloated JSON and query results

Large JSON or `DB` results carry many unused fields, `IDs` and serialisation wrappers, and the whole payload is re-billed every turn.

API detection: tool_result text that parses as JSON above a size line; unused-field share needs a later-use check, so alert only (hermes A98-A100, thresholds not given). CLI detection: Gateway signal only; MCP tool names visible via `OTel` with OTEL_LOG_TOOL_DETAILS=1 (C:002).

Technique	How it works	Effect	Loses when	Source
response_format enum (concise vs detailed) and high-signal fields	Tools return names and content, not uuid, mime_type or 256px_image_url, and expose a ResponseFormat enum so the agent picks concise or detailed. The guide likens extra formats to GraphQL field choice.	Concise 72 tokens vs detailed 206 tokens, about one third, one Slack example.	Agent may need `IDs` for follow-up calls; best format varies by task and model, test it.	A:284: Writing effective tools for `AI` agents, Anthropic
Tool-result clearing for re-fetchable payloads	clear_tool_uses removes old API and file results the agent can fetch again.	Cookbook peak 335,279 to 173,137 tokens, single synthetic run. Figures are from A:271.	Cache rewrite per clear; not for results that cannot be re-fetched (use exclude_tools).	A:271, C:065: Context engineering cookbook; Context editing on the Claude Developer Platform, Anthropic
`LLMLingua` coarse-to-fine prompt compression (untested on JSON)	A budget controller allocates compression per segment, then token-level iterative compression drops low-information tokens using a small LM's perplexity.	`GSM8K` 2,366 to 117 tokens at 77.33 `EM` vs 78.85 (about 20x).	Tested on prose prompts; byte-exact JSON and agent traces are not in the corpus; C:024 saw compression break exactness.	A:079, A:081, C:024: `LLMLingua` (Jiang et al., Microsoft Research); `LLMLingua-2` (Pan et al., Microsoft Research); Token Reduction Is Not Cost Reduction

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
LiteLLM budget caps, Langfuse, `LLMLingua`, RouteLLM, `GPTCache` semantic cache	Roundup: LiteLLM max_budget per key; `LLMLingua` prompt compression; RouteLLM routes between two models by threshold; `GPTCache` semantic caching.	RouteLLM 85% cost reduction keeping 95% GPT-4 quality on `MT` Bench (45% MMLU, 35% `GSM8K`); LongLLMLingua +21.4% RAG with 1/4 tokens.	Vendor blog citing others' numbers; RouteLLM is binary routing only.	C:061

Retained reasoning plus native compaction, programmatic tool calling, low reasoning effort, small-tier model (Luna) selection	Persists reasoning across turns with compaction, runs tool orchestration in code outside context, and uses cheaper tiers or lower effort.	`ARC-AGI-3` 13.3% to 38.3% with about 6x fewer output tokens; BrowseComp 84.04% at $1.33 versus 84.36% at $33.27 for GPT-5.5; PlayerZero cost -64%.	Vendor and customer testimonials; harness-specific.	A:207

Test before enabling: For each MCP tool with large JSON: A/B concise vs detailed format on replayed tasks; metric billed cost per success and tool-selection and retrieval accuracy (the guide asks you to test response structure per task). Not covered: No corpus source for JSON minification, `TOON-style` formats or `SQL` column pruning. Field projection at a gateway is a DevX proposal.

36. Wordy and repeated output

Whole-file rewrites, echoed questions, repeated paragraphs and reprinted code inflate output tokens.

API detection: Write of a file already read; question echo (A52 Jaccard 0.7); repeated paragraph 300+ chars (A53); reprinted code 300+ chars (A58); stop_reason max_tokens (A60). CLI detection: Transcript: whole-file Write after Read of the same file, repeated text blocks, reprinted code blocks.

Technique	How it works	Effect	Loses when	Source
Constrained-CoT (`CCoT`)	Prompt asks for concise output; conciseness metrics measure redundancy and information flow.	not extracted	Over-brevity can hurt accuracy.	A:103: Constrained-CoT
text.verbosity	Default level of detail low, medium or high, set per request.	not measured in corpus	Can be too brief.	A:205: GPT-5.6 guide
response_format concise	Enum letting the tool return concise vs detailed output.	72 vs 206 tokens in the Slack example	Extra call if detail needed.	A:284: Writing effective tools for `AI` agents
max_tokens ceiling	A hard cap on output length.	maxOutputTokens=300 did not cut thinking on simple tasks and broke 7 of 25 hard ones.	Truncation; a ceiling, not a reducer.	A:052: Cost per task (Google blog)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Chain of Unconscious Thought prompt plus token-efficient strategies	Training-free prompt with two parts. `RPI` (Reasoning Process Internalization): solve in hidden thinking, output a bare-minimum single-line reasoning. `TES` (Token-Efficient Strategies): ' `TOKEN CONSERVATION MODE ACTIVE` ', symbols and abbreviations, omit articles, strip greetings, and a point rule: each saved token +1, each accuracy error -100.	Ablation averages: 354.46 tokens vs CoT 676.85 (-47.6%), accuracy 91.19 vs 88.40 (-2.79 points); `RPI` alone -31.1% tokens; `TES` alone -33.9%; Figure 1 claim: -20.51% tokens vs best baseline, 0.1% accuracy drop; Arithmetic sets: 190.19 tokens, 94.28% vs CoT 93.5%	QwQ-32B stays high in tokens whatever the prompt; math benchmarks only; accuracy loss vs CoT in the ablation.	A:090

`TRIM` distilled output plus small-model reconstruction	Prompt the LLM to omit a set W of 25 readily inferable function words; a small encoder-decoder or decoder model reconstructs the full text. W is chosen by RoBERTa masked-prediction ranking (cut-off level 5, about 18% tokens saved). Break-even: C_D^o*G >= C_D^i*I + C_R^i*I_R + C_R^o*O_R.	GPT-4o with T5: 19.41% tokens saved, Theta F1 85.51, cosine 99.99%; Tokens saved: GPT-4.1-mini 22.79%, Sonnet 4 17.83%, Opus 4.1 17.38%; Added instruction costs 169 tokens; break-even answer length about 216 tokens (GPT-4o), about 190 (Sonnet 4)	English only; rewrite/summarize/translate tasks about 5.77% lower; not tested on code, conversation or reasoning models; effect of omission on generation quality untested; needs a trained reconstructor.	A:105
Idle-session thinking clearing (clear_thinking_20251015, keep:1) and effort-default / length-limit regression lessons	Clearing old thinking after 1h idle was meant to shrink the uncached resume; a bug repeated it every turn, causing cache misses; a 25-word between-tool limit cost quality.	Prompt ablation showed a 3% eval drop; medium default effort felt less intelligent and was reverted to high/xhigh.	Pruning history or verbosity limits can silently hurt quality and cause cache misses; need per-change evals.	A:054
Length Controlled Policy Optimization (`LCPO`) for reasoning-length control	`LCPO` : prompt adds 'Think for n_gold tokens' with n_gold ~ Uniform(100,4000); GRPO reward. L1-Exact: correctness minus 0.0003*|n_gold - n_y|. L1-Max: correctness times clipped length slack with delta 0.5, trained 120 steps after 700 of Exact.	Beats S1 by 20-25% absolute at 512/1024 tokens. L1-Max matches DeepScaleR-4K with up to 2x fewer tokens. Mean length error about 3% on math (20-40% `OOD`). Short CoT beats GPT-4o by about 2% at equal length.	May not generalize to longer requested lengths; 4K training context; 1.5B math model (7B in appendix); little tuning.	A:095

Test before enabling: Compare output tokens and edit success with whole-file vs diff edits on the same tasks. Not covered: no corpus source opened for diff-vs-whole-file edit format savings or shingle echo detection. DevX proposal: edit-format switch tested by paired CPS.

37. Web pages and search results

Fetched pages and search results are long, overlap each other, and are refetched, so the same text is billed repeatedly.

API detection: Page over 4,000 tokens (A101); more than 10 distinct `URLs` (A103); 8-word-shingle overlap 0.5+ (A104); same URL refetched (A105); overlapping chunks (A107); refetch after compaction (A110). CLI detection: Transcript: WebFetch/WebSearch tool results by size and URL repeat; hook counts distinct `URLs` per session.

Technique	How it works	Effect	Loses when	Source
Provence	Context pruning as sequence labelling, unified with reranking, so irrelevant sentences are dropped.	Negligible to no performance drop; 49% compression noted at one point.	Needs a trained pruner.	A:087: Provence (arXiv 2501.16214)
LongLLMLingua	Query-aware prompt compression for long contexts.	+21.4% on NaturalQuestions with about 4x fewer tokens (GPT-3.5-Turbo); 94.0% cost cut on LooGLE.	Compression can drop needed facts.	A:080: LongLLMLingua (arXiv 2310.06839)
Self-Route	Model self-reflection routes each query to RAG or long context.	Cost down 65% on Gemini-1.5-Pro and 39% on GPT-4o at comparable quality.	Pure RAG is worse than long context.	A:117: Retrieval Augmented Generation or Long-Context LLMs?
Tool-response design	Pagination, filtering, truncation and a response_format enum; concise 72 tokens vs detailed 206 in the Slack example.	72 vs 206 tokens in the example	Model may need a second call.	A:284: Writing effective tools for `AI` agents

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
`OP-RAG` : order-preserve retrieval-augmented generation	Split the document into 128-token chunks, score each by cosine similarity (`BGE-large-en-v1` .5), take the top-k, then order the selected chunks by their original document position rather than by score (`OP-RAG`). Answer quality follows an inverted U against the amount of retrieved context.	En. `QA` Llama3.1-70B: `OP-RAG-16K` 44.43 F1; `OP-RAG-48K` 47.25 F1 vs 34.26 F1 for full 117K context; Vanilla RAG at 128 chunks 38.40 vs `OP-RAG` 44.43; En. `MC` : `OP-RAG-24K` 88.65 vs 71.62 without RAG; vanilla at 192 chunks 81.22; 8B peaks near 16K tokens; 70B peaks at 48K (En. `QA`) and 32K (En. `MC`)	Long-document `QA` only; sweet-spot context length varies by model and task; advantage over vanilla RAG is small at around 8 chunks; no limitations section.	A:118
LiteLLM budget caps, Langfuse, `LLMLingua` , RouteLLM, `GPTCache` semantic cache	Roundup: LiteLLM max_budget per key; `LLMLingua` prompt compression; RouteLLM routes between two models by threshold; `GPTCache` semantic caching.	RouteLLM 85% cost reduction keeping 95% GPT-4 quality on `MT` Bench (45% MMLU, 35% `GSM8K`); LongLLMLingua +21.4% RAG with 1/4 tokens.	Vendor blog citing others' numbers; RouteLLM is binary routing only.	C:061
AttentionRAG attention-focus pruning (next-token reformulation of the query)	GPT-4o-mini builds an answer-hint prefix in next-token form. Context split into m-token chunks; per chunk one focal token is generated ('none' drops the chunk). Attention to the focal token is summed over layers; top-k tokens select the sentences kept. Training-free, batched.	Llama 2WikiMQA: 6.3x (small) and 15x (large) compression, `EM` 0.42 vs 0.49 original; HotpotQA latency 3.99s + 4.94s vs 19.36s original; Compression only 1.7x-2.0x on TriviaQA	Needs attention access; attention aggregated over all layers; no explicit compression-ratio control; low ratio on TriviaQA.	A:088
RAG vs long-context routing guidance (LaRA benchmark)	LaRA benchmark comparing RAG (600-token chunks, 5 chunks, 100 overlap, `GTE-large` + `BM25`) vs long context.	32k: long-context +2.4% over RAG; 128k: RAG +3.68%. 128k RAG gains 6.48% (Llama-3.2-3B), 38.12% (Mistral-Nemo-12B). Comparison favours long-context (+15.22% at 32k). GPT-4o hallucination accuracy 56.34%.	GPT-4o-judged, mostly GPT-4o-generated `QA` , fixed RAG config, no cost measure.	A:119
Selective Context (self-information pruning)	A base causal LM (`LLaMA-7B` ; GPT-3 curie for OpenAI models) computes self-information -log2 P(x_t|x_<t) sentence by sentence. Tokens are merged into lexical units (tokens, noun phrases via spaCy, sentences). Units at or above the p-th percentile of self-information are kept.	At ratio 0.5: `BERTScore-F1` drop .023, unfaithfulness .038. At ratio 0.2: `BERTScore-F1` .931 vs .899 random. Phrase-level units best. One vicuna-13b case: 36% memory (77,695 to 61,885 `MB`), 1.32x speed.	Needs phrase-boundary detection (no verb phrases). Best percentile varies by task and context. Quality drops noticeably from ratio 0.5 up. Never tested on code or tool output.	A:083
Long-context vs RAG comparison (chunk vs summarization-based retrieval)	Compare long-context (`LC`) vs RAG with the same LLM (GPT-4o). Filter questions answerable without context; compare 5 retrievers (`BM25` , Contriever, text-embedding-3-small, Llama-Index Tree/Window, `RAPTOR`); pick the best; analyse by source and question type.	`LC` 56.3% correct vs RAG (`RAPTOR`) 49.0%. `LC-only` correct 2,287 vs `RAG-only` 1,294 (about 10% only RAG). `RAPTOR` best retriever at 38.5% on the retriever-comparison sample.	Text only; Wikipedia long contexts are synthetic noise-padded (biases against RAG); no cost or token measurements.	A:116

Test before enabling: Paired replay of research tasks with pruned vs full pages; metric answer accuracy and cost per successful task. Not covered: no corpus source for 8-word-shingle dedup or `ETag` refetch skipping. Semantic caching exists in corpus (A:240 Portkey, A:113) but correctness is unproven; DevX proposal.

38. Copied history

Background text, chat transcripts and forked conversations are pasted or duplicated, so the same tokens enter context twice.

API detection: Gateway: 400+ char paragraphs whose whitespace-normalised hash was seen in an earlier prompt of the project (A7); prompts of 2,000+ tokens with 4+ Human:/Assistant: markers (A8); shared message prefix with an earlier conversation (A9). CLI detection: Transcript entry uuids shared with earlier sessions (A9 needs CLI uuids); otherwise gateway signal.

Technique	How it works	Effect	Loses when	Source
Fork inherits parent prefix	A fork keeps the parent's system prompt, tools and history exactly, so its first request reads the parent's cache; /fork copies add isolation text at the end only.	not measured in corpus	Forks still replay the parent history; subagents do not inherit it.	C:055: How Claude Code uses prompt caching (Anthropic)
`CLAUDE.md` under 200 lines + path-scoped rules	Instructions loaded once per session; rules with paths: load on demand; survives /compact by re-read from disk.	not measured in corpus	Loaded every session; long files reduce adherence.	C:077: How Claude remembers your project (Anthropic)
Handoff file	Replace pasted transcripts with a short curated file the next thread reads.	not measured in corpus	Hand-maintained.	C:035: Context Management Across Codex Surfaces (Developer Toolkit)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
One chat per coherent outcome with fork, compact, subagents and `AGENTS.md`	Plan mode before hard tasks, short `AGENTS.md`, /fork only when work branches, subagents for bounded work, one chat per outcome.	not measured	Qualitative guidance only.	C:098

Test before enabling: Count duplicated tokens per project before and after moving background to `CLAUDE.md`; check task success unchanged. Not covered: No corpus source for dedup of pasted text; the thresholds are hermes's own, unsupported.

39. Compaction cost

Compaction writes a summary (an LLM call) and the next turn rewrites the cache from scratch.

API detection: Gateway: sudden context drop plus cache_creation spike on the following request; summary call visible as a separate request. Cost = summary output tokens plus the rewrite premium. CLI detection: PreCompact/PostCompact hooks with trigger manual or auto (C:074); compact_boundary entries in transcripts.

Technique	How it works	Effect	Loses when	Source
Server-side compaction (compact_20260112)	Beta context edit that fires at `trigger` (default 150K, minimum 50K), summarises the conversation into a typed compaction block, accepts custom `instructions` and `pause_after_compaction`.	Cookbook: peak 335,279 to 169,164 tokens with a 2,783-token summary; high-level facts kept 3/3, obscure specifics 0/3. Single run. Figures are from A:271.	Loses verbatim detail; next turn rebuilds the cache; on short sessions the summary call costs more than it saves.	A:271, C:065: Context engineering: memory, compaction, and tool clearing (Claude cookbook)
Cache-warm delay rule	Delay compaction while the cache is warm; compact only when full-window quality loss exceeds the cache-invalidation cost. One compaction on a 125,000-token context costs $0.40, about 21 cached turns.	$0.40 per 125K compaction (secondary source).	Secondary blog; sources disagree on whether compaction destroys or reuses the cache.	C:112, C:055: Context Compaction Showdown (Vaughan); How Claude Code uses prompt caching (Anthropic)
Codex compaction tuning	model_auto_compact_token_limit (effective limit min(user, 0.9 x window)) and a custom compact prompt; every compaction is an LLM call.	not measured in corpus	Multiple compactions reduce accuracy; no threshold study.	C:106: Mastering Context Compaction in Codex CLI (Vaughan)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Copilot CLI background auto-compaction at ~80% (dynamic to 90%) plus /context, /usage, /compact	Compresses history in background near 80% of token limit, waiting until 90% when static context is 75% or more; /compact manual, /usage and /context for visibility.	Threshold behavior only; no savings measured.	Compaction reclaims less when static context is large.	A:154
OpenCode prune-then-summarise; Claude Code three-layer trim, cache-friendly tail trimming, 9-section summary	OpenCode hides old tool outputs only if it frees over 20,000 tokens and keeps the last 40,000; Claude Code trims the tail to preserve the cache prefix and auto-compacts at window minus 13,000.	Worked example on a 15,400-token session; no cost figures.	Codex handoff summary is all-or-nothing; source based on reverse engineering.	C:007
Model natively trained for compaction across multiple context windows	The model auto-compacts near the window limit and repeats; adds xhigh reasoning effort.	30% fewer thinking tokens at medium effort on `SWE-bench` Verified versus GPT-5.1-Codex.	Vendor claim; Codex-specific.	A:250
Codex compaction backends (default summarization, remote, remote_v2) and two-phase memory consolidation	Auto-compaction at model_auto_compact_token_limit ; memory phase 1 extracts raw memories, phase 2 consolidates into `MEMORY` .md and summaries.	Not measured.	Reverse-engineered reading of one Codex version.	C:047

Test before enabling: Compaction break-even sweep at 50K, 100K, 200K contexts with accuracy after 1, 2 and 3 compactions (the design R3 also lists); billed cost with caching on. Not covered: Compaction threshold comparison (50% vs 90% vs window minus 33,000) has no head-to-head in the corpus.

40. Git and GitHub output

Full diffs, lockfile diffs, log metadata, repeated status and whole `PR` or issue histories are large and mostly unread, and stay in history.

API detection: Bash tool_use matching git diff/show/log/status or gh pr/issue view; result above 4,000 tokens; lockfile sections above 500; identical git status hash twice (hermes A91-A96). CLI detection: Gateway signal only; hook sees the command; `OTel` tool_result has tool_name.

Technique	How it works	Effect	Loses when	Source
PreToolUse command rewrite (updatedInput)	Hook inspects the Bash command and returns a bounded replacement; the doc's example does this for test runners. Applying it to git commands is a DevX proposal.	not measured in corpus.	Only one hook may rewrite the same input; rewritten output can hide lines the agent needs.	A:215, C:090: Manage costs effectively (Claude Code); Automate actions with hooks (guide), Anthropic
Deterministic CLI output compression (vendor claim, unverified)	A commercial command-output compressor claims 75-80% savings for git diff, git log, ls and grep; the paper found only the vendor's reference literals, no paired billed run.	not measured in corpus (vendor claim only).	C:024 found hook compression gave -2.7% to +48.4% billed cost, so token cuts do not imply cost cuts.	C:024: Token Reduction Is Not Cost Reduction
Dynamic context injection and subagent delegation	Skills can inject command output such as git diff `HEAD` or gh pr diff before the model sees the skill, so size is bounded only by the command. A forked Explore or Plan agent starts without git status and `CLAUDE.md` to keep context small.	not measured in corpus.	Injected diff is unbounded unless the command is bounded.	C:075: Extend Claude with skills, Anthropic

Test before enabling: Replay traces with the rewrite on; metric billed cost per success and rate of follow-up git calls (the agent re-asking for the full diff). Hold out `PR` review tasks where the full diff is needed. Not covered: Corpus has no measured technique for git or GitHub output. Named git flags are DevX proposals, not from a source.

41. Dependency, vendor and generated files

Reads and search hits in node_modules, build output, lockfiles and generated code add tokens with no task value and widen later searches.

API detection: Read path or search result lines (5 or more) matching node_modules, .venv, dist, build, target, vendor, __pycache__, or generated patterns (.lock, .min.js, .map, _pb2.py) in tool_use input and result (hermes A76, A77). CLI detection: Gateway signal only; PreToolUse hook sees file_path.

Technique	How it works	Effect	Loses when	Source
permissions.deny Read rules	Add rules such as Read(./**/dist/**/*) and Read(./**/vendor/**/*) to .claude/settings.json; the built-in Grep and Glob omit denied paths on a best-effort basis, and file commands in Bash are covered.	not measured in corpus.	A Bash grep -r over a directory with denied files still includes them; subprocesses that open files are not covered.	C:083: Set up Claude Code in a monorepo or large codebase, Anthropic
Respect .gitignore in content search	Claude Code content searches skip paths in .gitignore (node_modules, dist, build) with no configuration.	not measured in corpus.	Only for ignored paths; checked-in vendor code needs deny rules.	C:083: Set up Claude Code in a monorepo or large codebase, Anthropic

.claudeignore exclusion list (blog claim)	Blog lists node_modules, build, dist, .venv, vendor, *.lock, *.min.js, generated and __pycache__ for exclusion.	Blog claims about 60% less context retrieval; no method given, unverified.	Official doc in corpus does not mention .claudeignore; use permissions.deny.	C:056: Claude Code Cost Optimization: A Benchmark-Backed Playbook for Cutting Token Spend (blog)
Cline strict path filtering (failure description)	Cline docs describe directory listings and searches without strict path filtering pulling node_modules, target and dist trees into history.	not measured in corpus.	Description only, no fix measured.	C:126: How to Monitor and Optimize Token Usage in Cline

Test before enabling: Before enabling deny rules, count how often the dependency reads were followed by an edit to that path (should be near zero). Then paired run, billed cost per success and a check that no task needed the denied path. Not covered: No measured saving from an official source. No Codex or Kiro ignore setting in the corpus. `API-agent` gateway fix is a DevX proposal.

42. Verbose agent reports and briefs

A long helper report re-enters the parent context and is re-billed every later turn.

API detection: Response size of helper-shaped conversation versus the next parent request growth. CLI detection: SubagentStop output tokens; parent context growth after return.

Technique	How it works	Effect	Loses when	Source
Report length cap in prompt	Instruct the helper to return at most 30 lines or a 1,000-2,000 token summary.	Not measured in corpus.	Summary may drop detail.	C:035, C:080, A:084: Claude Code docs and best practices; context engineering guide
tool_output_token_limit	Caps tool result size; 25k default.	Not measured.	Truncated results can hide needed data.	C:110, A:284: Codex config docs; tool design guide
Observation masking (M=10)	Replace old observations beyond the last M=10 with placeholders.	See source; summaries mask failure signals.	Hides failure signals.	C:130: C:130 trajectory elongation study

Test before enabling: Compare parent cost and pass rate with cap versus without on matched tasks. Not covered: No measured effect of report caps.

43. Repeated reads of unchanged files

The same file is read again with no edit between, so a second copy of its tokens is added to history and re-billed each turn.

API detection: Same Read (path, offset, limit) in tool_use input twice with no Edit/Write of that path between; duplicate tool_result tokens (hermes A72). Cross-agent re-reads need helper identity (row 24 style, C). CLI detection: Hook transcript gives agent id for cross-agent repeats; `OTel` tool_result and PostToolUse give path. Gateway signal for same-agent repeats.

Technique	How it works	Effect	Loses when	Source
DevSkills anti-repetition principles (SubRetrv, ReTest)	Seven principles: state a hypothesis before retrieval, reuse context, justify re-reads, rerun tests only when code or evidence changes, break unproductive loops.	Cost cut 7.88-41.73% in 6 of 8 settings; subsumed retrieval is 5.01-11.41% of cost, test re-execution up to 5.39%.	One setting lost 1.5 pp Pass@1; replicate locally.	A:059: Analyzing and Mitigating Cost-Inefficient Behaviors in Coding Agents
Observation masking	Older tool observations, including earlier copies of a file, are replaced by a placeholder outside a rolling window of M turns.	M=10: -56.1% cost (Gemini 2.5 Flash); loses at 7 events or fewer (-45% to -149%).	Edits history so it breaks the cache unless applied at a trigger.	C:130, C:133: The Complexity Trap (JetBrains Research); Evaluating Memory Condensation Strategies for Coding Agents
TokenEconomizer micro-skill injection	An external controller learns repeated per-repository actions and injects operational micro-skills so facts are not rediscovered; key for a repeated action is the normalised command's first 6 tokens or tool plus file base name.	Average cost -18.4 to -30.9% overall; micro-skills alone -11.8% and -14.3% in Claude Code settings.	Learned over a task stream, costs more on early tasks, comparable success only.	C:028: TokenEconomizer: Experience-Driven Black-Box Cost Control for Coding Agents (Li et al., University of California, Berkeley and Microsoft Research)
Persistent project memory (`CLAUDE.md` , `AGENTS.md`)	Facts that must survive compaction belong in `AGENTS.md` , which Codex re-reads at session start and after compaction, instead of in re-read files.	not measured in corpus.	Codex `AGENTS.md` truncates at 32 KiB; Claude Code advice is under 200 lines.	C:106, C:036: Mastering Context Compaction in Codex CLI; Codex Context Management: Auto-Compact and /context (blogs)

Test before enabling: Detector first on matched history. Then A/B with the DevSkills text (A:059 design: held-out tasks, repeats, Pass@1 tracked) and report re-read count, billed cost per success and Pass@1. Test masking window M at 5, 10, 20. Not covered: No corpus implementation of a gateway repeat-read stub or measured cache impact of one. No Kiro or Aider re-read control in corpus.

44. Re-learning the codebase

Each session re-reads the same files before first edit because what earlier sessions learned was never saved.

API detection: Gateway: Read tool_use paths before the first edit, seen in 2+ earlier conversations of the same project (needs project key from caller/header). CLI detection: Transcript/hook Read paths per project across sessions; PostToolUse hook gives tool_input.file_path (C:074).

Technique	How it works	Effect	Loses when	Source
TokenEconomizer experience micro-skills	Learns repository facts (commands, test invocations, edited files) from earlier trajectories and injects them as micro-skills confirmed across non-overlapping tasks.	Tokens down 18.4-30.9%; micro-skill injection alone -11.8% and -14.3% in two Claude Code settings.	Skills go stale; cost rises on first tasks.	C:028: Experience-Driven Black-Box Cost Control for Coding Agents (TokenEconomizer)
Memory tool and notes (memory_20250818)	Agent writes /memories files and views them in later sessions.	Session 2 with memory: peak 172,623 tokens vs 333,977 without (reads 4 of 8 documents). Single synthetic run.	Notes can go out of date; Pokemon example shows over-writing makes recall costly.	A:271, A:269, C:077: Context engineering cookbook; The components of agent memory (Anthropic)

Aider repo map	Tree-sitter extracts symbols, a graph is ranked with a PageRank variant, and the top symbols are emitted within a token budget (default 1k, up to 4k for large repos).	Built in 0.4 s cold; token saving not measured.	Standing charge sent every turn; may omit what the task needs.	C:041, C:127: Claude Code vs Aider benchmarks (C:041); How to Reduce Aider Token Usage (C:127)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Layered `AGENTS.md` discovery with project_doc_max_bytes cap	Codex concatenates global then root-to-cwd `AGENTS.md` files, stopping at 32 KiB default.	not measured	Large files truncated at cap.	C:099
Codex compaction backends (default summarization, remote, remote_v2) and two-phase memory consolidation	Auto-compaction at model_auto_compact_token_limit ; memory phase 1 extracts raw memories, phase 2 consolidates into `MEMORY` .md and summaries.	Not measured.	Reverse-engineered reading of one Codex version.	C:047

Test before enabling: Sequential task stream per repo as in C:028: with vs without memory/micro-skills, tokens before first edit, pass@1 within 3 points. Not covered: Corpus has no per-file exploration cost figure; savings from `CLAUDE.md` alone are not measured.

45. High effort on trivial turns

Reasoning tokens are billed as output; a trivial reply with thousands of reasoning tokens wastes the highest-priced tokens.

API detection: One call, no tools, reply 300 tokens or fewer, output minus reply over 3,000 tokens; cost (reasoning - 1,000) * out_price. CLI detection: Transcript usage per turn plus effort setting; same signature on short chat turns.

Technique	How it works	Effect	Loses when	Source
Effort parameter	output_config.effort low to max governs thinking, text and tool calls; per-message effort preserves the prompt cache.	not measured in corpus	Behavioural signal; run an effort sweep.	A:282, A:200: Effort parameter (Claude); extended thinking tips
Prompt-Induced Waste (effort arm)	Paired comparison of effort, prompt and harness on cost per successful task.	Effort-only arm gave 91% of the full-stack reduction (about 75% under DeepSeek Harness vs about 19% in Claude Code).	Effort can add cost without benefit or help on hard tasks.	C:026: Prompt-Induced Waste in Coding Agents
Overthinking evidence	o1-like models use 1,953% more tokens on 2+3; self-training cuts output 48.6%.	48.6% fewer tokens at maintained accuracy	Needs training; not a gateway fix.	A:089: Do not Think That Much for 2+3=?

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source

GPT-5-Codex adaptive thinking time per task complexity	Model spends less reasoning on simple turns and more on complex ones, rather than a fixed effort.	For the bottom 10% of user turns by generated tokens, uses 93.7% fewer tokens than GPT-5; top 10% reasons about twice as long. OpenAI-reported.	Vendor claim, model-specific; no cost in dollars given.	A:055
Adaptive Length Penalty (`ALP`) RL objective	Adaptive Length Penalty RL. Reward = 1[correct] - beta*N*max(p_solved,1/K), p_solved estimated online from K=16 rollouts per prompt. simple prompts (high p_solved) get strong length penalty, hard prompts weak. Works with GRPO, `RLOO` , Reinforce++.	About 50% fewer tokens; `MATH-500` Pass@1 0.81→0.80 with tokens 2326→646. 40% higher Pass@1 at 1024 budget on simple math. Adaptation ratio 5.35x vs 2.41x base. R1 and QwQ-32B use >10,000 tokens on '2+3'.	Math only; compared against public checkpoints; mixed-difficulty stress test accuracy fell 0.89→0.52.	A:096
Empirical token-consumption analysis and self-prediction of cost on `SWE-bench` Verified	No new algorithm; empirical study. OpenHands on `SWE-bench` Verified (500 tasks x 4 runs) across 8 models, phase analysis (Setup, Explore, Fix, Validate, Closeout), round-level cost decomposition with Sonnet 4.5, and agent self-prediction of its own token use.	Agentic coding ~1000x more tokens than code reasoning and chat; input/output ratio 153.85 vs 1.33 and 0.16; cost $1.857 vs $0.016 and $0.023; Most expensive run ~2x cheapest on average (abstract says up to 30x); accuracy peaks at intermediate cost; Cache reads dominate tokens and dollars in every phase; output ~80x pricier per token than cache reads; Self-prediction correlation up to 0.39; every model underestimates; Kendall tau-b 0.32 vs difficulty	8 models, one framework (OpenHands), one benchmark; self-prediction noisy and underestimates.	C:023
Edit-format and reasoning-effort cost/accuracy benchmark on polyglot tasks	Reports pass rate, edit format and total cost per model across 225 Exercism tasks.	gpt-5 high 88.0% at $29.08 vs medium 86.7% at $17.69; o3-pro high 84.9% at $146.32.	Single benchmark; cost shown at benchmark run time.	C:038
Reasoning-mode energy measurement (CodeCarbon-based benchmark)	Benchmarks energy per query across tasks, adding reasoning on versus off for the same models.	Reasoning models use about 30x more energy on average than non-reasoning or reasoning-off runs.	Energy, not dollars; not agentic coding.	A:127
CoThink: instruct model drafts outline, thinking model expands	CoThink: an instruct model (Qwen2.5-32B-Instruct) writes a 2-4 step outline with no numbers; the thinking model then expands it using only those steps. Defines token efficiency tau=Q/C and relative reasoning efficiency eta=(`CR/CI`)^beta, beta 0.3-0.5.	Tokens cut 21.1% on average vs Solo-Thinking, up to 41.8%; Accuracy +1.66% on average; Win rate 37/60 (61.7%): `GSM8K` 37.5%, `MATH500` 87.5%, `AIME24` 60%	Weak on `GSM8K` (`DAPO` +6.3% tokens, Qwen3 no-think +25.7%); math only; `DAPO` follows outline inconsistently; scaling law speculative; needs two models.	A:135
Dynamic value-thresholding abstention (early termination of reasoning)	Dynamic value-thresholding: add abstention action with reward r_bot; abstain at any token position when estimated value V_t (probability the final answer is correct given the prefix) falls below r_bot. Value estimated by a two-layer `MLP` probe (~10^4 params) on last-layer hidden states, trained with binary cross-entropy over all non-terminal prefixes of completed traces. Threshold T_alpha is the alpha-quantile on held-out data.	OlympiadBench Phi-3 at 90% abstention: 0.64 selective accuracy vs 0.34 best baseline (base 16%); Qwen OlympiadBench at 90% abstention: 0.91 vs 0.75 best fixed-position baseline (base 43%); Retains 63-86% of input-only token savings at 10% abstention, 92-95% at 90%; Threshold hits target abstention rate within 1.2 points `MAE` ; zero-shot probe transfer works across datasets	Needs hidden-state access and a trained probe per model; theory assumes oracle value; experiments on math and toxicity only; saves fewer tokens than input-only abstention at low rates.	A:030
Chain of Unconscious Thought prompt plus token-efficient strategies	Training-free prompt with two parts. `RPI` (Reasoning Process Internalization): solve in hidden thinking, output a bare-minimum single-line reasoning. `TES` (Token-Efficient Strategies): ' `TOKEN CONSERVATION MODE ACTIVE` ', symbols and abbreviations, omit articles, strip greetings, and a point rule: each saved token +1, each accuracy error -100.	Ablation averages: 354.46 tokens vs CoT 676.85 (-47.6%), accuracy 91.19 vs 88.40 (-2.79 points); `RPI` alone -31.1% tokens; `TES` alone -33.9%; Figure 1 claim: -20.51% tokens vs best baseline, 0.1% accuracy drop; Arithmetic sets: 190.19 tokens, 94.28% vs CoT 93.5%	QwQ-32B stays high in tokens whatever the prompt; math benchmarks only; accuracy loss vs CoT in the ablation.	A:090

NoWait (suppress Wait-like self-reflection tokens at inference)	Logit processor at inference sets logits of reflection keywords to a large negative value. 15 seed keywords were mined from 32 QwQ-32B runs on `AIME` 2025 (wait, alternatively, hmm, but, however, check, maybe, verify, again, now) and expanded to all vocabulary tokens containing a keyword as substring. No training or weight change.	QwQ-32B `AMC23` 95.50 (+4.25) at -30% tokens; `AIME25` 68.00 (+1.33) at -31%; Phi4-R+ `AMC23` +6.00 at -28%; Qwen3-32B -13% to -16% tokens; `Kimi-VL` -40% to -60% tokens on some sets (-7.25 accuracy on `EMMA-mini`); QwQ-32B-Preview `AIME24` 42.00 at 5764 tokens vs 8979 baseline; O1-Pruner 33.33	Distilled Qwen3-4B/8B/14B degrade (>5 points `AIME24` , >12 `AIME25`); suits `RL-trained` reasoning models; math-heavy benchmarks.	A:091
Retained reasoning plus native compaction, programmatic tool calling, low reasoning effort, small-tier model (Luna) selection	Persists reasoning across turns with compaction, runs tool orchestration in code outside context, and uses cheaper tiers or lower effort.	`ARC-AGI-3` 13.3% to 38.3% with about 6x fewer output tokens; BrowseComp 84.04% at $1.33 versus 84.36% at $33.27 for GPT-5.5; PlayerZero cost -64%.	Vendor and customer testimonials; harness-specific.	A:207
Shapley-value cost attribution of thinking tokens and turns; price reversal	Cost c = p_in*n_in + p_out*n_out (+ cache terms for agents). Shapley-value attribution of the cost gap between two models over eight multilinear factors (turns, input, output, cache, thinking tokens, and three prices), with a closed form. Cost treated as a distribution with quantile-level reversal analysis.	106 of 336 pairs (32%) show price reversal; 11% on ArenaHard to 57% on `MMLU-Pro` ; Gemini 3 Flash: 80% lower listed price than GPT-5.4 but 38% higher total cost ($705 vs $509); Single-turn: thinking tokens over 95% of the gap; agentic: turns over 80% (Kimi vs GPT-5.4); Repeat-run cost on the same query varies up to 9.7x	Costs only, no quality; one price snapshot (May 2026); cost prediction left open; no limitations section.	A:069
`CGRS` : suppress reflection-trigger tokens when certainty is high	At paragraph breaks, probe a tentative answer by appending a final-answer prefix. Certainty C = 1 - mean token entropy / log|V|. Suppression probability p = max(0,(C-delta)/(1-delta)), delta=0.9. With probability p, trigger-token logits (Wait, But, Alternatively, Hmm and variants) are set negative.	18.5-41.9% token reduction, accuracy drop up to about 3 points. Qwen3-14B: 25.4% fewer tokens, accuracy 80.1 to 80.8; NoThinking cuts 77.3% but loses 17.8% accuracy. `AMC23` ablation: fixed p=1 gives 76.7 acc at 2373 tokens vs certainty-guided 88.3 at 3406 (vanilla 87.5 at 5861).	Needs decoding-time logit access (open models). No overhead analysis found in the text read. Probing runs concurrently with decoding.	A:092
One chat per coherent outcome with fork, compact, subagents and `AGENTS.md`	Plan mode before hard tasks, short `AGENTS.md` , /fork only when work branches, subagents for bounded work, one chat per outcome.	not measured	Qualitative guidance only.	C:098
`THINK-Bench` efficiency metrics (useful tokens, reflection quality)	`THINK-Bench` : 1375 questions; metrics for thinking efficiency, useful tokens and reflection quality over reasoning-model chains.	1375 questions.; Efficiency Grok-3-mini 61.69% vs QwQ-plus 44.58%.	Benchmark only; no mitigation.	A:093
Budget forcing (truncate thinking or append Wait)	Budget forcing: cap thinking by appending the end-of-thinking delimiter (optionally 'Final Answer:'), or lengthen it by suppressing that delimiter and appending 'Wait'. Applied to Qwen2.5-32B-Instruct fine-tuned on s1K (1,000 curated questions from a 59K pool).	`AIME24` 50.0 to 56.7 with budget forcing; `MATH500` 93.0; beats o1-preview by up to 27% (`MATH` , `AIME24`). 59K-full training costs 394 H100 hours vs 7 for s1K with no substantial gain.	Scaling flattens after about six forced Waits and can loop. Context window caps scaling. Token-conditional control fails (model cannot count tokens); rejection sampling shows inverse scaling.	A:131
Per-message effort change without cache invalidation; cache price tiers	Fable 5.1 supports mid-conversation effort changes (beta) and preserved thinking across model switches; cache read $0.25 vs 5m write $12.50 and 1h write $20 per `MTok` .	Cache read is a quarter of the earlier price; input $10 vs Opus 5.5 $4 per `MTok` .	Forced tool use errors; editing earlier turns invalidates thinking blocks.	A:285
Fast mode with separate cache and switch cost	Fast mode is up to 2.5x faster at $8/$40 on Opus 5.5; first enable pays uncached full price and fast and standard keep separate caches.	Up to 2.5x speed at higher per-token cost.	Each switch rereads context uncached.	C:086

Explicit cache breakpoints, cache diagnostics (reason: tools_changed), effort change via configuration_update, prewarm	Stable tool order, allowed_tools and appended developer messages preserve prefix; diagnostics name the miss reason; prewarm at startup; effort change without cache break.	Customers: hit rate 83% to 91%, cache writes down about two-thirds, cost down 36%; another 85% to above 90%; 30-minute discount window.	Customer-quoted figures, vendor-curated.	A:072
Compute-optimal test-time scaling by prompt difficulty	Compute-optimal test-time scaling: estimate prompt difficulty (base pass@1 from 2048 samples, 5 quantile bins, or predicted via verifier scores); per bin and budget choose the best strategy on a validation fold. Strategies: `PRM-guided` search (best-of-N, beam, lookahead) and sequential revisions mixed with parallel sampling.	Up to 4x less test-time compute than best-of-N (16 vs 64 generations search; 64 vs 256 revisions). Beam search better at low budget and hard questions, best-of-N on simple and high budget. About 38% of correct answers flipped to wrong by naive revision. vs about 14x larger model: test-time compute wins on simple/medium or low inference load only.	Math only, one model family, needs fine-tuned revision and verifier models, difficulty-estimation cost excluded.	A:094
Idle-session thinking clearing (clear_thinking_20251015, keep:1) and effort-default / length-limit regression lessons	Clearing old thinking after 1h idle was meant to shrink the uncached resume; a bug repeated it every turn, causing cache misses; a 25-word between-tool limit cost quality.	Prompt ablation showed a 3% eval drop; medium default effort felt less intelligent and was reverted to high/xhigh.	Pruning history or verbosity limits can silently hurt quality and cause cache misses; need per-change evals.	A:054
Length Controlled Policy Optimization (`LCPO`) for reasoning-length control	`LCPO` : prompt adds 'Think for n_gold tokens' with n_gold ~ Uniform(100,4000); GRPO reward. L1-Exact: correctness minus 0.0003*|n_gold - n_y|. L1-Max: correctness times clipped length slack with delta 0.5, trained 120 steps after 700 of Exact.	Beats S1 by 20-25% absolute at 512/1024 tokens. L1-Max matches DeepScaleR-4K with up to 2x fewer tokens. Mean length error about 3% on math (20-40% `OOD`). Short CoT beats GPT-4o by about 2% at equal length.	May not generalize to longer requested lengths; 4K training context; 1.5B math model (7B in appendix); little tuning.	A:095
Model natively trained for compaction across multiple context windows	The model auto-compacts near the window limit and repeats; adds xhigh reasoning effort.	30% fewer thinking tokens at medium effort on `SWE-bench` Verified versus GPT-5.1-Codex.	Vendor claim; Codex-specific.	A:250
opusplan alias, subagent model setting, effort levels, organization effort caps	opusplan uses Opus in plan mode and Sonnet in execution; CLAUDE_CODE_SUBAGENT_MODEL sets subagent model; effort sets adaptive reasoning.	Not measured.	Risk of mismatch with effort after fallback model.	C:073

Test before enabling: Paired bootstrap, McNemar and Holm on cost per successful task, per C:026, across effort levels. Not covered: NoWait (A:091) and `ALP` (A:096) need decoding or training control, not available at a gateway. Codex multipliers (C:105) are community estimates.

46. Tool discovery and skill loading

Repeated ToolSearch calls and skill file reads add tokens; big invoked skill bodies are charged until compaction.

API detection: Gateway signal only: tool_use names for ToolSearch above 2 per conversation key. CLI detection: Hook events: ToolSearch calls beyond 2 per session; Read of /skills/ paths not ending `SKILL` .md; invoked skill bodies above 3k tokens.

Technique	How it works	Effect	Loses when	Source

skillOverrides and disable-model-invocation	States on, name-only, user-invocable-only, off; disable-model-invocation keeps a skill out of context until /name.	Not measured in corpus.	Model can no longer auto-invoke.	C:075, C:079: Claude Code, Extend Claude with skills; Anthropic, Explore the context window
Skill re-attach budget	After compaction invoked skills return at 5,000 tokens each, 25,000 total, most recent first.	Not measured in corpus.	Large skills consume the budget.	C:075, C:079: Claude Code, Extend Claude with skills
On-demand schema access	get_tool_schema/invoke_tool proxy keeps a stable wrapper, cache-friendly.	See row 1: 17,600 to 500 tokens.	Namespacing prefixes.	C:022: Atlassian, MCP Compression

Test before enabling: Count ToolSearch and skill reads per session before and after trimming; guard on task success. Not covered: No corpus source for the A:65 ToolSearch threshold; DevX proposal.

47. Usage while idle

Prompts fired with no human hook activity (loops, scheduled reruns) keep spending tokens while nobody is working.

API detection: Gateway sees only timing and regularity: fixed-interval or off-hours calls with identical prefix. A138 flags cross-session scheduled reruns. CLI detection: Hook data: prompts with no hook record within 15 seconds, counted only after the first hook record.

Technique	How it works	Effect	Loses when	Source
Idle compaction	After idle on the one-hour TTL, Claude Code can compact the conversation; idleCompaction false turns it off.	not measured in corpus	Compaction rewrites the prefix and costs a cache miss.	C:055: How Claude Code uses prompt caching
Handoff file	A handoff file and two prompts move a task between threads without re-planning.	not measured in corpus	Manual discipline.	C:035: Context Management Across Codex Surfaces
Budget alerts	Alerts at 75%, 90% and 100% of a set budget.	not measured in corpus	Alerts, not enforcement; Copilot premium-request scope.	A:213: Overview of request-based billing (legacy)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Session `AI-credit` cap (/limits set, --max-ai-credits)	A soft cap on credits per session (min 30; 1 credit = $0.01) stops the agent cleanly at the limit and lets you raise it.	not measured	Soft limit: in-progress response completes so spend can exceed it; preview feature.	A:153
Code Review run-frequency setting (once after creation vs every push vs manual)	Cost multiplies by pushes under After every push; choose Once or Manual and track Average cost per repo.	not measured	Default for new repos is the costliest setting.	C:082
Premium request model multipliers (code review = 13)	Each prompt uses one premium request times the model's rate; Copilot code review consumes 13 from June 1 2026.	not measured	Applies to legacy annual plans only.	C:117

Idle-session thinking clearing (clear_thinking_20251015, keep:1) and effort-default / length-limit regression lessons	Clearing old thinking after 1h idle was meant to shrink the uncached resume; a bug repeated it every turn, causing cache misses; a 25-word between-tool limit cost quality.	Prompt ablation showed a 3% eval drop; medium default effort felt less intelligent and was reverted to high/xhigh.	Pruning history or verbosity limits can silently hurt quality and cause cache misses; need per-change evals.	A:054

Test before enabling: Backtest the 15-second no-hook rule on recorded sessions and review flagged prompts manually before alerting. Not covered: no corpus source for detecting idle usage at a gateway; A:053 and hook docs not opened. DevX proposal: regularity (fixed-interval) detector.

48. MCP tool schema bloat

Every unused mcp__ tool definition is re-read each call; median session carries about 91k tokens of MCP definitions.

API detection: Hash and size of tools[] by namespace prefix; flag tools never called. CLI detection: prompt_snapshot.tools[] versus tool_use calls per server; /context.

Technique	How it works	Effect	Loses when	Source
mcp-compressor	MCP proxy replacing tools with get_tool_schema/invoke_tool.	94-tool server: 17,600 to 500 tokens at max.	Quality depends on on-demand schema access.	C:022: Atlassian, MCP Compression
Server disable and Codex tool allow/deny lists	mcp_servers.<id>.enabled, enabled_tools, disabled_tools applied after enabled_tools; omit tools from direct, deferred or code_mode surfaces.	GitHub MCP with 93 tools is about 55,000 tokens per turn.	Loses the disabled tools.	C:104, C:107: Codex configuration reference; Vaughan, Codex CLI Performance Optimisation
Code execution mode and mcp2cli	Code-mode tool calls and CLI list/help lookups at about 16 and 120 tokens per tool.	Up to 98.7% reduction reported; 96-99% under the source's "token savings measured" label.	Simulation or vendor claim.	C:019: bswen, How MCP Tool Definitions Inflate Your `AI` Agent Token Costs
Tool consolidation and curation	Consolidate and namespace tools; minimal viable tool set.	Not measured in corpus.	Redesign effort.	A:284, A:084: Anthropic, Writing effective tools for `AI` agents; Effective context engineering

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Total-cost-of-ownership framing: tokens per task, calls per action, retries, routing	Argues cost per task depends on tokens consumed, model calls, routing, context size and retries, not token price.	Cites Gartner `AI` spend $2.52 trillion in 2026, +44%.	Vendor marketing; no technique numbers.	A:180

Test before enabling: Per server, compare calls versus tokens; disable one server at a time on paired tasks and watch success. Not covered: Vercel server 212 tools/140k tokens has no specific corpus fix.

49. Unused skills/plugins

Skill and agent listings are injected every session even when never invoked.

API detection: Gateway signal only: system prompt skill_listing text never matched by a Skill tool_use. CLI detection: skill_listing and agent_listing_delta entries versus Skill invocations.

Technique	How it works	Effect	Loses when	Source
skillOverrides states and disableBundledSkills	Skills can be on, name-only, user-invocable-only or off; description plus when_to_use is truncated at 1,536 characters.	Example listing costs about 450 tokens.	Skill no longer auto-triggers.	C:075, C:079: Claude Code, Extend Claude with skills
/doctor prompt-audit and `CLAUDE.md` linting	Audit instructions; `CLAUDE.md` under 200 lines; path-scoped rules load only on matching reads; claudeMdExcludes for monorepos.	Not measured in corpus.	Imports do not reduce cost.	C:077: Claude Code, How Claude remembers your project
Plugin skills are appended	Plugin skills, commands, agents and hooks never invalidate the cache.	Not measured in corpus.	Still cost tokens each call.	C:055: Claude Code docs, How Claude Code uses prompt caching

Test before enabling: Count invocations per skill over 30 days; retire those at zero and track starting tokens. Not covered: Hermes' $15.60 is small; no corpus measurement of skill-listing cost beyond the 450-token example.

50. No visibility

Sessions with no hook or telemetry data leave categories, ratings and idle time unknown; a coverage gap, not a cost.

API detection: Gateway: share of requests with no caller key, session or project tag header; share of traffic that bypasses the gateway. CLI detection: Share of gateway sessions (x-claude-code-session-id) with no matching hook or `OTel` rows.

Technique	How it works	Effect	Loses when	Source
Claude Code OpenTelemetry	Telemetry flag plus exporters emits token and cost metrics and events tagged with session.id (OTEL_METRICS_INCLUDE_SESSION_ID); admin can lock exporters in managed settings; prompt text is redacted unless OTEL_LOG_USER_PROMPTS=1.	coverage; no saving	A developer can still set selectors to console or none unless managed settings lock them.	A:217, C:002: Monitoring usage (Claude Code OpenTelemetry) (Anthropic)
Gateway tagging with session header	Gateway on the request path records tokens/cost per request tagged to developer and team; Claude Code sends x-claude-code-session-id on every request; headers prefixed x-bf-lh- are copied to log metadata.	coverage; no saving	Needs the developer to route via the gateway (ANTHROPIC_BASE_URL).	C:001, C:088: Claude Code Token Usage: Monitor It with an `AI` Gateway; Run Claude Code through a gateway

Managed hooks and Langfuse coding-agent tracing	allowManagedHooksOnly and managed hooks cannot be disabled by user settings; Langfuse traces every agent LLM call via hooks or a gateway, per team, repository and model.	coverage; no saving	Hooks need harness support; unmanaged hooks can be disabled.	C:074, A:234: Hooks reference (Anthropic); Track and optimize coding agent usage (Langfuse)

More techniques from the source coverage pass. Each is one source's claim and is untested here.

Technique	How it works	Effect	Loses when	Source
Dollar spend-limit rules scoped by provider, model or metadata, 429 on breach	Each rule gives a rolling or fixed-window dollar budget, split or filtered by dimension such as agent_id; blocks with 429.	not measured	Eventually consistent: concurrent bursts can briefly exceed limit.	A:191
Cost or token budget limits with alert thresholds and periodic reset	Per-key USD or token caps that expire the key when reached; alert thresholds; weekly or monthly reset.	not measured	Enterprise plan only; not retroactive; not editable once set.	A:242
LiteLLM budget caps, Langfuse, `LLMLingua`, RouteLLM, `GPTCache` semantic cache	Roundup: LiteLLM max_budget per key; `LLMLingua` prompt compression; RouteLLM routes between two models by threshold; `GPTCache` semantic caching.	RouteLLM 85% cost reduction keeping 95% GPT-4 quality on `MT` Bench (45% MMLU, 35% `GSM8K`); LongLLMLingua +21.4% RAG with 1/4 tokens.	Vendor blog citing others' numbers; RouteLLM is binary routing only.	C:061
Status line script receiving JSON with context percentage and cost	Shell script gets JSON on stdin including context_window.used_percentage, cost.total_cost_usd and duration fields.	not measured	Cost is client-side list-price estimate, may differ from bill.	C:076
Decoupled controller model approves extra token budget from workspace snapshot	Worker freezes at a 50,000-token limit; a separate controller model scores approve-vs-deny with expected value p>1/2 using task, files, tests snapshot.	Injected budget counter referenced 0 times in 14,000+ messages; request_more_budget used 0 of 5,000 turns; approval 97% default, 79% neutral.	Controller models biased to approve (four of six); 100-task evaluation set.	C:136
Per-developer tier-fit recommendation (upgrade anytime, downgrade only last 5 days of month)	Ingests daily per-user reports, accumulates month-to-date credits, and recommends the cheapest tier given base plus $0.04/credit overage.	Example: a Pro user at 3,000 credits pays $100, so a higher tier saves $80/mo.	Kiro-specific seat pricing; downgrade logic waits until month end.	A:056
Dedupe usage by message `ID`; cost fields are client-side estimates	Parallel tool calls share one message `ID`, so count each once; read output tokens and totals from the result message.	not measured	Estimates drift from billing; per-step output_tokens is a placeholder; subagent usage unrecoverable after a crash.	A:216
Four-layer cost management: track, attribute, alert, then optimize	Compute cost per request from token buckets, attribute by user/session/feature/prompt version, alert, and enforce budgets at a gateway.	not measured	Names pitfalls: double-recorded calls, overlapping cached-token buckets, unmatched model names.	A:233
--bare mode and --max-budget-usd cap for claude -p	--bare skips hooks, skills, plugins, MCP servers, auto memory and `CLAUDE.md` for faster startup; budget cap stops background work.	not measured (startup time claim only)	Bare mode drops project context and `OAuth`; only `API-key` auth.	C:094
Premium request multipliers and monthly allowance	Premium models consume allowance with multipliers; overages billed per request; budgets can be set.	Reasoning models may consume 5x or 20x the standard rate.	Legacy billing for annual plans only after June 2026.	C:116
Gateway via ANTHROPIC_BASE_URL with managed allowedProviders lock	Route Claude Code through an org gateway for credentials, usage attribution, budgets, rate limits and audit logs; lock via managed settings.	not measured	Gateway must be kept updated to forward new Claude Code features.	A:193

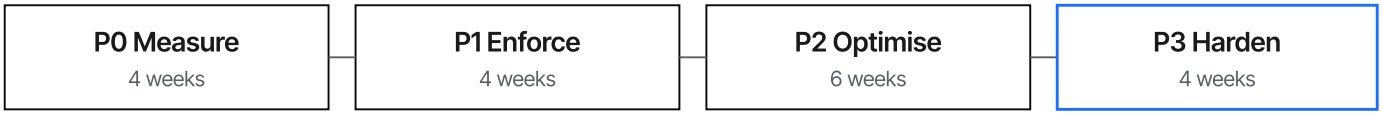
Application-specific capability profiling testbed (cost-aware model selection per application)	Eight-step profiling method, Stage 1 built as testbed: sparse versioned model profile where missing never equals zero; comparability only for same attribute, context and dataset; evidence levels L0-L4; Pareto set with evidence counts; plan gate with `SHA-256` plan hash and baseline arm; null-agent check; range noise rule with three-state verdict; fail-closed reporting. Constrained model-to-task-part selection (Eq. 11) is planned, not built.	Migration fully-correct 98 to 108 of 216 (45.4 to 50.0%, Fisher p=0.39); accepted-but-wrong or unverifiable 9.7% to 21.9% (p=0.023); about 78% of ok verdicts proved nothing. Breadth: agent 91.9% vs cheap 90.5% (p=0.21); agent used 3.0% fewer tokens. Automation: model effect p=0.007, framework p=0.40; tokens 1,953-9,465; CrewAI about 4x. Judge churn 16.2% at T=0; full run $3-4.	Stage 1 only; model-vs-framework comparisons are non-causal separate sweeps. Migration inference unpaired, no multiplicity correction. Only P1 and chatbot integrated as probes; Grafana JSON never imported. Adapter-generation hook unevaluated. 15 automation samples unreadable. Caller key crosses a process boundary unprotected. Range rule loses power as repeats grow.	A:047
Copilot CLI background auto-compaction at ~80% (dynamic to 90%) plus /context, /usage, /compact	Compresses history in background near 80% of token limit, waiting until 90% when static context is 75% or more; /compact manual, /usage and /context for visibility.	Threshold behavior only; no savings measured.	Compaction reclaims less when static context is large.	A:154
OpenRouter provider routing (effective price with real cache hit rate, sort, max_price, :floor/:nitro, fallbacks)	Picks a provider per request by measured effective price reflecting its cache hit rate, throughput, latency and reliability; supports sort, max_price and fallbacks.	Not measured; reacts to provider changes within minutes.	Provider choice not model choice; bypassed when order or own keys set.	A:227
Rule-first classifier plus decision model with probability threshold (0.85) routing fast versus frontier tier, per-tier API key budgets	Explicit rules first, then a decision model picks tier; fast tier accepted only if probability >= 0.85, else frontier; `AI` Gateway fallbacks and spend caps per tier.	Not measured; threshold 0.85 given as an example.	Experimental Decision API; threshold needs tuning.	A:245
Cache-aware cost comparison with TCO model and hybrid routing counterfactual replay	Longitudinal case study: Langfuse and vLLM telemetry plus Git mining. Fix Commit Ratio via regex over commit messages; difficulty-stratified Mantel-Haenszel odds ratio; TCO = M x (compute + ops + T_iter x wage); hybrid-routing offline counterfactual replay by difficulty tier.	99.3% cache hit cut API cost 88.6% to $0.57/M tokens vs $2.83/M shared on-prem; `FCR` 74.9% vs 45.9%; Mantel-Haenszel `OR` 3.61; Shared on-prem saves 40.1% true TCO; dedicated costs 43.8% more; High-to-Claude hybrid saves 26.3%	One developer, non-randomized, tasks differ between periods; keyword-classified fixes; Period A fix count inflated by CI work; replay assumes tier exchangeability.	C:053
Message Batches API and Token Counting API	Batches run asynchronously at a 50 percent cost reduction; `POST` /v1/messages/count_tokens estimates tokens before sending.	50% batch discount.	Page is mainly an index; no usage detail.	A:220
Gateway failover and caching:'auto' with cache-invalidation warning	Gateway reissues failed requests to another provider and applies provider-specific cache markers; warns abstraction layers can silently invalidate cache hits.	3.5% of requests succeed only through failover, carrying 4.9% of spend; zero token markup.	A documented AutoGPT bug put a per-call random token in the cached prefix, voiding the 90% discount.	A:246
`AI-credit` billing with user, cost-center and organization budgets and hard stops	Tokens priced per model convert to `AI` credits (1 credit = $0.01); budgets cap consumption, with user budgets always a hard stop.	Not measured; a $10 budget covers 1,000 credits.	Hard stop can block users mid-task.	A:263
Longitudinal gateway-log cost analysis of nine coding agents	Computes cost per active user and cached_tokens/input_tokens across a year of gateway traffic.	$92/month per active user, Claude Code $108 (P95 $291); 86% cache hit overall, 92% Claude Code; input per call grew 50K to 200K.	Single gateway's users; correlational.	C:054
Cursor Router modes (Cost, Balance, Intelligence) and Cursor Token Rate	Router picks Cursor or third-party models per request by mode; third-party use is billed at API list price from prepaid credit.	Max Mode on legacy plans costs API rate plus 20%.	Pricing doc, not measured.	C:119
system.billing.usage `SQL` cost queries with tags	Query billing usage system table by sku_name and custom tags to aggregate `DBUs` per day.	not measured	Databricks model serving only.	A:188

cached_tokens usage field for verifying cache hits	Read usage.prompt_tokens_details.cached_tokens: 0 is a miss, otherwise tokens reused; multi-turn shows it rising each turn.	Example 98 of 125 prompt tokens cached.	Verification only; no pricing figure read.	A:221
Dynamic route flow with conditional, percentage, rate-limit and budget-limit nodes	Versioned `JSON/visual` route sends segments to different models and falls back when a key's budget or rate quota is exceeded.	not measured	Gateway configuration only.	A:189
Session-id cost attribution, custom property segmentation, graduated budget alerts (50/80/95%), response cache	Headers tag requests by session and property; alerts at 50%, 80%, 95% of budget; Cache-Control max-age enables response caching.	Example dashboard: 73% hit rate saving $1,247 a month (illustrative).	Gateway-dependent; example figures.	A:239
Automatic token caching for `API-key` and Vertex users; /stats cached-token display	CLI reuses system instructions and context under API key or Vertex; `OAuth` users get no caching.	not measured	Unavailable for `OAuth` personal accounts; CLI later replaced.	C:121
Per-use-case token tracking schema with hub-and-spoke `AI` proxy and per-use-case keys	Centralized proxy where each use case has its own key and logs input/output tokens to a tracking table, versus decentralized `SDK-based` tracking.	Not measured.	Misattributes cost if hub logging is poorly configured; decentralized attribution is harder.	A:125
cf-aig-custom-cost header for negotiated input/output/cache rates	Request-level header overrides per-token input, output, cache-read and cache-write prices so gateway cost logs match negotiated rates.	Example: 1,000 input with 600 cache-read and 200 cache-write leaves 200 fresh tokens costed separately.	Needs token counts in responses; cache hits at the gateway always cost 0.	A:190
/usage breakdown and ENABLE_PROMPT_CACHING_1H	/usage shows parallel sessions, subagents, cache misses, long context as percent of last 24 hours; PreCompact hooks can block compaction; env var opts into 1h TTL.	Not measured.	Release notes only.	C:092
Credit pool with user limits and workspace overage limits	Work draws from a shared credit pool; alerts do not stop spending, overage limits do.	Not measured.	Estimates by role are not fixed prices.	C:101

Test before enabling: Coverage audit: ratio of sessions with hook/ `OTel` rows to gateway sessions, per harness and version; target set from the P0 baseline. Not covered: Corpus does not say what coverage share is adequate.

Mechanism, measured effect, where each technique loses and the test design for every row are in `plan/12a-fix-technique-detail.md` , and open under each row above.

Build order and exit gates



The four estimates sum to 18 weeks for three engineers. They are estimates, not findings from the sources.

Phase	Ship	Exit gate
P0 Measure	Gateway in observe mode, ledger, price book, Claude Code and SDK adapters, then Codex CLI. Four spikes: LiteLLM as gateway core, Claude Code against a 402, cache loss on a model switch, subagent multiplier.	Computed cost within 1 percent of the vendor report for two weeks per provider. A 30 day baseline of median and P95 spend per developer. Spike results written down.
P1 Enforce	Budgets, alerts, turn cap, loop guard, backend fallback, reconciler, dashboard.	A test key cannot exceed its budget by more than the measured overshoot. Fault injection never double-counts or loses a row.
P2 Optimise	Eval harness first. Then cache planner, context guard, router, shadow runs.	Each shipped optimisation has a paired result with the success bound held. One that fails stays off, and that counts as a pass.
P3 Harden	Canary, kill-switches, secrets, multi-tenant limits, audit log, runbooks.	A rollback drill reverts a bad optimisation without a deploy. A load test holds the latency budget set in P0.

Definition of done

Phase	Rows covered	Done when
P0 Measure	14 alert or recommendation rows: 7, 9, 10, 22, 26, 35, 38, 41, 44, 46 to 50	All 50 detectors run in alert mode. Cost within 1 percent of the vendor report for two weeks. A 30 day baseline exists. Every row marked proposal has a spike or a reason it waits.
P1 Enforce	3 guard rows: 15, 36, 42	Each guard leaves task success unchanged on the replay set. Budget overshoot measured. Fault injection never double-counts or loses a row.
P2 Optimise	33 behaviour-change rows: 1 to 6, 8, 11 to 14, 16 to 21, 23 to 25, 27 to 34, 37, 39, 40, 43, 45	Every candidate row has a result file. A row ships for a harness and version only after a paired win in cost per successful task. A row with no win stays off and is listed.
P3 Harden	No new rows. Wraps every shipped P2 fix	Each shipped fix has a canary, a kill-switch and an auto-rollback drill. Load test holds the latency budget. Security review closed. A named owner for each alert.

The count of shipped P2 rows is a result, not a target. Per-deliverable evidence is in `plan/PHASES.md`.

Keep these out until they are measured: agent teams and voting, output compressors, mid-session model switching, bandit routing and self-hosting.

What to take care of

Trap	Rule
Observation fails	Fail open for observation so Token Economics never blocks work. Fail closed for budgets so a store outage cannot become unlimited spend.
Double counting	One ledger row per response id. A retry or a stream cut must not add or lose a row.
Cold cache on fallback	Switching provider or model starts a cold cache. Price that before routing.
Cache busting by the gateway	A proxy that strips `cache_control` silently multiplies cost. Test it and alert on per-caller hit rate.
Stale prices	Block billing on price rows past expiry or older than 30 days.
Prompt storage	Store metadata only by default. Bodies are opt-in.
Cache isolation	Never share a cache key across tenants.
Savings that do not transfer	One compressor raised cost 48.4 percent on one harness and cut it 12.49 percent on another. Test per harness.

What the evidence does not tell you

Model readers covered all 423 sources in three passes. 86 are cited in the fix table with the passage read for each row. 112 papers were re-read in full text, each read through the main text, though some had no bibliography or appendix read. C:043 was read from a PDF you supplied after SSRN blocked the download.

The other 225 vendor docs, blogs and pages were read at abstract and key-section depth, and 14 had too little text to assess. Nobody read every source cover to cover. One reader once fetched an arXiv PDF without permission and deleted it; the six abstract-only papers, that one included, were later downloaded with your approval and read. Codex finished 24 of its 93 notes before it hit a usage limit, and Claude readers took the rest. Many figures come from vendors describing their own products. Items with no source in the corpus are labelled DevX proposal in the plan.

Unknown	Settle it in
How Claude Code behaves on a 402	P0 spike
Cache loss on a model switch	P0 spike
A measured subagent multiplier (sources quote 3 to 5x and 7x, and disagree)	P0 spike
Reconciliation tolerance between ledger and invoice	P0

Soft-budget overshoot size	P1
Anomaly-detection method, power formulas for the eval	P1 and P2 pilot
Any production shadow or canary result	P3

Where the detail lives

File in `plan/`	Read it for
`README.md`	Reading order and source id rules (`A:nnn` API corpus, `C:nnn` CLI corpus)
`01-architecture.md` to `08-security-reliability.md`	One file per concern, each rule tied to sources
`11-detect-measure-apply.md`	Detectors, fix classes, the gate, and the hermes-engine metric map with open-source options
`12b-source-coverage.md`	All 423 sources: what each gives the 50 rows, new technique names, read depth
`12-fix-table-50-rows.md` , `12a-fix-technique-detail.md`	The 50 problems: fix for API agents and CLI agents, named techniques, mechanism, limits and test design
`PHASES.md`	Build lists, exit gates and definition of done for P0 to P3
`10-requirements-traceability.md`	233 requirements with sources, components and phase
`token-economics-lld.html` , `token-economics-lld.pdf`	Drill-down to component spec cards

Start with P0: stand up the gateway in observe mode and write the first ledger row.